Data Science Team Training

Stephen D. Turner, Ph.D.

Contents

Preface

Public health has always been a data-driven enterprise, but the tools available to practitioners have changed dramatically. Spreadsheets that once required weeks of manual work can now be analyzed in seconds. Reports that used to mean emailing static PDFs can now execute code, update automatically when data changes, and be produced in multiple formats from a single source file. And workflows that depended on one person's institutional knowledge can be documented, versioned, and shared across teams. The goal of this book is to help public health professionals take advantage of these tools.

This book grew out of the CSTE Data Science Team Training (DSTT) program, I serve as a project coach working with public health agencies across the country to build data science capacity in the public health workforce. The material here includes code, resources, workshop notes, and practical guidance accumulated and refined over several years of coaching teams at local and state health departments.

The chapters cover the foundational practices that make data science work sustainable and collaborative in a public health setting. Technical chapters address organizing and validating data, connecting to and querying relational databases, pulling data from public APIs and open data portals, migrating legacy SAS and Excel workflows, writing clean and well-documented code, managing reproducible environments and package dependencies, building R packages to share functions across projects, producing accessible reproducible reports and dashboards, automating and scheduling recurring work, and working effectively with AI coding assistants. Nontechnical chapters cover building and sustaining a data science team, project management, peer review of analytical work, navigating the data governance and IT relationships that shape what public health teams can actually do with their data, and communicating findings clearly to audiences who did not run the analysis. These are the practical skills that separate ad hoc analyses from reproducible, maintainable work.

While this material was created with DSTT participants in mind, it is intended to be broadly useful to anyone working at the intersection of data science and public health, whether you are a current or former DSTT participant, a public health practitioner looking to strengthen your analytical skills, or someone new to data science in a public health context.

No single book can cover everything, and this one does not try to. Appendix A. points to additional reading and training for topics that go deeper than what is covered here.

Cite this book:

Turner, Stephen D. (2026). *Data Science Team Training*. Retrieved from https://dstt.stephenturner.us/.

```
@book{
  title = {Data {{Science Team Training}}},
  author = {Turner, Stephen D.},
  date = {2026},
  url = {https://dstt.stephenturner.us/}
}
```

I Technical

1. Version Control

Git is a distributed version control system that tracks changes to files over time, allowing you to review history, revert mistakes, and collaborate without overwriting each other's work. For data science teams, version control is essential for reproducibility (knowing exactly which code produced which results), collaboration (multiple people working on the same project safely), and auditability (a clear record of what changed and why). GitHub is the most widely used platform for hosting Git repositories and adds features like pull requests, code review, and issue tracking on top of Git's core capabilities.

1.1 GitHub Setup

1.1.1 Creating a GitHub Account

Visit github.com and sign up for a free account. The free tier includes unlimited public and private repositories and is sufficient for most team workflows.

1.1.2 Adding an SSH Key

SSH keys let you authenticate with GitHub without entering your username and password on every push or pull. To generate a key:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

Accept the default file location (`~/.ssh/id_ed25519`) and optionally set a passphrase. Then copy the public key:

```
cat ~/.ssh/id_ed25519.pub
```

In GitHub, go to **Settings → SSH and GPG keys → New SSH key**, paste the output, give it a descriptive title (e.g., your machine name), and save.

Verify the connection works:

```
ssh -T git@github.com
```

You should see a message like `Hi username! You've successfully authenticated`.

1.1.3 Cloning a Repository via SSH

When cloning a repository, prefer the SSH URL over HTTPS. On any GitHub repository page, click **Code** and select the **SSH** tab to get the URL.

```
git clone git@github.com:org/repo.git
```

Using SSH avoids repeated password prompts and works with SSH agent forwarding for server-based workflows.

1.1.4 Pulling and Pushing

After cloning, your local repository is linked to the remote (called `origin` by default). To sync your local branch with the latest changes from the remote:

```
git pull
```

After committing changes locally, push them to the remote:

```
git push
```

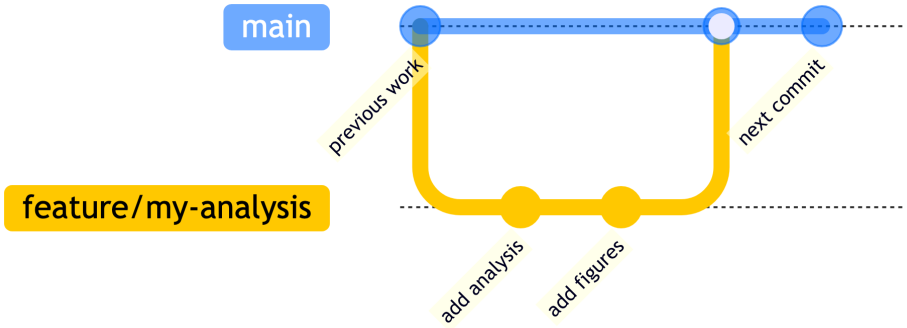
Git tracks the relationship between your local branch and the corresponding remote branch automatically, so these short-form commands will work once the tracking relationship is established.

1.2 Branching and Merging

Branches let multiple people work on different features or fixes simultaneously without interfering with each other. The `main` branch (or `master` in older repositories) typically represents the stable, production-ready state of the project.

1.2.1 Internal Organizational Workflow

In most team workflows, the `main` branch is protected — direct pushes are disabled and changes must go through a pull request (PR) with at least one reviewer. Developers create short-lived branches for each piece of work, then open a PR when ready.



Step 1: Create a branch from `main`

```
git checkout main
git pull
git checkout -b feature/my-analysis
```

Step 2: Make changes, stage, and commit

```
git add analysis.R writeup.qmd
git commit -m "Add initial exploratory analysis for Q1 data"
```

Step 3: Push the branch to GitHub

```
git push -u origin feature/my-analysis
```

The `-u` flag sets the upstream tracking branch so future `git push` and `git pull` commands work without specifying the remote and branch name.

Step 4: Open a pull request

On GitHub, navigate to the repository. A banner will appear prompting you to open a PR from your recently pushed branch. Click **Compare & pull request**, add a description, assign reviewers, and submit.

Step 5: Review, merge, and delete

A reviewer approves the PR and merges it into `main` via the GitHub UI. After merging, delete the branch on GitHub (there is a button on the merged PR page). Locally, clean up with:

> **i** Note
>
> Code review is easier when diffs contain only meaningful changes. If one contributor reformats a file by hand and another does not, a PR's diff may be dominated by whitespace changes that obscure the actual logic. Using a formatter like Air (see Section 15.2) on every save means diffs reflect intent, not style preferences.

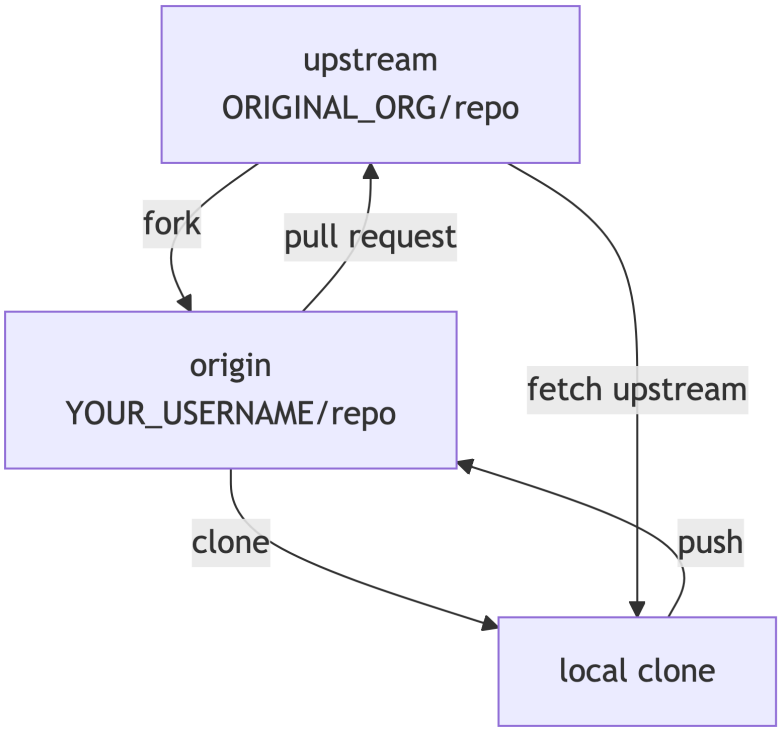
```
git checkout main
git pull
git branch -d feature/my-analysis
```

> **Tip**
>
> **Branch naming conventions** help the team understand what a branch is for at a glance. Common prefixes:
>
> - `feature/` — new functionality (e.g., `feature/survival-analysis`)
> - `fix/` — bug fixes (e.g., `fix/date-parsing-error`)
> - `analysis/` — exploratory or one-off analyses (e.g., `analysis/q2-cohort`)
> - `docs/` — documentation updates

> **Note**
>
> Keep branches short-lived — ideally merged within a few days. Long-running branches diverge significantly from `main`, making merges painful and increasing the likelihood of conflicts.

1.2.2 Contributing to External Repositories

When contributing to a repository you don't have write access to — such as an open-source tool or a public project from another team — the fork workflow is used instead. A fork is a personal copy of the repository under your GitHub account.



Step 1: Fork the repository

On the GitHub repository page, click **Fork** (top right) and choose your account as the destination.

Step 2: Clone your fork

```
git clone git@github.com:YOUR_USERNAME/repo.git
cd repo
```

Step 3: Add the original repository as `upstream`

```
git remote add upstream git@github.com:ORIGINAL_ORG/repo.git
```

Step 4: Branch, work, commit, and push to your fork

```
git checkout -b fix/typo-in-readme
# ... make changes ...
git add README.md
git commit -m "Fix typo in installation section"
git push -u origin fix/typo-in-readme
```

Step 5: Open a pull request to the upstream repository

On your fork's GitHub page, click **Contribute → Open pull request**. This creates a PR from your fork's branch into the original repository's `main` branch.

Step 6: Keep your fork in sync with upstream

As the original repository receives new commits, your fork will fall behind. Sync it with:

```
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

> **Note**
>
> In the fork workflow, `origin` refers to **your fork** and `upstream` refers to the **original repository**. Keeping these straight avoids accidentally pushing to or pulling from the wrong remote.

1.3 Git in Positron

Positron includes a built-in Source Control panel (the branching icon in the left sidebar, or `Ctrl+Shift+G` / `Cmd+Shift+G`) that provides a graphical interface for the most common Git operations.

Viewing changes

The Source Control panel lists all files with uncommitted changes. Files are grouped into **Staged Changes** and **Changes** (unstaged). Click any file to open a diff view showing exactly what was added or removed.

Staging files

Hover over a file and click the `+` icon to stage it, or click the `+` next to the **Changes** heading to stage all modified files at once. To unstage, click the `-` icon next to a staged file.

Committing

Type a commit message in the text box at the top of the Source Control panel and click the **Commit** button (or press `Ctrl+Enter` / `Cmd+Enter`). This is equivalent to `git commit -m "your message"`.

Branching

The current branch name appears in the status bar at the bottom of the window. Click it to open the branch picker, where you can switch to an existing branch or create a new one. This is equivalent to `git checkout` or `git checkout -b`.

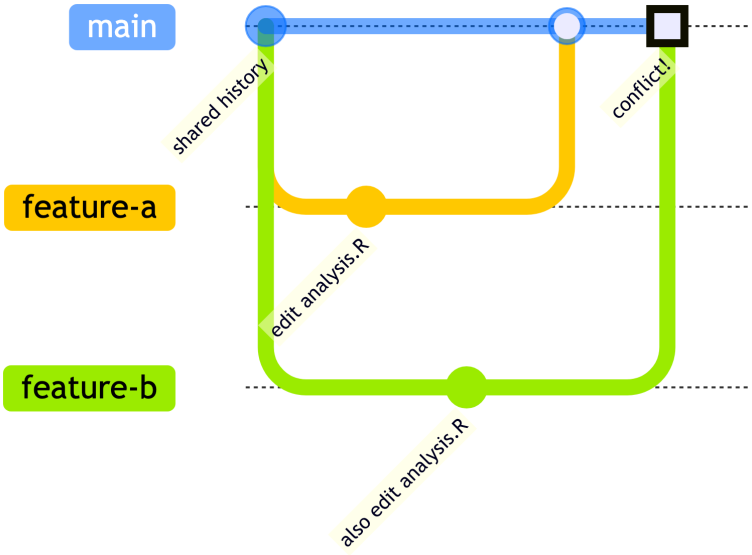
Pushing and pulling

The Source Control panel has **Push** and **Pull** buttons in its toolbar (the `...` menu or the sync icon in the status bar). The sync button performs a pull followed by a push in one action.

> **Tip**
>
> The integrated terminal in Positron (**Terminal → New Terminal**) is always available for Git operations that the UI doesn't expose — such as adding a remote, rebasing, cherry-picking, or any command requiring flags. The UI and terminal work on the same repository state, so you can mix both freely.

1.4 Merge Conflicts

A merge conflict occurs when two branches have made changes to the same lines of the same file, and Git cannot automatically determine which version to keep. Conflicts most commonly arise during `git merge`, `git rebase`, or when accepting a pull request that conflicts with recent changes to `main`.



1.4.1 What conflict markers look like

When a conflict occurs, Git edits the affected file to mark the conflicting regions:

```
<<<<<<< HEAD
flu_clean <- flu |> filter(cases > 0, !is.na(county))
=======
flu_clean <- flu |> filter(!is.na(county), cases >= 1)
>>>>>>> feature/update-filter-logic
```

- Everything between `<<<<<<< HEAD` and `=======` is the version from your current branch.
- Everything between `=======` and `>>>>>>>` is the version from the branch being merged in.

1.4.2 Resolving conflicts

1. Run `git merge <branch>` (or let a PR trigger the conflict). Git will report which files have conflicts.
2. Open each conflicted file. Positron highlights conflict regions with inline buttons: **Accept Current Change**, **Accept Incoming Change**, **Accept Both Changes**, or **Compare Changes**. Click the appropriate option or edit the file manually to produce the correct final version.
3. After resolving all conflicts in a file, save it and stage it:

```
git add path/to/resolved-file.R
```

4. Once all conflicted files are resolved and staged, complete the merge:

```
git commit
```

Git will pre-populate a commit message describing the merge; you can accept it as-is.

> ⚠ Warning
>
> Never leave conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`) in committed code. The file will be syntactically broken and the code will not run. Always verify the conflict is fully resolved before staging.

Tip

`git status` is your best friend during a conflict. It lists which files are in conflict (`both modified`), which are already resolved and staged, and what step to take next (commit or continue a rebase).

```
git status
```

1.5 .gitignore

A `.gitignore` file tells Git which files and directories to leave untracked. Anything listed there will not show up in `git status`, will not be staged by `git add .`, and will never be committed. Every project should have one.

1.5.1 Why it matters for data science

Data science projects routinely contain files that should never go into version control:

- **Raw and processed data.** Large files bloat a repository and slow every operation. Sensitive data files may have legal or compliance restrictions on where they can be stored. The project directory structure in Section 2.3 places a `.gitignore` directly inside `data/` and `output/` to block these files at the source.
- **Credentials and secrets.** `.env` files, config files with API keys or database passwords, and personal config files with paths to secure shared drives (see Section 2.3.2) should never be committed. Once a secret is in Git history, it is effectively public even if you delete it later.
- **Derived outputs.** Figures, rendered reports, and cached results can be regenerated from code. Committing them creates noise in diffs and false impressions that something changed.
- **Environment and editor artifacts.** Files like `.DS_Store` (macOS), `.Rhistory`, `.RData`, and IDE-specific folders communicate nothing useful to collaborators.

1.5.2 Syntax

A `.gitignore` at the root of the repository applies to the whole project. The syntax is straightforward:

```
# Lines starting with # are comments

# Ignore a specific file
.env

# Ignore all files with this extension
*.csv

# Ignore a directory and everything in it
data/raw/

# Ignore a pattern anywhere in the tree
**/.DS_Store

# Exception: track this one file even though *.csv is ignored
!data/raw/small-example.csv
```

Patterns without a `/` match anywhere in the tree. A leading `/` anchors to the repository root. A trailing `/` matches directories only.

1.5.3 A starting point for R and Python projects

GitHub maintains a collection of language-specific templates at github.com/github/gitignore. When you create a new repository on GitHub, you can select one of these as a starting point. For most data science work in R and Python, a reasonable `.gitignore` covers:

```
# R
.Rhistory
.RData
.Rproj.user/
*.Rproj          # add back if you want to commit the project file

# Python
__pycache__/
*.py[cod]
.venv/
.env

# Data and outputs -- better handled with a .gitignore inside each folder
# (see project structure in the data organization chapter)
data/
output/

# OS artifacts
.DS_Store
Thumbs.db

# Credentials and personal config
config.yml       # if it contains paths or secrets
*.key
*.pem
```

Warning

A `.gitignore` only affects untracked files. If you accidentally committed a file before adding it to `.gitignore`, it will continue to be tracked. To stop tracking it without deleting it locally:

```
git rm --cached path/to/file
```

Then commit the removal and add the file to `.gitignore`.

Tip

Per-directory `.gitignore` files work alongside the root one. The project structure in Section 2.3 places a `.gitignore` inside `data/` and another inside `output/`. This is a good pattern: the intent is visible right where the data lives, and it protects those folders even if the root `.gitignore` is later modified.

2. Data Organization

Good data organization is foundational to reproducible, efficient research. Decisions made at the moment of data collection and storage ripple through every downstream step—analysis, visualization, sharing, and archiving. This chapter covers practical principles for organizing data in spreadsheets, naming files consistently, structuring project directories, and storing data in formats that work well with code.

2.1 Data Organization in Spreadsheets

Broman and Woo's "Data Organization in Spreadsheets" [1] offers twelve practical recommendations for anyone who stores data in Excel or similar tools. The core philosophy: structure your data so that it can be read directly into R or Python without manual cleanup.

2.1.1 Be Consistent

Consistency is the single most important principle. Pick conventions and stick to them within a project:

- Use the same codes for categorical variables (e.g., always `"M"` and `"F"`, not sometimes `"Male"`)
- Use the same missing value code throughout (e.g., `NA`)
- Use the same variable names across files and time points
- Use the same date format everywhere

2.1.2 Make It a Rectangle

Raw data should be a single, clean rectangle:

- **Rows are observations** (subjects, samples, time points)
- **Columns are variables**
- **One header row** at the top with short, meaningful column names
- No merged cells, no blank rows used for visual separation, no summary rows mixed in

This is consistent with the principles of **tidy data** [2], where each variable forms a column, each observation forms a row, and each type of observational unit forms a table.

2.1.3 One Thing Per Cell

Each cell should contain exactly one value. Don't cram multiple pieces of information into a single cell (e.g., `"120/80"` for blood pressure—use two columns instead). Don't use a cell to store a note alongside a value.

2.1.4 Fill in All Cells

Don't leave cells blank to imply "same as above." Every cell should contain an explicit value. Use a consistent missing data code (e.g., `NA`) rather than leaving cells empty—blank cells are ambiguous: was the value missing, not collected, or zero?

2.1.5 Write Dates as YYYY-MM-DD

Use ISO 8601 format for all dates: `2024-03-15`. This format:

- Sorts correctly alphabetically
- Is unambiguous across locales (`03/04/2024` means different things in the US vs. Europe)
- Is not mangled by Excel (unlike many other date formats)

> ⚠ Warning
>
> Excel silently converts many text strings to dates (e.g., the human gene name `SEPT1` becomes a date). Consider storing year, month, and day in separate columns if you're collecting data in Excel, or prefix dates with an apostrophe to force text storage.

2.1.6 Choose Good Names for Things

Apply consistent naming rules to columns, sheets, and files:

- No spaces—use underscores (`_`) or hyphens (`-`)
- No special characters (`@`, `#`, `%`, `(`, `)`)
- Short but meaningful (`participant_id`, not `p` or `the_participant_identifier_number`)
- All lowercase is a safe default
- Avoid starting names with numbers

2.1.7 No Calculations in Raw Data Files

Keep raw data raw. Don't add sum rows, averages, or derived columns to the data file—do that in your analysis script. The raw data file is a permanent record of what was collected; calculations belong in code that can be inspected, corrected, and rerun.

2.1.8 Don't Use Formatting as Data

Color-coding, bold text, or cell highlighting to indicate something (e.g., red = outlier, bold = reviewed) is invisible to code. Add an explicit column instead: `is_outlier` (`TRUE`/`FALSE`) or `review_status` ("reviewed", "pending").

2.1.9 Create a Data Dictionary

Every dataset should be accompanied by a data dictionary (also called a codebook) that documents:

- Variable name
- Description of what it measures
- Units
- Allowable values or range
- How missing values are coded

A simple spreadsheet or CSV with one row per variable works fine.

2.1.10 Save Data in Plain Text

Save the analysis-ready version of your data as CSV (comma-separated values), not as `.xlsx`. CSV files:

- Can be opened by any software
- Are readable in version control diffs
- Don't have hidden formatting or macros
- Are stable long-term

Keep the original Excel file if needed, but always export a clean CSV as the file you hand off to analysis.

2.1.11 Make Backups

Raw data should be treated as read-only and backed up in at least two places (e.g., local drive + cloud storage). Consider these tiers:

- **Primary:** Your working copy
- **Local backup:** External drive or NAS
- **Off-site/cloud:** Institutional storage, OneDrive, or similar

Version control (see Chapter 1.) is excellent for code and small text files but is not ideal for large binary data files.

2.2 File Naming

Jenny Bryan's "How to Name Files" [3] lays out three principles that make file names work well for both humans and computers. See also her talk from NormConf 2022:
https://www.youtube.com/watch?v=ES1LTlnpLMk

2.2.1 Three Principles

1. Machine readable
File names should be parseable by code without heroics:

- No spaces (use `_` or `-` instead)
- No special characters (`&`, `*`, `#`, `(`, `)`, accented letters)
- No case sensitivity issues—stick to lowercase
- Use delimiters consistently so you can split on them: underscores between metadata fields, hyphens within a field

2. Human readable
The name should tell you what's in the file without opening it:

- Include a descriptive "slug" that summarizes the content
- `2024-03-15-enrollment-summary.csv` is better than `data_v3_FINAL.csv`

3. Plays well with default ordering
Files should sort into a useful order automatically:

- Use ISO 8601 dates (`YYYY-MM-DD`) at the start of the name so files sort chronologically
- Left-pad numbers with zeros: `fig-01.png`, `fig-02.png`, ..., `fig-10.png` (not `fig-1.png`, `fig-2.png`, `fig-10.png`)

2.2.2 Recommended Pattern: `YYYY-MM-DD-slug`

For dated outputs (reports, snapshots, exports), the pattern `YYYY-MM-DD-descriptive-slug.ext` works well:

```
2024-03-15-enrollment-summary.csv
2024-06-01-adverse-events-cleaned.csv
2024-09-30-q3-interim-analysis.html
```

For numbered sequences (figures, scripts with a natural order), use zero-padded numbers:

```
01-import-data.R
02-clean-data.R
03-fit-models.R
04-make-figures.R
```

For files that don't change often and aren't dated, a simple descriptive slug is fine:

```
protocol.pdf
data-dictionary.xlsx
consent-form-v2.pdf
```

> **Tip**
>
> A good file name is essentially metadata. When you can glob a directory and parse the file names with code—extracting dates, conditions, or sample IDs—you've done it right.

2.3 Project Directory Structure

2.3.1 Simple project-oriented workflow

A consistent folder structure makes projects navigable by collaborators (and your future self). One convention is shown below:

```
project-name/
├── data/
│   ├── .gitignore    # Don't commit data to version control.
│   ├── raw/          # original data -- never edited directly
│   └── processed/    # cleaned, analysis-ready files
├── R/                # scripts and functions
├── output/
│   ├── .gitignore    # Don't commit large output files.
│   ├── figures/
│   └── tables/
└── report.qmd
```

Key rules:

- **Raw data is sacred.** The `data/raw/` folder is read-only. Never overwrite or edit raw data files. Also, you don't want to commit large files to version control, so be careful with your `.gitignore`.
- **All data manipulation is scripted.** Cleaned data in `data/processed/` is generated by a script, not by hand.
- **One project = one directory.** Keep all files related to a project together.

The ProjectTemplate (R) and Cookiecutter Data Science (Python) tools can scaffold these structures automatically.

2.3.2 Version controlling code with data on a shared drive

Finally, it's not uncommon in public health settings to have a setup where you're working with collaborators version controlling code as described in Chapter 1., but the data lives on a secure shared drive that cannot be moved or copied into the repository. This creates a tension between reproducibility and security that can be resolved with careful project organization and path management. The governance and access implications of working with data on secure shared drives, including data use agreements and secure data environments, are covered in Chapter 19..

Imagine that you have a distributed research team working with sensitive data that:

- Lives on a secure shared drive that cannot be moved
- Is organized in nested folders, for example, by state and year (e.g., `Lab/StateA/2023/`, `Lab/StateB/2024/`, etc.)
- Requires collaborative code development across Windows, Mac, and Linux machines
- Needs version control for reproducibility and collaboration

Storing data in a *database* would be preferable to a shared drive, but many public health agencies don't have the resources or infrastructure to set up and maintain databases. So how can we manage this?

2.3.2.1 The Conflict

Traditional approaches create impossible trade-offs:

❌ Option 1: One code repository without proper path management

- Forces hardcoded paths: `setwd("Z:/Lab/StateA/2024")`
- Breaks on different operating systems (Z:/ vs /Volumes/)
- Breaks when different team members have different drive mappings
- Violatesproject-oriented workflow principles
- Code is not reproducible

❌ Option 2: Code repositories inside each data folder

- Creates dozens of scattered git repositories
- Makes code reuse nearly impossible
- Version control nightmare (which repo has what function?)
- Accidental data commits to version control
- Multiple people doing git operations in the same shared folder causes conflicts. Not merge conflicts, but file locking and permission issues that can break everyone's workflow.

❌ Option 3: Move data into code repository

- Violates data security requirements
- Data cannot leave the secure shared drive
- Makes git repository enormous and slow
- Not feasible

2.3.2.2 A Better Way: Centralized Code Repository with Configurable Paths

✅ One way to manage this is with a **single centralized code repository with configurable paths**:

- One repository for all analysis code (easy to find, maintain, collaborate)
- Reproducible across team members with different operating systems
- Secure: data never leaves the shared drive or enters version control
- Flexible: works with nested folder structures
- Collaborative: proper git workflow without conflicts
- Project-oriented: no `setwd()`, paths are configurable not hardcoded

See stephenturner/demo-project-path-config for an example of one way to do this using configuration files. This can also be achieved with environment variables. **How it works:**

- Code lives in a git repository on each person's **local machine**
- Each team member has a personal `config.yml` file (not version controlled) with their specific paths
- Code uses helper functions to construct paths dynamically
- Data stays on the shared drive (read-only for most operations)
- Outputs go to each person's local directories
- Everyone commits code changes, never data

2.4 File Formats

Format	Best for	Avoid when
`.csv`	Tabular data, analysis input/output	Binary data, very wide datasets
`.tsv`	Tabular data with commas in values	—
`.json`	Nested/hierarchical data, configs	Large flat tables
`.parquet`	Large tabular data, columnar access	Simple small datasets
`.xlsx`	Data entry, sharing with non-coders	Final analysis-ready data

Plain text formats (CSV, TSV, JSON) are preferred for long-term storage and reproducibility. Proprietary binary formats (`.xlsx`, `.sav`, `.sas7bdat`) may become unreadable as software evolves. If you inherit data in these formats, Section 14.3 covers reading them into R and converting them to open formats.

2.5 Additional Best Practices

Use version control for code, not data. Git is designed for text files. Large or binary data files should live in dedicated storage (cloud drives, data repositories, databases) and be referenced in your code, not committed to the repository. Use a good `.gitignore`.

Document your cleaning steps. The gap between raw and processed data should be bridged entirely by a script that anyone can run. Comments in that script explaining *why* you made certain decisions are invaluable.

Plan for sharing. Before a project ends, ask: could someone else reproduce this analysis from the raw data? Good file organization, consistent naming, and a clear README make the answer yes.

Avoid "final" in file names. `analysis-final-FINAL-v3-USE-THIS-ONE.R` is a sign that version control was not used. Use Git instead, and name the file simply `analysis.R`.

3. Data Validation and QA

The failure mode that causes the most damage in public health data work is not a crashed script — it is an analysis that runs to completion on bad data. A negative case count, a duplicate record, a silently coerced column type: these do not throw errors. They propagate quietly through every table, figure, and report downstream, and the problem surfaces only when someone questions a number that seems wrong. By then, it is often too late to trace the source.

The remedy is to validate data at the moment of ingestion, before any analysis runs. If the data does not meet your expectations, the code should fail immediately and loudly, with a message that explains why. This chapter covers two tools for doing that: base R assertions for simple, targeted checks, and the `pointblank` package for structured validation of an entire dataset.

Throughout this chapter, the examples use the following simulated weekly influenza surveillance dataset. It has four problems baked in: a negative case count, a missing county, a missing rate, and a duplicate record.

```
flu <- data.frame(
  week = c(1, 2, 3, 4, 4),
  county = c("Fairfax", "Arlington", NA, "Loudoun", "Loudoun"),
  disease = c("Flu", "Flu", "Flu", "Flu", "Flu"),
  cases = c(23, 41, 18, -5, 12),
  rate = c(2.1, 3.8, 1.6, NA, 1.1)
)

flu
```

```
  week    county disease cases rate
1    1   Fairfax     Flu    23  2.1
2    2 Arlington     Flu    41  3.8
3    3      <NA>     Flu    18  1.6
4    4   Loudoun     Flu    -5   NA
5    4   Loudoun     Flu    12  1.1
```

3.1 Failing Fast

The simplest validation is a condition check paired with `stop()`. If the condition fails, execution halts and the message you wrote appears in the console — and in a Quarto document, in the output.

```
if (any(flu$cases < 0, na.rm = TRUE)) {
  stop("Negative case counts detected. Inspect raw data before proceeding.")
}
```

```
Error: Negative case counts detected. Inspect raw data before proceeding.
```

The `error: true` chunk option lets the document continue rendering even after an error — useful for demonstrating what a failed check looks like. In a real analysis, you would not set this option; you would want rendering to stop until the data problem is fixed.

For multiple checks, `stopifnot()` is more concise. Named arguments become the error message, so the output tells you which check failed:

```
stopifnot(
  "Negative case counts" = all(flu$cases >= 0, na.rm = TRUE),
  "Missing county values" = !anyNA(flu$county),
```

```
  "Duplicate records" = !anyDuplicated(flu[, c("week", "county")])
)
```

```
Error: Negative case counts
```

The error stops at the first failure. If `cases` and `county` both have problems, you only learn about `cases` until you fix it and re-run. For a dataset with many potential issues, that feedback loop is slow.

3.2 Structured Validation with pointblank

`pointblank` replaces the one-at-a-time assertion model with a declarative pipeline: you describe every expectation up front, run them all at once, and get a structured report showing which passed and which failed — without stopping at the first problem.

```
install.packages("pointblank")
```

3.2.1 Basic Workflow

The core workflow has three steps: create an agent bound to the data, add validation steps, then interrogate. Running all three checks against `flu` — which has a negative case count, a missing county, and a duplicate row — should produce three failures.

> **Note**
>
> The pointblank outputs are only rendered in html format at https://dstt.stephenturner.us/. These interactive formats are not rendered in the PDF or EPUB versions of this guide.

```
library(pointblank)

agent <- create_agent(tbl = flu, label = "Weekly flu surveillance") |>
  col_vals_gte(
    columns = cases,
    value = 0,
    label = "Case counts must be non-negative"
  ) |>
  col_vals_not_null(
    columns = c(week, county),
    label = "Week and county cannot be missing"
  ) |>
  rows_distinct(
    columns = c(week, county),
    label = "No duplicate week/county records"
  ) |>
  interrogate()

agent
```

All three checks fail: row 4 has `cases = -5`, row 3 has `county = NA`, and rows 4 and 5 are both `week = 4, county = "Loudoun"`. Each failure row shows the count of offending records. Critically, all three ran — the report does not stop at the first problem the way `stop()` does. You see the full picture at once.

3.2.2 Adding More Checks

Real surveillance data has more constraints. A few more validations on the same dataset:

```
create_agent(tbl = flu, label = "Weekly flu surveillance — extended") |>
  col_is_numeric(
    columns = c(cases, rate),
    label = "Case count and rate must be numeric"
```

```
  ) |>
  col_vals_in_set(
    columns = disease,
    set = c("Flu", "COVID-19", "RSV"),
    label = "Disease must be from the approved list"
  ) |>
  col_vals_between(
    columns = week,
    left = 1,
    right = 52,
    label = "Week must be between 1 and 52"
  ) |>
  col_vals_gte(
    columns = rate,
    value = 0,
    na_pass = TRUE,
    label = "Rate must be non-negative (NAs allowed)"
  ) |>
  interrogate()
```

The `na_pass = TRUE` argument on the rate check lets missing values pass that particular step — sometimes a value being missing is acceptable, while a negative value never is. These two conditions are separate checks and should be stated separately.

3.2.3 Common Checks for Public Health Data

Validation	pointblank function	Use for
No negative counts	`col_vals_gte(value = 0)`	Case counts, deaths, population denominators
No missing required fields	`col_vals_not_null()`	County, date, disease code
No duplicate records	`rows_distinct()`	Surveillance records keyed by geography + time
Values from a known list	`col_vals_in_set()`	Disease codes, county names, FIPS codes
Dates within a valid range	`col_vals_between()`	Report dates, surveillance weeks
Correct column type	`col_is_numeric()`, `col_is_date()`	Catches silent type coercion on import
Counts match a known total	`col_vals_lte()`	Deaths ≤ total cases, OD deaths ≤ total deaths

3.2.4 Stopping on Failure

By default, `interrogate()` returns the full report without stopping execution. If any validation failure should halt the pipeline — for example, in an automated report where you never want to proceed with bad data — pass the agent to `all_passed()`:

```
if (!all_passed(agent)) {
  stop("Data validation failed. Review the agent report before proceeding.")
}
```

This gives you the structured report from pointblank for diagnosis while still failing loudly enough to stop an automated run.

3.3 Where to Put Validation

Validation belongs in the setup block, immediately after data is read in and before any transformation or analysis begins. This is the one place where bad data can be caught before it contaminates everything downstream.

```
library(readr)
library(pointblank)

flu <- read_csv("data/flu-2024.csv")

# Validate immediately after reading
agent <- create_agent(tbl = flu, label = "flu-2024 validation") |>
  col_vals_gte(columns = cases, value = 0, label = "No negative counts") |>
  col_vals_not_null(columns = c(week, county), label = "No missing keys") |>
  rows_distinct(columns = c(week, county), label = "No duplicate records") |>
  interrogate()

if (!all_passed(agent)) {
  stop("Validation failed — see agent report above.")
}

# Analysis only runs if validation passes
```

Putting validation here means the failure is traceable to a specific data file and a specific expectation. The message tells you what was wrong, not just that something broke three steps later in an `aes()` call.

> **Tip**
>
> Commit your validation code to version control alongside the analysis (see Chapter 1.). When a data supplier changes a format, adds a new disease code, or starts providing a column in a different type, your validation catches it on the next render rather than silently producing wrong output. Data pulled from a web API deserves the same treatment, for reasons covered in Section 13.7. When validation failures trace back to a change in an upstream data system, Chapter 19. covers how to work with IT and data stewards to resolve it.

4. Reproducible Reporting

Every data team has lived through some version of this: the analysis is done, tables and figures have been generated, and now begins the tedious work of copying numbers into a Word document. Then the data updates. Or a reviewer catches an error in the underlying code. And every number in the document has to be found and replaced – manually, one at a time, with no guarantee that any were missed. Meanwhile, the number in the executive summary and the number in the appendix table, which should be identical, have quietly drifted apart.

Reproducible reporting breaks this cycle. Instead of treating code and document as separate artifacts, the report *is* the code: prose, tables, and figures are assembled from a single source file that executes the analysis and weaves results directly into the output. Update the data, re-render, and every number updates automatically – because it was never copied in the first place.

Quarto is the tool that makes this practical. A Quarto document (`.qmd`) is a plain-text file that combines a YAML header describing the output format, prose, and code blocks that execute and embed their results. Render it once and get an HTML page. Change one line and render again to get a PDF or a Word document from the same source.

> **Quarto documentation**
>
> The Quarto documentation is the definitive reference for document options, output formats, and formatting syntax. This chapter covers the concepts and patterns that matter most for reproducible reporting; the Quarto docs are the place to go for comprehensive reference material.

4.1 Not Just R

Quarto works with R, Python, Julia, and Observable JavaScript – in the same document if needed. The rest of this chapter uses R in most examples because R is the primary language in many public health teams, but Python code blocks work exactly the same way: they execute when the document renders and their output appears in the report.

A Python block that requires no external packages:

```
import statistics

weekly_cases = [127, 143, 98, 112, 156, 134, 89]
mean_cases = statistics.mean(weekly_cases)
peak_week  = max(weekly_cases)

print(f"Mean weekly cases: {mean_cases:.1f}")
print(f"Peak week: {peak_week}")
```

Output:

```
Mean weekly cases: 122.7
Peak week: 156
```

The `statistics` module is part of the Python standard library. The block executes at render time and its printed output appears in the document – no manual copying required.

> **Note**
>
> R and Python can coexist in the same `.qmd` file. R objects and Python objects live in separate sessions, but the `reticulate` package allows data to flow between them: `py$my_python_object` is useful when a team's data cleaning lives in Python and their visualization in R.

4.2 Document Anatomy

A Quarto document has three parts:

YAML front matter: a header block delimited by `---` that controls the title, author, date, and output format.

Prose: written in Markdown. The Quarto authoring guide covers formatting. The short version: `**bold**`, `*italic*`, `# Heading 1`, `## Heading 2`, and `- list item` cover the majority of what you need.

Code blocks: fenced blocks labeled with the language (```` ```r ```` or ```` ```python ````). When the document renders, each block executes in sequence and its output – printed values, tables, plots – is embedded in the output.

A minimal Quarto document:

````
---
title: "Influenza Surveillance Summary"
author: "Epidemiology Team"
date: today
format: html
---

## Overview

This report summarizes influenza activity for the current surveillance week.

```r
#| label: fig-trend
#| fig-cap: "Weekly influenza case counts"

library(ggplot2)
ggplot(flu_data, aes(x = week, y = cases)) +
  geom_line()
```
````

Render from the terminal with `quarto render report.qmd`, or from within Positron or RStudio using the Render button. The output file (here, `report.html`) appears in the same directory.

Code block options like `#| label:` and `#| fig-cap:` are set with the `#|` prefix on lines at the top of a block. Common options include:

Option	Effect
`echo: false`	Run the code but hide the source in the output
`include: false`	Run the code but hide both source and output
`message: false`	Suppress package loading messages
`warning: false`	Suppress warnings
`fig-cap: "..."`	Add a caption below a figure

A setup block at the top of the document – typically labeled `#| label: setup` with `#| include: false` – is the right place to load packages and read data so the rest of the document has access to them without displaying that machinery to readers.

> **Tip**
>
> R code cells inside `.qmd` files can be formatted with the Air formatter (see Section 15.2). Place your cursor in a cell and press `Cmd+K Cmd+F` (Mac) or `Ctrl+K Ctrl+F` (Windows/Linux) to format that cell. With format-on-save enabled for both `[r]` and `[quarto]` in your workspace settings, the active cell formats automatically when you save the document.

4.3 One Input, Many Output Formats

The same `.qmd` file can produce different output formats by changing the `format` key in the YAML header. This means you can share an HTML version with colleagues for day-to-day review, render a PDF for formal submission, and produce a Word document for stakeholders who need to annotate – all from one source file.

```
format: html    # interactive HTML page
format: pdf     # PDF via LaTeX
format: docx    # Microsoft Word
format: typst   # PDF via Typst (no TeX installation required)
```

To render multiple formats at once, list them:

```
format:
  html: default
  docx: default
  pdf: default
```

Running `quarto render report.qmd` then produces all three.

The practical benefit of format independence is durability. A report that exists only as a finished Word document is locked to the moment it was created – any change requires re-opening it and editing manually. The `.qmd` file is the permanent, authoritative version. The Word document is just one of its outputs, and it can always be regenerated. For guidance on choosing which format best serves a given audience, see Section 21.3.

> **Note**
>
> Some content behaves differently across formats. Interactive elements like `plotly` charts and `DT` tables work in HTML but fall back to static output in PDF and Word. If Word or PDF is a primary deliverable, test your render early – not after a report is due – so you aren't surprised by what doesn't translate.

4.4 Code Is the Report

The most consequential feature of reproducible reporting is not the output format – it is that computed values appear in prose automatically, without copy-paste.

In a Quarto document, inline R expressions surrounded by backticks evaluate and inject their result directly into text. Given a data frame called `cases` and an object `pct_change` computed in a prior code block:

```
A total of `r nrow(cases)` cases were reported in `r params$county` County
during `r params$year`, a `r pct_change`% change from the prior year.
```

When rendered, this becomes prose like:

> A total of 1,847 cases were reported in Fairfax County during 2024, a 12.3% change from the prior year.

Every number came from code. If the underlying data is revised and the document is re-rendered, every figure that depends on it updates everywhere it appears – the executive summary, the body, the footnote. There is no list of cells to hunt down in a Word document, and no way for the number on page one to silently disagree with the table on page seven.

> **Tip**
>
> Use `format()` or `scales::comma()` to control how numbers look inline. A raw expression produces `1847`; `scales::comma(nrow(cases))` produces `1,847`. For percentages, `scales::percent(pct_change, accuracy = 0.1)` gives `12.3%`. This formatting belongs in the inline expression, not in a separate variable, so the display logic stays close to where it's used.

This approach also makes the analytical decision trail explicit. A number that appears in a report had to come from somewhere in the code. That traceability is exactly what public health accountability requires – when a number is questioned, you can show exactly how it was produced.

4.5 Paths and Project Structure

The most common way a reproducible report breaks on a different machine – or for a different team member – is an absolute file path. A path like `/Users/maria/Documents/Projects/flu-report/data/flu-2024.csv` works exactly once, on Maria's laptop, and nowhere else.

The solution is to use paths relative to the project root and to structure the project so that the `.qmd` file and its data live in the same directory tree. As described in Chapter 2., a well-organized project directory has a clear, stable structure:

```
flu-report/
├── flu-report.qmd
├── data/
│   └── flu-2024.csv
└── output/
```

From `flu-report.qmd`, the data file is simply `"data/flu-2024.csv"` – no absolute path, no machine-specific prefix.

If `.qmd` files are nested in subdirectories, the `here` package resolves paths relative to the project root regardless of where the file sits:

```
library(here)
flu_data <- read_csv(here::here("data", "flu-2024.csv"))
```

`here::here()` finds the project root by looking for a `.Rproj`, `.git`, or similar marker file. It works whether code runs interactively from the console or from within a render.

> ⚠ Warning
>
> Never use `setwd()` inside a `.qmd` file. It changes the working directory mid-execution and will almost certainly break for anyone else who renders the document. Relative paths and `here()` are the right tools.

4.6 Parameterized Reports

Parameterized reports take the reproducibility model one step further: instead of hardcoding a county, disease, or time period into the analysis, you declare those values as parameters and pass them in at render time. One template becomes many reports.

4.6.1 Declaring Parameters

Parameters are declared in the YAML front matter under the `params` key. Each parameter gets a name and a default value:

```
---
title: "County Disease Surveillance Report"
subtitle: "`r params$county` County -- `r params$year`"
date: today
format: html

params:
  county:  "Fairfax"
  year:    2024
  disease: "Influenza"
---
```

The default values are what render when you run `quarto render` with no additional arguments. They also define the parameter type: a quoted string is a character, a bare number is numeric.

4.6.2 Using Parameters

Inside the document, parameters are available as `params$name` – in code blocks and in inline expressions alike:

```
#| label: setup
#| include: false

library(dplyr)
library(ggplot2)

# Filter to the selected county, year, and disease
county_data <- surveillance |>
  filter(
    county  == params$county,
    year    == params$year,
    disease == params$disease
  )

total_cases <- nrow(county_data)
pct_change  <- round((total_cases / prior_year_cases - 1) * 100, 1)
```

Then in prose:

```
This report summarizes `r params$disease` surveillance data for
`r params$county` County during `r params$year`.

A total of `r scales::comma(total_cases)` cases were reported,
a `r pct_change`% change from the prior year.
```

Nothing in the body of the document is hardcoded to a specific county or year. The entire report adapts to whatever parameters are supplied at render time.

4.6.3 Rendering with Different Parameters

To render the report for a different county without editing the file, pass parameters on the command line:

```
quarto render report.qmd \
  -P county:"Arlington" \
  -P year:2024 \
  --output "arlington-2024.html"
```

To generate one report per county automatically, use `quarto_render()` from the `quarto` R package:

```
library(quarto)
library(purrr)

counties <- c("Fairfax", "Arlington", "Alexandria", "Loudoun")

walk(counties, function(county) {
  quarto_render(
    "report.qmd",
    execute_params = list(county = county, year = 2024),
    output_file    = paste0(tolower(gsub(" ", "-", county)), "-2024.html")
  )
})
```

This loop renders one HTML report per county – same template, different data, different output file – with no manual editing between runs. For recurring reports (weekly surveillance summaries, monthly dashboards), this loop can be driven by a script that pulls the current list of counties or time periods from the data itself.

Quarto parameters documentation

The Quarto documentation on parameterized reports covers additional options including Knitr and Jupyter parameter handling, and how to use parameters with Quarto Projects to generate many outputs in a single command.

4.7 Consistent Branding with brand.yml

When a team produces reports across multiple formats – HTML pages, Word documents, PDFs – maintaining consistent colors, fonts, and logos is tedious if each format is configured separately. `brand.yml` is a specification for defining visual identity once and applying it across Quarto outputs and other Posit tools.

A `_brand.yml` file defines the elements of an organization's visual identity:

```yaml
meta:
  name: "County Health Department"
  link: "https://health.example.gov"

color:
  palette:
    navy:       "#1B4F72"
    teal:       "#0E6655"
    light-gray: "#F2F3F4"
  foreground: navy
  background: white
  primary:    navy

typography:
  fonts:
    - family: Source Sans Pro
      source: google
  base:     Source Sans Pro
  headings: Source Sans Pro

logo:
  small:  "assets/logo-small.png"
  medium: "assets/logo-medium.png"
```

Save this as `_brand.yml` in the project root. Quarto picks it up automatically at render time – no additional configuration required in each document's YAML header.

Brand settings cascade across formats: the same color palette applied to HTML output also applies to PDF, and both pick up the logo and typography definitions from the same file. Teams that produce recurring reports – weekly surveillance summaries, monthly dashboards, annual reports – can standardize the look across all outputs by maintaining a single `_brand.yml` rather than configuring each document separately.

Documentation

- **brand.yml specification:** posit-dev.github.io/brand-yml – the full reference for all supported fields, including color, typography, logo, and defaults.
- **Quarto + brand.yml:** quarto.org/docs/authoring/brand.html – how Quarto consumes a `brand.yml` file and which output formats support it.

5. Accessibility

A report that cannot be read by a screen reader, a chart that disappears for someone with color blindness, a PDF that fails institutional archiving requirements: these are not edge cases. Approximately 8% of men have some form of color vision deficiency. Screen reader users include blind and low-vision users but also people with dyslexia, motor impairments, and situational limitations. For most public health organizations, accessible outputs are a legal requirement, not a nice-to-have.

Quarto handles much of the structural work automatically: semantic HTML headings, document metadata flowing into PDF fields, proper reading order. What it cannot do automatically is supply meaning that only you have: descriptive alt text for figures, color choices that work without color, and table structure a screen reader can navigate. Quarto 1.8 added HTML accessibility checks, and Quarto 1.9 added PDF standards support, so there are now tools to verify your work.

This chapter assumes familiarity with Quarto documents and code chunks. For background on Quarto's output formats and rendering workflow, see Chapter 4..

5.1 The Regulatory Landscape

Digital accessibility is not optional for most public health organizations. The legal and policy framework has tightened considerably in recent years.

Section 508 of the Rehabilitation Act requires all U.S. federal agencies to make their electronic and information technology accessible to people with disabilities. The 2017 refresh (with compliance required from January 2018) aligned federal requirements with the Web Content Accessibility Guidelines (WCAG) 2.0 Level AA, which remains the formal legal floor. WCAG 2.1 and 2.2 are backwards compatible with 2.0, and agencies are encouraged to go beyond the 2.0 floor, but the Access Board's standards formally incorporate WCAG 2.0 by reference. CDC applies Section 508 as the baseline standard for all agency-developed and procured technologies, including reports and dashboards. CSTE updated its Special Emphasis Reports in December 2022 to adhere to federal 508 compliance regulations.

OMB Memorandum M-24-08 (December 2023) was the first major Section 508 guidance update in a decade. It requires federal agencies to designate a Section 508 program manager accountable for digital accessibility oversight, establish processes for tracking and resolving accessibility issues, and incorporate accessibility into procurement. If your team develops tools or reports for federal partners, M-24-08 shapes what they can accept.

ADA Title II extends accessibility requirements to state and local government entities. A 2024 DOJ final rule set WCAG 2.1 Level AA as the compliance standard. Compliance deadlines depend on entity size: April 24, 2026 for entities with a total population of 50,000 or more, and April 26, 2027 for those with populations under 50,000 or any special district government. State and local health agencies are covered directly by this rule. Note that these deadlines are specific to Title II public entities; private institutions receiving federal funding have general accessibility obligations under Section 504, but the specific WCAG conformance deadlines from the 2024 rule do not apply to them.

WCAG 2.2 organizes accessibility requirements around four principles, often abbreviated **POUR**:

- **Perceivable**: Information must be presentable in ways users can perceive, including text alternatives for non-text content
- **Operable**: Interface components and navigation must be operable, including keyboard navigation and sufficient time to interact
- **Understandable**: Information and operation must be understandable, including readable text and predictable behavior
- **Robust**: Content must work with current and future assistive technologies

WCAG organizes success criteria into three conformance levels: A (minimum), AA (the standard required by Section 508's WCAG 2.0 floor and ADA Title II's WCAG 2.1 rule), and AAA (enhanced). Level AA covers the requirements most organizations face.

> **WCAG versions and backwards compatibility**
>
> WCAG 2.0, 2.1, and 2.2 are backwards compatible: meeting 2.2 automatically satisfies 2.1 and 2.0. The most practically relevant additions in 2.2 include minimum touch target sizes (2.5.8) and constraints on cognitive demands during login (3.3.8). For static reports and dashboards, the changes in 2.1 and 2.2 are mostly felt in interactive applications rather than documents.

5.2 Alt Text for Figures

Every figure in a published report needs a text alternative (alt text) that conveys the same information to someone who cannot see the image. Screen readers announce alt text when they encounter a figure; without it, users hear only "image" or nothing at all.

In Quarto, the `fig-alt` chunk option sets alt text separately from the figure caption (`fig-cap`). They serve different purposes:

- `fig-cap` is the visible caption beneath the figure, typically a brief label like "Weekly influenza case counts, 2024."
- `fig-alt` is the invisible text alternative. It should describe what the figure shows: the trend, the comparison, the key value, for someone who cannot see it.

```
library(ggplot2)

ggplot(ili_data, aes(x = week, y = cases, color = region)) +
  geom_line(linewidth = 1) +
  labs(
    x = "Surveillance week",
    y = "ILI cases",
    color = "Region"
  ) +
  theme_minimal()
```

Good alt text describes the data, not the chart type. "A line chart" tells a screen reader user almost nothing. "Weekly ILI case counts peak in the Northeast in week 5 at 2,400 cases, then decline through summer" tells them what the chart is about.

A few guidelines:

- If the figure caption already conveys the key message, alt text can reinforce or expand on it, but they should not be identical
- If the underlying data is in a table elsewhere in the document, you can note that in the alt text: "See `@tbl-flu-counts` for the underlying data"
- Decorative images (logos, dividers) can use `fig-alt: ""`: an empty string signals to screen readers that the image carries no information

> **Generating alt text with Claude Code**
>
> Writing good alt text for many figures is tedious. Emil Hvitfeldt developed a Claude Code skill that automates it: given a `.qmd` file, it reads each code chunk, runs the code mentally, and inserts a `fig-alt` option with a descriptive summary. See this post on adding alt text to Quarto documents with a Claude Code skill for installation instructions and examples.
>
> Once installed, the skill is available as a slash command:
>
> ```
> /write-alt-text @my-report.qmd
> ```

Alt text travels to PDF

When you enable PDF accessibility standards (see Section 5.6), alt text set via `fig-alt` carries through to the tagged PDF. Writing it once in your code chunk covers both HTML and PDF output.

5.3 Color and Contrast

Approximately 8% of men and 0.5% of women have color vision deficiency, most commonly difficulty distinguishing red from green. Using color as the only way to distinguish categories means a meaningful share of your audience may not be able to read the chart.

Color should never be the *sole* differentiator. Combine it with shape, line type, or direct labels. When color is used, choose palettes designed for color vision deficiency.

The viridis palettes (available in the `viridis` package and built into `ggplot2` since version 3.0) are perceptually uniform and distinguishable under all common forms of color vision deficiency, including in grayscale. They work for both continuous and discrete data.

```
library(ggplot2)

ggplot(overdose_data, aes(x = year, y = deaths, fill = substance)) +
  geom_col(position = "dodge") +
  scale_fill_viridis_d(option = "D") +
  labs(
    x = "Year",
    y = "Deaths",
    fill = "Substance"
  ) +
  theme_minimal()
```

ColorBrewer palettes (available in the `RColorBrewer` package) were designed with accessibility in mind. The `"Dark2"` and `"Set2"` palettes are colorblind-safe for up to eight categories. For sequential data, `"YlOrRd"` and `"Blues"` are accessible and widely understood in public health contexts.

```
library(ggplot2)
library(RColorBrewer)

ggplot(vax_data, aes(x = coverage, y = county, fill = age_group)) +
  geom_col() +
  scale_fill_brewer(palette = "Dark2") +
  facet_wrap(~age_group) +
  labs(
    x = "Coverage (%)",
    y = NULL,
    fill = "Age group"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```

Text contrast is also a consideration. WCAG 2.2 Level AA requires a contrast ratio of at least **4.5:1** for normal-weight text and **3:1** for large text (18pt or 14pt bold). Light gray text on a white background, common in default ggplot2 axis labels and annotations, often fails this threshold. The axe-core checks described in Section 5.5 will flag contrast violations in HTML output.

Test your palette

To check whether a chart is readable without color, print it in grayscale or run it through a color vision simulator. The `colorblindr` package provides `cvd_grid()`, which renders a ggplot in several color-deficiency simulations side by side. If categories are still distinguishable, the palette works.

5.4 Accessible Tables

Screen readers navigate HTML tables by announcing row and column headers. A table without proper headers, or with merged cells that break the row/column relationship, can be confusing or unusable. The same structural requirements apply to tagged PDFs.

- Every table should have a caption that identifies what it contains
- Column headers should be descriptive, not cryptic abbreviations
- Avoid spanning cells (merged rows or columns): they complicate the header/cell relationship that screen readers rely on
- Use tables only for tabular data, not for layout

The `gt` package produces well-structured HTML tables with semantic markup. `knitr::kable()` with `kableExtra` is also widely used. Both produce accessible output when used with captions and clear headers.

Table 5.1: Reported disease cases, deaths, and case-fatality rates by condition, 2023.

```
library(gt)
library(dplyr)

disease_summary |>
  gt() |>
  tab_header(
    title = "Reportable Disease Summary, 2023",
    subtitle = "Selected conditions, all jurisdictions"
  ) |>
  cols_label(
    condition = "Condition",
    cases = "Cases",
    deaths = "Deaths",
    cfr = "Case-fatality rate (%)"
  ) |>
  fmt_number(columns = c(cases, deaths), decimals = 0) |>
  fmt_number(columns = cfr, decimals = 1)
```

The `tab_header()` and `cols_label()` calls provide the semantic title and full column labels that screen readers announce. A table without these is harder to navigate for users relying on assistive technology.

5.5 HTML Accessibility Checks

Quarto 1.8 introduced integrated support for axe-core, an industry-standard accessibility testing engine. Enabling it causes Quarto to inject the axe-core script into rendered HTML, which runs when the page loads in a browser.

To enable axe checks, add the `axe` option to your document's HTML format:

```
format:
  html:
    axe: true
```

With this basic configuration, violations are reported as messages in the browser developer console (open with F12 or Cmd+Option+I). To see a visible report embedded on the page itself, use the `document` output mode:

```
format:
  html:
    axe:
      output: document
```

This is useful during development, but you would not want to publish it. The recommended pattern is a debug project profile that enables the document report only locally:

```yaml
# _quarto-debug.yml (activated with: quarto render --profile debug)
format:
  html:
    axe:
      output: document
```

A third mode, `json`, produces machine-readable output compatible with browser automation tools like Puppeteer or Playwright:

```yaml
format:
  html:
    axe:
      output: json
```

> **i** What axe-core checks
>
> axe-core tests for a broad set of WCAG 2.x violations: missing alt text, insufficient color contrast, missing form labels, improper heading hierarchy, and missing landmark regions. It catches problems that are mechanically detectable. It cannot evaluate whether alt text is *meaningful*, only whether it is present. Human review is still necessary.

A minimal document that demonstrates a contrast violation:

```yaml
---
title: "Accessibility check demo"
format:
  html:
    axe:
      output: document
---

The following text fails contrast requirements:
[This text is nearly invisible.]{style="color: #eee"}
```

When previewed in a browser, the axe report flags the contrast failure and identifies the affected element. Resolve violations before publishing, then remove the `axe` option (or restrict it to the debug profile).

5.6 PDF Accessibility

Quarto 1.9 introduced experimental support for PDF accessibility standards, building on new tagging capabilities in LaTeX and Typst. Accessible PDFs embed semantic structure (document title, heading hierarchy, reading order, image alt text) in a form that screen readers and other assistive technologies can use.

Two standards are supported:

- **PDF/UA-1** (Universal Accessibility, first edition): the widely-used baseline standard, supported via Typst
- **PDF/UA-2** (second edition): a newer standard built on PDF 2.0, supported via LaTeX

5.6.1 Enabling PDF Standards

Typst (UA-1):

```yaml
format:
  typst:
    pdf-standard: ua-1
```

LaTeX (UA-2):

```yaml
format:
  pdf:
    pdf-standard: ua-2
```

Two things are always required, regardless of renderer:

1. A `title` in the YAML front matter
2. `fig-alt` on every figure (see Section 5.2)

Quarto's Markdown-based workflow handles most other requirements automatically: document metadata flows into the correct PDF fields, and heading structure from Markdown satisfies tagging requirements for reading order and document outline.

5.6.2 Validating with veraPDF

Quarto uses veraPDF, an open-source PDF/A and PDF/UA validator, to check that output meets the specified standard. Install it with:

```
quarto install verapdf
```

Once installed, Quarto runs veraPDF automatically after rendering when `pdf-standard` is set. Validation results appear in the terminal output.

5.6.3 Known Limitations

LaTeX: Content placed in the margin (using `.column-margin` divs, `cap-location: margin`, or margin-positioned references) prevents UA-2 compliance. The underlying LaTeX packages involved in margin placement do not cooperate with PDF tagging. Avoid margin content if UA-2 compliance is required.

Typst: Book-format documents are not yet fully compliant due to structural issues in the underlying orange-book template. For standalone documents, Typst validates cleanly and catches UA-1 violations automatically.

> Typst vs. LaTeX for accessibility
>
> Typst currently offers better practical PDF accessibility than LaTeX. It validates UA-1 cleanly on typical documents, requires no TeX installation, and is not affected by the margin-content restriction that blocks LaTeX's UA-2 compliance. If accessible PDF output is a priority and you are starting a new document, Typst is worth considering. See Chapter 4. for guidance on Quarto output formats.

5.7 Pulling It Together

Accessibility is easier to build in during authoring than to retrofit after the fact. A practical workflow for a Quarto report:

1. **Write alt text** for every figure using `fig-alt`. Do this when you write the code chunk, not at the end. Consider using the Claude Code alt text skill to generate first drafts.
2. **Use an accessible palette**: viridis or a colorblind-safe ColorBrewer palette. Verify by simulating color vision deficiency or viewing in grayscale.
3. **Structure tables** with captions and descriptive column labels via `gt` or `kable`.
4. **Check HTML output** with `axe: true` during local preview; resolve flagged violations before publishing.
5. **Enable `pdf-standard`** for PDF deliverables (`ua-1` for Typst, `ua-2` for LaTeX) and install veraPDF to validate.
6. **Avoid margin content in LaTeX** if UA-2 compliance is required.

i Accessibility benefits everyone

Accessible outputs are not only for users with disabilities. Alt text improves search engine indexing. Captions help non-native English speakers and people reading in noisy environments. High-contrast text is easier to read on mobile screens in bright light. Well-structured tables are easier for all users to navigate.

6. Coding with AI

AI tools for coding have matured rapidly, covering a spectrum from simple inline completions to fully autonomous agents that can read, write, and execute code across an entire project. This chapter provides a brief orientation to the tools most relevant to data scientists working in Positron.

6.1 Code Completion with GitHub Copilot

GitHub Copilot is a code completion service that suggests code as you type. Install the **GitHub Copilot** extension in Positron, and suggestions will appear inline in the editor and in notebooks — press Tab to accept a suggestion or Escape to dismiss it.

Copilot draws on the surrounding code and comments to infer what you're trying to write. It works well for boilerplate, common patterns, and filling in function arguments you'd otherwise have to look up.

A GitHub Copilot subscription is required; a free tier is available for individual developers.

6.2 Positron Assistant

Positron has a built-in AI assistant (available since version 2025.07.0-204) accessible via the chat panel in the left sidebar. Unlike Copilot, which only completes code, the assistant understands your full session context — loaded data frames, console history, plots, and project structure — making it well-suited to data analysis workflows.

The assistant operates in three modes:

- **Ask** — conversational Q&A. Use it to explain error messages, get help with a function, or talk through an analysis approach without modifying any files.
- **Edit** — makes targeted changes to selected code based on a natural-language instruction. Useful for refactoring, renaming, or reformatting a block of code.
- **Agent** — more autonomous. The agent can create and modify files, run terminal commands, and work across multiple files to complete a larger task.

The assistant uses a bring-your-own-key model: you connect it to an AI provider (Anthropic, OpenAI, GitHub Copilot, or AWS Bedrock) using your own API key. Posit does not see your prompts or code.

> Positron Assistant documentation
>
> The Positron Assistant documentation covers setup, model configuration, and mode details.

6.3 Databot

Databot is an experimental Positron extension purpose-built for exploratory data analysis. Where Positron Assistant helps you write code, Databot *writes and executes* short analysis snippets on your behalf, treating insights as the goal rather than the code. You describe what you want to understand about your data, and Databot iterates through code execution to get there.

Databot is currently in research preview and requires an Anthropic API key. It is designed for experienced R and Python users who can critically evaluate the code it produces — AI models can and do make mistakes in open-ended data exploration.

Open Databot via the Command Palette (`Ctrl+Shift+P` / `Cmd+Shift+P`) and search for "Databot".

> Tip
>
> See the Databot documentation for installation instructions and warnings that are worth reading.

6.4 Claude Code

Claude Code is a terminal-based AI coding agent from Anthropic. Unlike the Positron Assistant chat panel, Claude Code operates directly in a shell session with full access to the filesystem: it can read and write files, search the codebase, run commands, and coordinate changes across many files in a single session.

Install it (instructions below for MacOS, Linux, WSL, see docs for Windows).

```
curl -fsSL https://claude.ai/install.sh | bash
```

Then launch it from Positron's integrated terminal (**Terminal → New Terminal**):

```
claude
```

Claude Code is particularly effective for multi-file refactors, complex debugging sessions, and tasks that are tedious to coordinate manually. It is also well suited to translating and explaining legacy SAS programs, a workflow covered in Section 14.4.

6.5 Cost, Privacy, and Model Choice

6.5.1 Capability

Frontier models from OpenAI and Anthropic are substantially more capable than open-weight alternatives for complex reasoning and coding tasks. For most everyday coding work (fixing errors, writing functions, refactoring) a mid-tier model like Anthropic's Claude Sonnet is fast, inexpensive, and more than capable enough. Save the most powerful (and expensive) frontier models for genuinely hard problems.

6.5.2 Cost

API pricing is metered by token (roughly, by word). Typical coding assistance sessions cost cents to a few dollars; heavy agent sessions that read and write many files may cost a bit more. GitHub Copilot charges a flat monthly subscription instead, which can be more predictable for high-volume users.

6.5.3 Privacy

A common concern is whether proprietary code sent to a cloud AI service will be used to train future models. The short answer is: **not if you're paying for API access**.

Both Anthropic and OpenAI state explicitly that inputs and outputs from paid API usage are not used for model training. This is distinct from their free consumer products, which may use interactions for improvement. If your team is using Positron Assistant, Claude Code, or the OpenAI API with your own key, your code is not becoming training data.

> **i** Note
>
> See the authoritative statements directly:
>
> - Anthropic's privacy policy
> - OpenAI's privacy policy

Open-weight models (such as Meta's Llama or Google's Gemma) can be run entirely on local hardware, which provides the strongest possible privacy guarantee and meets strict data-residency or air-gap requirements. The trade-off is lower capability and the infrastructure overhead of running the model yourself. For most teams using cloud APIs with a commercial agreement, the privacy risk is lower than it is commonly assumed to be.

7. Dashboards

Quarto dashboards use `format: dashboard` to create multi-panel HTML data displays — no server required. They're a middle ground between a static report (a single linear document) and a Shiny app (interactive but requires a running R process). Dashboards are ideal for sharing summaries, KPIs, and plots that need to look polished without the overhead of maintaining a server. Chapter 21. covers how to decide whether a dashboard is the right output format for a given audience and finding.

The examples in this chapter are drawn from a real dashboard tracking U.S. drug overdose mortality data from the National Vital Statistics System (2015–2023).

> **Live demo and code**
>
> - **Live demo:** stephenturner.github.io/quarto-dashboard-demo
> - **Source code:** github.com/stephenturner/quarto-dashboard-demo

> **Quarto dashboard documentation**
>
> The Quarto documentation is the definitive resource for dashboard features, options, and examples. This chapter provides a practical walkthrough of building a dashboard, but the Quarto docs are the place to go for comprehensive reference material.

7.1 Dashboard Anatomy

7.1.1 YAML Front Matter

A minimal Quarto dashboard needs only two things in the front matter: a title and `format: dashboard`.

```yaml
---
title: "My Dashboard"
format: dashboard
---
```

Additional options control the theme, layout behavior, and navigation elements:

```yaml
---
title: "Drug Overdose Mortality"
subtitle: "National Vital Statistics System, 2015–2023"
format:
  dashboard:
    scrolling: false
    theme: cosmo
date: today
---
```

Setting `scrolling: false` (the default) sizes each row to fill the viewport height, producing a fixed-height layout. Setting `scrolling: true` allows the page to scroll freely, which is useful when the content is too tall to fit on one screen.

7.1.2 Rows and Columns

Quarto dashboards lay out content in rows by default. Level-2 headings (`##`) define rows, and each code cell within a row becomes a card that fills its share of the available width.

```
## Row {height=25%}

[cards here — share the row equally]

## Row {height=75%}

[cards here — taller row for plots]
```

The `{height=}` attribute (as a percentage or pixel value) controls how much vertical space each row takes. To switch to a column-based layout, use `orientation: columns` in the YAML and `{width=}` attributes on headings instead.

7.1.3 Cards

Every executable code cell in a dashboard automatically becomes a card. The cell's output (a plot, a table, text) fills the card. Use the `#| title:` cell option to give the card a header:

```
#| title: "Monthly Admissions"

ggplot(admissions, aes(x = month, y = n)) +
  geom_col()
```

7.1.4 Value Boxes

Value boxes are compact summary cards ideal for KPIs and headline numbers. They are created with the `#| content: valuebox` cell option. The cell should return a named list with `icon`, `color`, and `value` keys.

```
#| content: valuebox
#| title: "Total Patients"

list(
  icon  = "person-fill",
  color = "primary",
  value = nrow(patients)
)
```

Icons are Bootstrap Icons names (without the `bi-` prefix). Colors can be Bootstrap theme colors (`"primary"`, `"success"`, `"danger"`, `"warning"`, `"info"`) or any CSS color string.

7.1.5 Tabsets

Adding `.tabset` to a row makes each card within that row a tab instead of a side-by-side panel. Each cell's `#| title:` becomes the tab label.

````
## Row {.tabset}

```r
#| title: "Plot"

# plot code
```

```r
#| title: "Data"

# table code
```
````

Tabsets are useful when you have multiple views of the same data (e.g., a plot and the underlying table) and don't want them to compete for screen space.

7.2 Building a Dashboard

This section walks through building the overdose mortality dashboard step by step. All code blocks here are display-only; the executable versions live in the demo repository.

7.2.1 Project Structure

A Quarto dashboard lives in its own project directory with a `_quarto.yml` file and an `index.qmd`:

```
quarto-dashboard-demo/
├── _quarto.yml
├── index.qmd
└── od-data-clean.csv
```

The `_quarto.yml` sets the project type and default format options:

```yaml
project:
  type: default

format:
  dashboard:
    theme: cosmo
    scrolling: false
```

7.2.2 Loading Data

The setup chunk loads packages and reads the data. Compute any summary values you'll reuse across multiple cells here so they're available throughout the document.

```
#| label: setup
#| message: false
#| warning: false

library(readr)
library(dplyr)
library(ggplot2)
library(plotly)
library(DT)

od <- read_csv("od-data-clean.csv")

# Computed once, reused in value boxes and plots
total_state_years <- nrow(od)
year_min          <- min(od$year)
year_max          <- max(od$year)

top_state_recent <- od |>
  filter(year == max(year)) |>
  arrange(desc(pod)) |>
  slice(1)
```

The `pod` column is the proportion of all deaths attributable to overdose — the primary metric throughout the dashboard.

7.2.3 Value Boxes

Three value boxes occupy the first row. Each is a separate code cell with `#| content: valuebox` and a `#| title:`. Place them all under the same `## Row` heading so they share the row equally.

```
## Row {height=20%}
```

```
#| content: valuebox
#| title: "State-Year Observations"

list(
  icon  = "table",
  color = "primary",
  value = scales::comma(total_state_years)
)
```

```
#| content: valuebox
#| title: "Years Covered"

list(
  icon  = "calendar-range",
  color = "info",
  value = paste0(year_min, "–", year_max)
)
```

```
#| content: valuebox
#| title: "Highest OD % (Most Recent Year)"

list(
  icon  = "graph-up-arrow",
  color = "danger",
  value = paste0(
    top_state_recent$state, " — ",
    scales::percent(top_state_recent$pod, accuracy = 0.1)
  )
)
```

7.2.4 Static plots (ggplot2)

A static ggplot2 plot renders as a PNG image inside a card. It works well for PDF exports and environments without JavaScript, but users can't hover for values or zoom in.

```
#| title: "Overdose % Over Time — Selected States"

highlight_states <- c("WV", "DC", "OH", "KY")

od |>
  filter(state %in% highlight_states) |>
  ggplot(aes(x = year, y = pod, color = state, group = state)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 0.1)) +
  scale_x_continuous(breaks = year_min:year_max) +
  labs(
    x     = NULL,
    y     = "Overdose deaths (% of all deaths)",
    color = "State"
  ) +
  theme_minimal(base_size = 13)
```

7.2.5 Interactive plots (plotly)

Wrapping a ggplot in `plotly::ggplotly()` converts it to an interactive plot with hover tooltips, zoom, and pan — no layout changes needed.

```
#| title: "Overdose % Over Time — Selected States"

p <- od |>
  filter(state %in% highlight_states) |>
  ggplot(aes(x = year, y = pod, color = state, group = state)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 0.1)) +
  scale_x_continuous(breaks = year_min:year_max) +
  labs(
    x     = NULL,
    y     = "Overdose deaths (% of all deaths)",
    color = "State"
  ) +
  theme_minimal(base_size = 13)

ggplotly(p)
```

> **Tip**
>
> Swapping a static ggplot2 plot for `ggplotly()` is a one-line change that requires no modifications to the dashboard layout. Add interactivity where it helps users explore data; keep static plots where simplicity or export quality matters more.

For finer control over tooltips, use the `text` aesthetic and pass `tooltip = "text"` to `ggplotly()`:

```
bar_data |>
  ggplot(aes(
    x    = reorder(state, total_od),
    y    = total_od,
    fill = highlighted,
    text = paste0(state, ": ", scales::comma(total_od))
  )) +
  geom_col() +
  coord_flip()

ggplotly(p2, tooltip = "text")
```

7.2.6 Tables (DT)

`DT::datatable()` produces an interactive HTML table with built-in search, sorting, and pagination.

```
#| title: "Data"

od |>
  mutate(pod = scales::percent(pod, accuracy = 0.01)) |>
  rename(
    State          = state,
    Year           = year,
    `Total Deaths` = ndeath,
    `OD Deaths`    = nod,
    `OD %`         = pod
  ) |>
  DT::datatable(
    filter   = "top",
    options  = list(pageLength = 10, scrollX = TRUE),
    rownames = FALSE
  )
```

`filter = "top"` adds a per-column search box above each column header, allowing users to filter by state or year without any server-side code.

7.3 Previewing and Rendering

From inside the project directory:

```
quarto preview index.qmd   # live-reload preview in browser
quarto render index.qmd    # render to index.html
```

> **Note**
>
> `quarto preview` on a dashboard opens a standalone browser window showing the dashboard itself. If you also have the book open in another preview, they run independently — the dashboard preview is not embedded in the book.

7.4 Hosting on GitHub Pages

Because `format: dashboard` is incompatible with `format: html` (the book format used here), a dashboard cannot be a page in a Quarto book. The cleanest solution is to keep the dashboard in its own GitHub repository and deploy it separately.

7.4.1 Repository Setup

Create a new GitHub repository (e.g., `quarto-dashboard-demo`) and push the project files:

```
git init
git add .
git commit -m "Initial dashboard"
git remote add origin git@github.com:stephenturner/quarto-dashboard-demo.git
git push -u origin main
```

7.4.2 Publishing

`quarto publish gh-pages` renders the project and pushes the output to the `gh-pages` branch. GitHub Pages serves that branch automatically.

```
quarto publish gh-pages
```

After the first publish, GitHub Pages will be available at `https://stephenturner.github.io/quarto-dashboard-demo`. Subsequent publishes update the live site.

> **Tip**
>
> **Automate with GitHub Actions.** Instead of running `quarto publish` manually on your local machine, you can set up a GitHub Actions workflow that re-renders and deploys on every push to `main`. The Quarto documentation provides a ready-to-use workflow YAML — copy it into `.github/workflows/publish.yml` in your repository and GitHub will handle the rest.

8. Automation and Scheduling

Every Monday morning the respiratory surveillance report goes out. Someone opens the project, pulls the latest line list, renders the Quarto document, uploads the HTML, and emails the link to the program team. It takes twenty minutes if nothing goes wrong, longer if the data moved or a package updated over the weekend. It happens fifty-two times a year, and it depends entirely on a person remembering to do it.

The reporting chapter (Chapter 4.) showed how to build a report whose numbers update when the data changes, and the dashboards chapter (Chapter 7.) showed how to publish one to the web. This chapter is about the next step: making those things happen on a schedule, or in response to an event, without a person in the loop. The recurring weekly report becomes a job that runs at 6am whether or not anyone remembers, and the team finds out only if something breaks.

8.1 What to Automate (and What Not To)

Automation pays off when the same work happens the same way, repeatedly. A weekly surveillance render, a nightly data refresh from a public API (Chapter 13.), a validation check that should run every time a file lands: all of these are worth the up-front effort to set up, because the effort is amortized over dozens or hundreds of runs.

It pays off poorly, or not at all, for one-off analyses, for work that requires a human decision partway through, and for exploratory work where the whole point is that you do not yet know what you are going to do. A useful test: would you run this code, in this same way, more than a handful of times? If yes, automate it. If no, the time spent automating is time not spent on the analysis.

One caution before going further. Automation amplifies whatever you point it at. A correct, validated pipeline run a hundred times produces a hundred correct outputs; a subtly broken one produces a hundred broken outputs, faster than anyone can check them. Automation presumes the underlying work is already validated (Chapter 3.) and reproducible (Chapter 11.). Automating an unreliable pipeline does not save labor, it industrializes the errors.

8.2 Scheduling on Your Own Machine

The simplest form of automation is a saved script that the operating system runs on a schedule. The first requirement is that the script run without anyone clicking anything. Instead of opening the project and rendering interactively, put the render in an R script that can be run headlessly:

```
# render_report.R
quarto::quarto_render(
  input = "respiratory_report.qmd",
  execute_params = list(week = Sys.Date())
)
```

This is the same parameterized render from Section 4.6, just invoked from a script instead of by hand. From a terminal, `Rscript render_report.R` runs it start to finish with no interaction.

Once the work is a single command, the operating system can run it on a schedule. On macOS and Linux that scheduler is `cron`. A crontab entry is a schedule followed by a command; this one runs the render every Monday at 6am:

```
0 6 * * 1 Rscript /Users/you/projects/surveillance/render_report.R
```

The five fields before the command are minute, hour, day-of-month, month, and day-of-week. The `cronR` package can create and manage these entries from R if you would rather not edit a crontab by hand. On Windows the equivalent is Task Scheduler, which the `taskscheduleR` package wraps in the same way.

Machine scheduling is easy to set up and good enough for plenty of internal work, but it has real limits. The machine has to be powered on and awake at 6am Monday, which rules out a laptop that goes home in a bag on Friday. Paths must be absolute, because a scheduled job does not start in your project directory

or with your usual environment. And, most dangerously, if the job fails there is nobody watching: the report simply does not appear, and you find out when the program team asks where it is. The next two sections address each of these in turn.

8.3 Continuous Automation with GitHub Actions

For work whose code already lives on GitHub (Chapter 1.), GitHub Actions runs jobs on GitHub's machines instead of yours. A workflow is a YAML file in the `.github/workflows/` directory of the repository. It describes when to run (a trigger) and what to do (a sequence of steps), and GitHub spins up a fresh virtual machine each time to carry it out. Because the machine is GitHub's, it does not matter whether your laptop is open.

A workflow can be triggered by a push, by a manual button, or by a schedule. This one renders a Quarto report every Monday at 6am UTC, can also be run on demand, and publishes the result to GitHub Pages (Section 7.4):

```yaml
# .github/workflows/render.yml
name: Render surveillance report

on:
  schedule:
    - cron: "0 6 * * 1"      # every Monday at 06:00 UTC
  workflow_dispatch:         # also allow manual trigger

jobs:
  render:
    runs-on: ubuntu-latest
    permissions:
      contents: write
    steps:
      - uses: actions/checkout@v4

      - uses: quarto-dev/quarto-actions/setup@v2

      - uses: r-lib/actions/setup-r@v2

      - uses: r-lib/actions/setup-renv@v2

      - uses: quarto-dev/quarto-actions/publish@v2
        with:
          target: gh-pages
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

The steps read top to bottom: check out the repository, install Quarto and R, restore the exact package versions recorded by `renv` (Section 11.1), then render and publish. The maintained building blocks here are `r-lib/actions` for R setup and `quarto-actions` for rendering and publishing; the Quarto documentation on GitHub Actions, linked in Appendix A., is the reference for the publishing options.

> **Note**
>
> The `cron` schedule in a GitHub Actions workflow runs on UTC, not your local time zone, and does not adjust for daylight saving. A "6am Monday" job will land at a different local hour depending on the season. For a surveillance report this rarely matters; when it does, schedule against UTC deliberately.

When a job needs a credential (an API token as in Section 13.5, a database password), that secret never goes in the workflow file or anywhere else in the repository. GitHub stores it under the repository's **Settings → Secrets and variables**, and the workflow reads it as an environment variable at run time (the `secrets.GITHUB_TOKEN` reference above is the built-in example). This is the same discipline as the `.Renviron` pattern in Chapter 12.: credentials live outside version control, and code reads them from the environment.

8.4 Multi-Step Pipelines with targets

A single render is one step. Many real projects are a chain: read the raw extract, clean it, validate it, summarize, then render a report from the summary. When any one input changes, you want to rerun the steps that depend on it and skip the ones that do not. Rerunning everything every time wastes compute on a large dataset; rerunning the wrong subset is how stale intermediate files poison a result.

The `targets` package solves this by treating the pipeline as a dependency graph. You declare each step and what it depends on in a `_targets.R` file:

```
# _targets.R
library(targets)
tar_option_set(packages = c("dplyr", "readr"))

list(
  tar_target(raw_file, "data/raw/line_list.csv", format = "file"),
  tar_target(raw,      read_csv(raw_file)),
  tar_target(clean,    clean_line_list(raw)),
  tar_target(summary,  summarize_by_week(clean)),
  tar_target(report,   quarto::quarto_render("report.qmd"))
)
```

Running `tar_make()` builds the pipeline. The first run executes every step. On later runs, `targets` reruns only the steps whose inputs changed and reuses the cached results of the rest; if only the report template changed, the data is not re-read or re-cleaned. `tar_visnetwork()` draws the graph and colors each node by whether it is up to date, which is a fast way to see what a change will trigger before you run it.

For automation, this self-correcting behavior is the point. A scheduled job that calls `tar_make()` does the minimum work required to bring every output up to date, and an output is rebuilt exactly when, and only when, something it depends on has actually changed. This is the same reproducibility discipline framed as a project commitment in Chapter 18.; `targets` is the tool that enforces it mechanically. The RAPS book linked in Appendix A. goes much deeper on building pipelines this way.

8.5 Running in a Reproducible Environment

A scheduled job that relies on whatever packages happen to be installed on the machine is living on borrowed time. Eventually a package updates, a function's default changes, and the Monday report renders differently or fails outright. And because nobody changed the code, the cause is hard to find.

The fix is to pin the environment so that the scheduled run matches the one you tested. Recording dependencies with `renv` (Section 11.1) lets the job restore the exact package versions it was built against; the GitHub Actions example above did this with the `setup-renv` step. For jobs with system-level dependencies, or where you want the operating system itself fixed, running inside a Docker container (Section 11.2) freezes the whole environment, not just the R packages. Section 11.4 covers when each approach is worth the overhead. The principle is the same either way: an automated job should run in a defined environment, not an incidental one.

8.6 Knowing When It Breaks

A failure mode of automation is often silence instead of a crash. The dashboard still loads, the page still shows numbers, and nobody notices that the data feeding it stopped refreshing three weeks ago. A job that fails with an alert is recoverable; a job that fails silently erodes trust in everything the team publishes.

Build in failure signals from the start. GitHub Actions emails the repository owners when a workflow fails, which covers the basic case at no effort. For higher-stakes pipelines, add a step that posts to a Slack or Teams channel on failure so the alert lands where the team actually looks.

Validation belongs inside the automated job in addition to interactive work. A `pointblank` stop-on-fail check (Section 3.1) placed before the publish step turns a data problem into a hard failure that halts the run and triggers the alert, rather than letting bad numbers sail through to a published report. The validation gate is a circuit breaker: better a missing report and an alert than a confidently wrong one.

8.7 Sensitive Data and Automation

Most of this chapter's cloud examples assume the data is safe to send to GitHub's machines. Much public health data is not. Protected health information, restricted-use files, and data governed by a use agreement generally must not run on GitHub-hosted runners or any other shared cloud infrastructure, because doing so moves the data outside the boundary where its handling is authorized.

This does not rule out automation; it changes where the automation runs. The options are to use self-hosted runners that sit inside the agency's secure network (so GitHub orchestrates the job but the data never leaves), to schedule the job on an approved on-premises server with cron or Task Scheduler, or to use whatever managed analytics platform the agency already operates. The mechanics from earlier in this chapter still apply; only the location changes.

The decision of where a scheduled job may run is a governance decision before it is a technical one. The terms of the relevant data use agreement, the HIPAA considerations, and the secure-environment requirements in Chapter 19. determine what is permissible, and provisioning a scheduled job on a shared server is an IT request that should start early (Section 18.4). Work that out before building the pipeline, not after, because a pipeline built for the wrong environment cannot simply be moved into the right one.

8.8 Further Reading

- The `targets` user manual [4] is the definitive guide to building reproducible pipelines in R, covering branching, cloud storage, and high-performance computing well beyond the minimal example here.
- The `r-lib/actions` and `quarto-actions` repositories document the GitHub Actions building blocks for R and Quarto, including ready-made example workflows.
- *Building Reproducible Analytical Pipelines with R*, linked in Appendix A., connects scheduling, pipelines, and reproducible environments into a single framework and goes substantially deeper on each.

9. Package Loading

Loading packages in R is easy enough to do carelessly. A script accumulates ten or fifteen `library()` calls, and everything runs until it does not. An analyst's `select()` call starts returning the wrong results, and nothing errors to say so. Scrolling to the top of the script reveals that MASS was loaded after dplyr, giving `MASS::select()` precedence over `dplyr::select()`. No warning, just the wrong function running.

9.1 How R's search path works

When you call `library()`, R attaches the package to the *search path*, an ordered list of environments R searches when resolving a name. You can inspect it at any time:

```
search()
```

```
 [1] ".GlobalEnv"        "package:stats"     "package:graphics"
 [4] "package:grDevices" "package:utils"     "package:datasets"
 [7] ".env"              "package:methods"   "Autoloads"
[10] "package:base"
```

After loading a few packages, the path grows:

```
library(dplyr)
```

```

Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':

    filter, lag
```

```
The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(MASS)
```

```

Attaching package: 'MASS'
```

```
The following object is masked from 'package:dplyr':

    select
```

`package:MASS` now sits above `package:dplyr` in the search path. When R encounters the name `select`, it finds `MASS::select` first and stops. R prints a message when this happens:

```
# Attaching package: 'MASS'
# The following object is masked from 'package:dplyr':
#     select
```

In a script that loads fifteen packages, these messages scroll past. The script runs, and there is no error to flag that `select()` is now going to the wrong package.

⚠ R's masking message is easy to miss

R prints a note at load time when a package masks a function from an earlier-loaded one, but in a long block of `library()` calls the messages are easily overlooked. R replaces the earlier function without any error. The `conflicted` package, covered in Section 9.4, changes this: any ambiguous name throws an error rather than silently resolving to whichever package was loaded last.

9.2 Load only what you need

Every `library()` call extends the search path. A package block loaded by habit rather than intention is a reliable source of masking bugs in shared scripts.

Load a package with `library()` only if you call its functions throughout the script using bare names. If a package appears once or twice, use `::` instead (see Section 9.3).

Loaded by habit:

```
library(tidyverse)
library(lubridate)
library(janitor)
library(readxl)
library(openxlsx)
library(scales)
library(zoo)
library(forecast)
library(MASS)
library(car)
```

Trimmed to what the script uses:

```
library(dplyr)
library(ggplot2)
library(lubridate)
library(readr)
```

The shorter list documents dependencies. A reader can see what the script uses and trust the list is intentional.

Loading tidyverse

`library(tidyverse)` loads ggplot2, dplyr, tidyr, readr, purrr, tibble, stringr, forcats, and lubridate in a single call. That is a reasonable choice when you are genuinely using several of those packages. For a script that only needs dplyr and ggplot2, loading individual packages is more explicit about what you depend on and puts fewer packages on the search path.

You'll get the following message when you load tidyverse:

```
── Attaching core tidyverse packages ──── tidyverse 2.0.0 ──
✔ dplyr     1.2.0     ✔ readr     2.1.6
✔ forcats   1.0.1     ✔ stringr   1.6.0
✔ ggplot2   4.0.1     ✔ tibble    3.3.0
✔ lubridate 1.9.4     ✔ tidyr     1.3.1
✔ purrr     1.2.0
── Conflicts ────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package to force all conflicts to become errors
```

9.3 Use `::` for one-off functions

The `::` operator calls a function directly from a specific package without attaching it to the search path, so no masking occurs.

```
# Reading a single Excel file
data <- readxl::read_excel("surveillance_data.xlsx")

# Reading a large CSV with data.table's fast parser
raw <- data.table::fread("large_file.csv")

# Fitting a distribution without loading MASS
fit <- MASS::fitdistr(x, "normal")
```

`::` also documents function origin at the call site. `readxl::read_excel()` names its source; `read_excel()` does not.

When to use `::` versus `library()`:

- Use `library()` when you call functions from a package throughout the script. Loading `dplyr` makes sense if you are writing a dozen `filter()` and `mutate()` calls.
- Use `::` when you need one or two functions from a package and do not use it elsewhere.

> **i** `::` works even without `library()`
>
> You do not need to call `library()` before using `::`. As long as a package is installed, `readxl::read_excel()` will find and call it. You get the function without attaching the package or affecting the search path.

9.4 The `conflicted` package

The `conflicted` package [5] changes the default behavior: any ambiguous name throws an error, forcing an explicit choice rather than silently inheriting load order.

Install from CRAN:

```
install.packages("conflicted")
```

Without `conflicted`, load order silently determines which `select()` runs:

```
library(dplyr)
library(MASS)

# Silently uses MASS::select(), not dplyr::select()
result <- mtcars |> select(mpg, cyl)
```

```
Error in select(mtcars, mpg, cyl): unused arguments (mpg, cyl)
```

With `conflicted` loaded first, the same call throws an informative error:

```
library(conflicted)
library(dplyr)
library(MASS)

result <- mtcars |> select(mpg, cyl)
```

```
Error:
! [conflicted] select found in 2 packages.
Either pick the one you want with `::`:
• MASS::select
• dplyr::select
Or declare a preference with `conflicts_prefer()`:
```

```
• `conflicts_prefer(MASS::select)`
• `conflicts_prefer(dplyr::select)`
```

This is the point. Load order was deciding which `select()` to use; `conflicted` makes you decide.

Declare your preferences with `conflicts_prefer()`, once at the top of the script alongside your `library()` calls:

```
library(conflicted)
library(dplyr)
library(MASS)

conflicts_prefer(dplyr::select)
```

```
[conflicted] Will prefer dplyr::select over any other package.
```

```
conflicts_prefer(dplyr::filter)
```

```
[conflicted] Will prefer dplyr::filter over any other package.
```

```
# Now select() and filter() unambiguously mean dplyr
result <- mtcars |> select(mpg, cyl)
```

Use `conflict_scout()` to see every conflict in your session before any function calls:

```
conflict_scout()
```

```
3 conflicts
```

```
• `filter()`: dplyr
```

```
• `lag()`: dplyr and stats
```

```
• `select()`: dplyr
```

Run it after loading your packages to see what needs a preference declared.

`conflicted` works well alongside tidyverse

Load `conflicted` before `tidyverse`, then declare preferences for the most common conflicts:

```
library(conflicted)
library(tidyverse)
```

```
── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
✔ forcats   1.0.1     ✔ readr     2.1.6
✔ ggplot2   4.0.1     ✔ stringr   1.6.0
✔ lubridate 1.9.4     ✔ tibble    3.3.0
✔ purrr     1.2.0     ✔ tidyr     1.3.1
```

```
conflicts_prefer(dplyr::filter)
```

```
[conflicted] Removing existing preference.
[conflicted] Will prefer dplyr::filter over any other package.
```

```
conflicts_prefer(dplyr::select)
```

```
[conflicted] Removing existing preference.
[conflicted] Will prefer dplyr::select over any other package.
```

```
conflicts_prefer(dplyr::lag)
```

```
[conflicted] Will prefer dplyr::lag over any other package.
```

After these declarations, `filter()`, `select()`, and `lag()` unambiguously refer to dplyr. Any other conflict produces an error rather than a silent wrong answer.

9.5 Team implications

Package load order affects reproducibility across the team. The same script can produce different results on different machines when analysts have different packages installed or load them in a different order. `dplyr::select()` always means `dplyr::select()`, but `select()` depends on who loaded what and when.

Code review (see Chapter 20.) is a natural checkpoint for package hygiene. Reviewers should flag unnecessary `library()` calls and bare function names that commonly conflict. Both are easier to catch in review than when writing.

`renv` (see Section 11.1) ensures everyone is using the same package versions, but it does not determine which `select()` runs when both dplyr and MASS are loaded. `renv` and `conflicted` are complementary: one locks down versions, the other makes conflict resolution explicit.

The package block at the top of a shared script documents what the script depends on. A bloated list loaded by habit, combined with silent masking, means different analysts running the same script may not be running the same code.

10. Package Development

Most R users start writing functions to avoid repeating themselves. The same data cleaning steps appear in three scripts, so they get pulled into a shared helper. That helper gets sourced into a fourth script. A colleague asks for it and it gets emailed. Then it breaks on their machine because they're missing some dependency. At some point the question becomes: why not just make this a package?

My general rule is: If you do something more than once, write a function. If you write more than one function, make it a package.

An R package is the right unit of sharing for reusable code. It gives your functions a home with documentation, a test suite, versioning, and a clear installation path.

This chapter is a brief introduction to R package development. It is not a comprehensive guide, but it will give you the basics and point you to resources for learning more.

10.1 Why build a package?

The most visible benefit is sharing, but that undersells it. Packaging imposes structure that improves code before anyone else touches it.

Documentation is required, not optional. Every exported function must have a help page describing what each argument does, what the function returns, and what edge cases exist. A comment in a script is optional; `R CMD CHECK` will warn if documentation is missing. Writing it forces careful thinking about the function's interface, and once written, it is available to anyone who types `?your_function`.

Dependencies must be declared. A script accumulates `library()` calls by habit. A package requires you to enumerate exactly what it depends on in a `DESCRIPTION` file. When someone installs your package, those dependencies install automatically. There is no guessing about which packages need to be present. This is the same problem `renv` addresses for project-level reproducibility (see Section 11.1), but solved at the code level rather than the project level.

Sharing becomes a single command. A package on GitHub can be installed by anyone with `remotes::install_github("yourname/yourpkg")`. No files to transfer, no paths to configure, no sourcing step to explain. The package can also be pinned in a project's `renv` lockfile like any CRAN package.

Testing has a natural home. Package development pairs with unit testing through the testthat framework. A suite of passing tests is a concrete claim that the functions behave as documented. Finding that a change broke something during development is far cheaper than finding it later.

A public health team that builds a shared package of common utilities (data validation functions, standard plot themes, reporting helpers) gets documented interfaces, declared dependencies, and a single install command that works for anyone on the team.

10.2 The package development workflow

The core loop of package development (edit code, load and run it, run tests, update documentation, check) is well captured by the Posit package development cheat sheet at https://rstudio.github.io/cheatsheets/html/package-development.html.

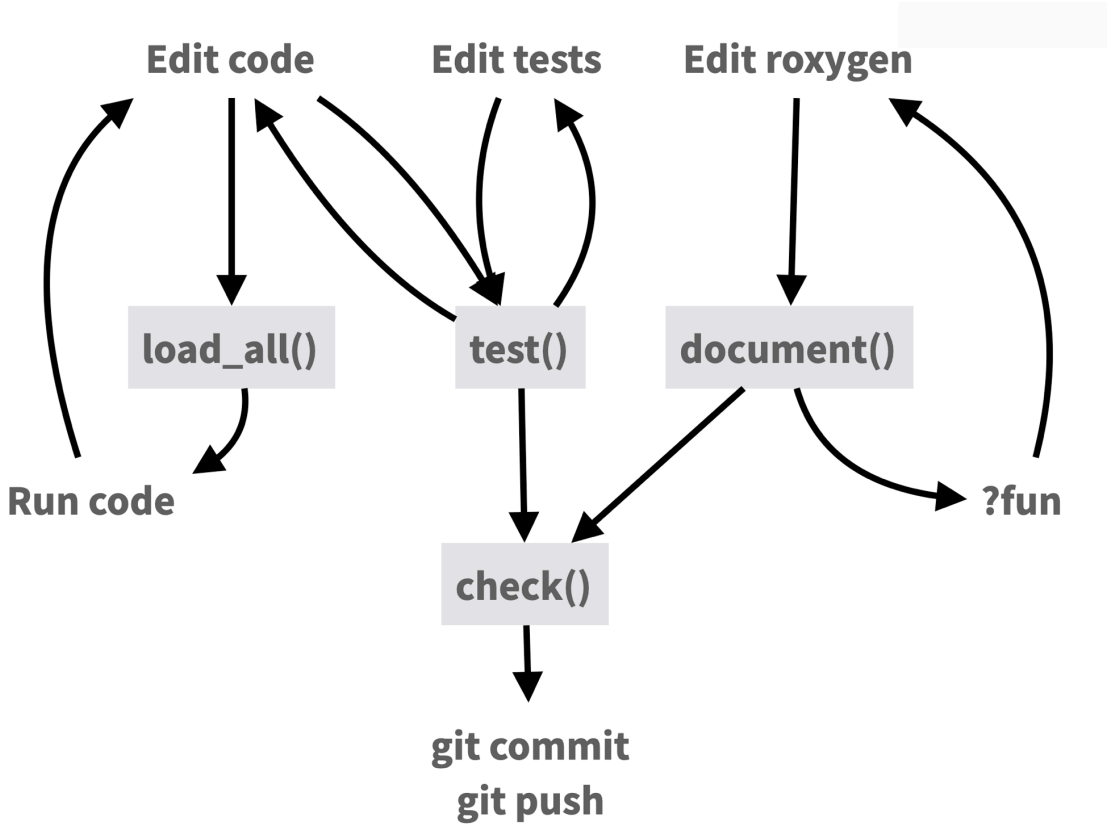


Figure 10.1: The package development workflow. Source: Posit Package Development Cheat Sheet, CC BY 4.0.

R package development is an entire topic in itself, and this section is just a brief introduction. Below I'll point you to some helpful resources.

10.3 Resources

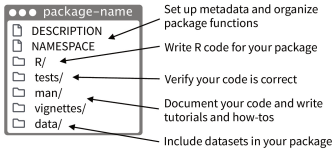
The definitive reference is *R Packages* (2nd ed.) by Hadley Wickham and Jennifer Bryan [6], available free online. It covers package structure, function documentation, testing, data, and the publication process.

The Posit Package Development cheat sheet is a quick reference for the devtools and usethis commands used throughout development.

Package Development : : CHEATSHEET

Package Structure

A package is a convention for organizing files into directories. This cheat sheet shows how to work with the 7 most common parts of an R package:



There are multiple packages useful to package development, including **usethis** which handily automates many of the more repetitive tasks. Install and load **devtools,** which wraps together several of these packages to access everything in one step.

Getting Started

Once per machine:

- Get set up with **use_devtools()** so **devtools** is always loaded in interactive R sessions

```
if (interactive()) {
   require("devtools", quietly = TRUE)
   # automatically attaches usethis
}
```

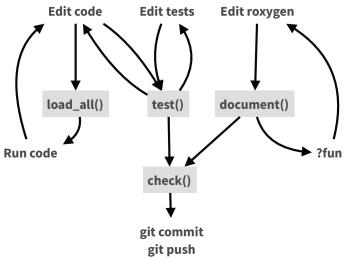
- **create_github_token()** — Set up GitHub credentials
- **git_vaccinate()** — Ignores common special files

Once per package:

- **create_package()** — Create a project with package scaffolding
- **use_git()** — Activate git
- **use_github()** — Connect to GitHub
- **use_github_action()** — Set up automated package checks

Having problems with git? Get a situation report with **git_sitrep()**.

Workflow



- **load_all()** (Ctrl/Cmd + Shift + L) — Load code
- **document()** (Ctrl/Cmd + Shift + D) — Rebuild docs and NAMESPACE
- **test()** (Ctrl/Cmd + Shift + T) — Run tests
- **check()** (Ctrl/Cmd + Shift + E) — Check complete package

R/

All of the R code in your package goes in R/. A package with just an R/ directory is still a very useful package.

- ☑ Create a new package project with **create_package**("path/to/name").
- ☑ Create R files with **use_r**("file-name").

- Follow the tidyverse style guide at **style.tidyverse.org**
- Click on a function and press **F2** to go to its definition
- Find a function or file with **Ctrl + .**

DESCRIPTION

The DESCRIPTION file describes your work, sets up how your package will work with other packages, and applies a license.

- ☑ Pick a license with **use_mit_license(), use_gpl3_license(), use_proprietary_license().**
- ☑ Add packages that you need with **use_package().**

Import packages that your package requires to work. R will install them when it installs your package.

use_package(x, type = "imports")

Suggest packages that developers of your package need. Users can install or not, as they like.

use_package(x, type = "suggests")

NAMESPACE

The NAMESPACE file helps you make your package self-contained: it won't interfere with other packages, and other packages won't interfere with it.

- ☑ Export functions for users by placing **@export** in their roxygen comments.
- ☑ Use objects from other packages with **package::object** or **@importFrom package object** (recommended) or **@import package** (use with caution).
- ☑ Call **document()** to generate NAMESPACE and **load_all()** to reload.

DESCRIPTION	NAMESPACE
Makes **packages** available	Makes **function** available
Mandatory	Optional (can use :: instead)
use_package()	**use_import_from()**

man/

The documentation will become the help pages in your package.

- ☑ Document each function with a roxygen block above its definition in R/. In RStudio, Code > Insert Roxygen Skeleton helps (Ctrl/Cmd + Alt + Shift + R).
- ☑ Document each dataset with roxygen block above the name of the dataset in quotes.
- ☑ Document the package with **use_package_doc().**
- ☑ Build documentation in man/ from Roxygen blocks with **document().**

vignettes/

- ☑ Create a vignette that is included with your package with **use_vignette().**
- ☑ Create an article that only appears on the website with **use_article().**
- ☑ Write the body of your vignettes in R Markdown.

Websites with pkgdown

- ☑ Use GitHub and **use_pkgdown_github_pages()** to set up pkgdown and configures an automated workflow using GitHub Actions and Pages.
- ☑ If you're not using GitHub, call **use_pkgdown()** to configure pkgdown. Then build locally with **pkgdown::build_site().**

tests/

- ☑ Set up test infrastructure with **use_testthat().**
- ☑ Create a test file with **use_test().**
- ☑ Write tests with **test_that()** and **expect_()**.
- ☑ Run all tests with **test()** and run tests for current file with **test_active_file().**
- ☑ See coverage of all files with **test_coverage()** and see coverage of current file with **test_coverage_active_file().**

ROXYGEN2

The **roxygen2** package lets you write documentation inline in your .R files with shorthand syntax.

- Add roxygen documentation as comments beginning with **#'**.
- Place a roxygen **@** tag (right) after **#'** to supply a specific section of documentation.
- Untagged paragraphs will be used to generate a title, description, and details section (in that order).

```
#' Add together two numbers
#'
#' @param x A number.
#' @param y A number.
#' @returns The sum of `x` and `y`.
#' @export
#' @examples
#' add(1, 1)
add <- function(x, y) {
  x + y
}
```

COMMON ROXYGEN TAGS

@description	@family	**@returns**
@examples	@inheritParams	@seealso
@examplesIf	**@param**	
@export	@rdname	

README.Rmd + NEWS.md

- ☑ Create a README and NEWS markdown files with **use_readme_rmd()** and **use_news_md().**

Expect statement	Tests
expect_equal()	Is equal? (within numerical tolerance)
expect_error()	Throws specified error?
expect_snapshot()	Output is unchanged?

```
test_that("Math works", {
  expect_equal(1 + 1, 2)
  expect_equal(1 + 2, 3)
  expect_equal(1 + 3, 4)
})
```

data/

- ☑ Record how a data set was prepared as an R script and save that script to data-raw/ with **use_data_raw().**
- ☑ Save a prepared data object to data/ with **use_data().**

Package States

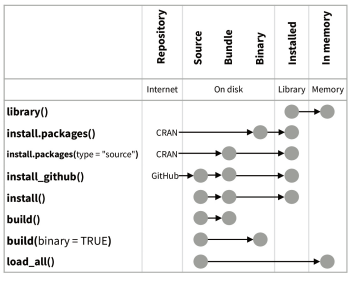
The contents of a package can be stored on disk as a:

- **source** - a directory with sub-directories (as shown in Package structure)
- **bundle** - a single compressed file (*.tar.gz*)
- **binary** - a single compressed file optimized for a specific OS

Packages exist in those states locally or remotely, e.g. on CRAN or on GitHub.

From those states, a package can be installed into an R library and then loaded into memory for use during an R session.

Use the functions below to move between these states.



Visit **r-pkgs.org** to learn much more about writing and publishing packages for R.

Figure 10.2: The Posit Package Development cheat sheet. Source: Posit, CC BY 4.0.

In January 2026, I gave a two-hour workshop on R package development in Positron covering this workflow end to end: package structure, writing and documenting functions, exported package data, programming with dplyr, unit tests with testthat, code coverage with covr, continuous integration with GitHub Actions, and a pkgdown documentation site. I also demonstrated using Positron Assistant with Claude in a few places..

All materials are available:

- Package documentation site: https://stephenturner.github.io/rpkgdemo/
- Full tutorial: https://stephenturner.github.io/rpkgdemo/articles/rpkgdemo
- Source code: https://github.com/stephenturner/rpkgdemo
- Recording: https://www.youtube.com/watch?v=hc_RwZx3wNE

https://www.youtube.com/watch?v=hc_RwZx3wNE

Finally, the Writing R Extensions manual is the official reference for package structure and metadata. It is a bit dry, but it is the source of truth for how packages work under the hood. I'd recommend starting with the R Packages book and asking your favorite AI for help when you run into trouble, but the R Extensions manual is the detailed authoritative guide and is there when you need it.

11. Reproducible Environments

Code that runs correctly today may fail six months from now, not because you changed anything, but because a package updated, R released a new version, or a system library changed underneath your analysis. This is sometimes called the "works on my machine" problem, and it is one of the most frustrating failure modes in data science work. Reproducible environments are the solution: explicit records of the exact software versions your code depends on, packaged in a way that others (and future you) can restore exactly.

This chapter covers two levels of environment management:

- **Package-level** with `renv`, which records and restores R package versions
- **System-level** with Docker and the Rocker project, which captures the entire computing environment including R itself, system libraries, and non-R tools
- **Bridging both** with `pracpac`, which containerizes R packages by combining `renv` dependency capture with Docker image generation

These tools exist on a spectrum of complexity and completeness. A good rule of thumb: start with `renv` for any analysis project involving collaborators, and reach for Docker when you need to share a full deployment or guarantee identical results across different operating systems.

11.1 Package Environments with renv

The `renv` package gives each R project its own private library. Instead of every project sharing a single system-wide package library (where upgrading a package for one project might break another), `renv` isolates dependencies per project and records the exact versions in a lockfile.

11.1.1 How renv Works

`renv` maintains three things:

1. **A project-private library** at `renv/library/` – packages installed here don't affect other projects
2. **A lockfile** (`renv.lock`) – a plain-text record of every package version used in the project
3. **An activation script** (`renv/activate.R`) – auto-loaded by `.Rprofile` to activate the project library when R starts in that directory

The lockfile is what makes sharing and restoring the environment possible. It records the package name, version, source (CRAN, Bioconductor, GitHub), and a hash for verification:

```
{
  "R": {
    "Version": "4.4.1",
    "Repositories": [
      {
        "Name": "CRAN",
        "URL": "https://packagemanager.posit.co/cran/latest"
      }
    ]
  },
  "Packages": {
    "dplyr": {
      "Package": "dplyr",
      "Version": "1.1.4",
      "Source": "Repository",
      "Repository": "CRAN",
      "Hash": "fedd9d00c2944ff00a0e2696ccf048ec"
    }
```

```
  }
}
```

This file should be committed to version control (see Chapter 1.). It's how collaborators and future you restore the same environment.

11.1.2 Core Workflow

Starting a new project:

```
renv::init()
```

This initializes `renv` in the current project: creates the private library, scans your scripts for `library()` and `require()` calls to discover dependencies, installs those packages, and writes the initial `renv.lock`.

After installing or updating packages:

```
# Install packages as usual
install.packages("ggplot2")

# or use renv's wrapper -- preferred
renv::install("tidyr")

# Then snapshot to record the new state
renv::snapshot()
```

`renv::snapshot()` updates `renv.lock` to reflect the current state of your project library. Run it whenever you add or update packages, then commit the updated lockfile.

Restoring an environment (e.g., on a new machine or after cloning a repo):

```
renv::restore()
```

This reads `renv.lock` and installs exactly the recorded versions. This is the key command for reproducibility – any collaborator who clones your repository and runs `renv::restore()` ends up with the identical package environment.

> Tip
>
> When you open a project with an existing `renv.lock`, `renv` will notice if your local library is out of sync with the lockfile and suggest running `renv::restore()`. You can also check status explicitly:
>
> ```
> renv::status()
> ```
>
> This reports any discrepancies between your installed packages and `renv.lock`.

Checking for package updates:

```
renv::update()      # update all packages to latest versions
renv::snapshot()    # then re-record the new state
```

11.1.3 Working as a Team

When collaborating on a project with `renv`:

1. One person initializes `renv` and commits `renv.lock`, `renv/activate.R`, and `.Rprofile` to the repository
2. Everyone else clones the repo and runs `renv::restore()` to set up their local environment
3. When anyone installs or updates a package, they run `renv::snapshot()` and commit the updated `renv.lock`

Commit these files; do not commit the `renv/library/` folder itself (add it to `.gitignore`; `renv` does this automatically). The library can be rebuilt from the lockfile; committing it would add thousands of binary files to your repository.

> **Note**
>
> `renv` uses a global package cache on each machine, so packages are only downloaded and installed once even if you use them in multiple projects. Restoring an environment the second time is nearly instant if the packages are already cached.

11.1.4 What renv Does Not Solve

`renv` captures R package versions but not:

- The **R version itself**: if collaborator A runs R 4.3 and collaborator B runs R 4.4, results may differ even with identical packages
- **System libraries**: packages that depend on compiled system libraries (e.g., `sf` needs GDAL, `xml2` needs libxml2) can still break if system libraries differ
- **Non-R software**: any external tools called from R (Python, command-line tools, etc.)
- **Namespace conflicts**: when multiple packages export functions with the same name, `renv` does not determine which one your script uses; that depends on load order. See Chapter 9. for how to manage this.

For version and system-level concerns, Docker provides a more complete solution.

11.2 Containers with Docker

Docker is a tool for packaging an entire computing environment (the operating system, system libraries, R, R packages, and any other software) into a portable, self-contained unit called a **container**. Containers run identically on any machine with Docker installed, regardless of the host operating system.

11.2.1 Key Concepts

Image: A read-only template that defines the environment. Think of it as a snapshot of a fully configured system.

Container: A running instance of an image. You can run multiple containers from the same image.

Dockerfile: A text file containing the instructions for building an image. Like a recipe: start from this base, install these packages, copy these files, run this command.

Registry: A place to store and share images. Docker Hub is the public default; organizations may run private registries.

11.2.2 Why Docker for R?

The limitations of `renv` described above (R version, system libraries, non-R software) are exactly what Docker addresses. A Docker image built with a specific R version and specific system libraries will behave identically on a laptop, a server, or a cloud VM, six months or six years from now, as long as the image is preserved.

Docker is especially useful for:

- **Deploying analyses to servers** – the same image that works locally runs in production (see Chapter 19. for working with IT on institutional deployment)
- **Sharing complete analyses** – collaborators run your container without installing anything
- **Pipelines that mix R with other tools** – Python, command-line tools, databases, and R all coexist in one image
- **Shiny apps** – containerized apps can be deployed anywhere without managing a server environment

11.2.3 The Rocker Project

Setting up Docker for R from scratch requires some Linux knowledge and wrestling around with installing R from source. The Rocker project solves this by providing a curated set of R Docker images that are well-maintained, regularly updated, and designed specifically for data science workflows.

11.2.3.1 The Versioned Stack

The Rocker versioned stack is built around pinned R versions, making it the right choice for reproducible work:

Image	Built on	Contents
`rocker/r-ver`	Ubuntu LTS	Base R at a specific version
`rocker/rstudio`	`r-ver`	Adds RStudio Server
`rocker/tidyverse`	`rstudio`	Adds tidyverse, devtools, remotes
`rocker/verse`	`tidyverse`	Adds TeX, Pandoc, Quarto
`rocker/geospatial`	`verse`	Adds spatial libraries (sf, terra, GDAL)

The versioned images accept an R version tag, which pins both R and the package repository snapshot to that time period:

```
# R 4.4.1 with RStudio Server
docker pull rocker/rstudio:4.4.1

# R 4.3.0 with tidyverse
docker pull rocker/tidyverse:4.3.0
```

> Tip
>
> Use a specific version tag rather than `latest` in production work. `rocker/rstudio:latest` will point to different R versions as new releases come out; `rocker/rstudio:4.4.1` always means exactly that version.

11.2.3.2 Running Rocker Containers

To start an RStudio Server session in your browser with R 4.4.1:

```
docker run --rm \
  -p 8787:8787 \
  -e PASSWORD=yourpassword \
  -v "$(pwd)":/home/rstudio/project \
  rocker/rstudio:4.4.1
```

Then open `http://localhost:8787` in a browser and log in with username `rstudio` and the password you set. The `-v` flag mounts your current directory into the container so you can access your local files.

Key flags: - `--rm` – remove the container when it exits (avoids accumulating stopped containers) - `-p 8787:8787` – map host port 8787 to container port 8787 - `-e PASSWORD=...` – set the RStudio login password - `-v host_path:container_path` – mount a local directory into the container

11.2.3.3 Writing a Dockerfile

For projects that need packages beyond what the base Rocker images provide, you write a Dockerfile that extends a Rocker image:

```
FROM rocker/tidyverse:4.4.1

# Install system dependencies (if needed)
RUN apt-get update && apt-get install -y \
    libssl-dev \
    && rm -rf /var/lib/apt/lists/*

# Install additional R packages
RUN R -e "install.packages(c('janitor', 'gt', 'gtsummary'))"
```

```
# Copy project files
COPY . /home/rstudio/project
```

Build and run the image:

```
docker build -t my-analysis:latest .
docker run --rm -p 8787:8787 -e PASSWORD=pass my-analysis:latest
```

> **Note**
>
> **Docker and renv together.** For the most complete reproducibility, use both: a Dockerfile that pins the R version (via a versioned Rocker base) and `renv` inside the container to pin package versions. The `renv.lock` file is copied into the image at build time, and `renv::restore()` installs the exact package versions during the build. This combination captures the full stack: OS, R, and packages.

11.3 pracpac: R Packaging with Docker

`pracpac` (Practical R Packaging with Docker) [7] bridges the R package development workflow and Docker. It provides a `usethis`-style interface that automates the generation of Docker configuration for R packages under development, using `renv` to capture dependencies.

While `renv` and basic Rocker Dockerfiles cover many use cases, `pracpac` is aimed at teams that have structured their analysis code as an R package and want to containerize that package, along with all its dependencies, for deployment or sharing.

11.3.1 What pracpac Does

`pracpac` automates a four-step process that would otherwise require manual effort:

1. **Dependency capture**: uses `renv` to snapshot all package dependencies matching the developer's current environment.
2. **Package building**: creates a source tarball (`.tar.gz`) of the R package.
3. **Dockerfile generation**: writes a Dockerfile that installs the captured dependencies and the package from source.
4. **Image building**: optionally builds the Docker image with version tags drawn from the package `DESCRIPTION` file.

The generated Dockerfile uses a Rocker base image, installs `renv` and `BiocManager`, restores dependencies from the `renv.lock`, and installs the package, mirroring the developer's environment inside the container.

11.3.2 Installation

```
install.packages("pracpac")
```

11.3.3 Workflow

11.3.3.1 Step 1: Set up your R package with renv

`pracpac` assumes your analysis is structured as an R package (with a `DESCRIPTION` file) and that `renv` is initialized:

```
renv::init()
# ... develop your package, install dependencies ...
renv::snapshot()
```

11.3.3.2 Step 2: Generate Docker configuration

From inside your R package project:

```
library(pracpac)
use_docker()
```

This creates a `docker/` subdirectory containing three artifacts:

```
your-package/
├── DESCRIPTION
├── R/
├── renv.lock
└── docker/
    ├── Dockerfile
    ├── renv.lock       # copy of the project lockfile
    └── your-package_1.0.0.tar.gz   # built source package
```

11.3.3.3 Step 3: Build the Docker image

Either pass `build = TRUE` to `use_docker()` to do everything in one step, or build separately:

```
# One-step: generate and build
use_docker(build = TRUE)

# Two-step: generate first, inspect/customize, then build
use_docker()
# ... optionally edit docker/Dockerfile ...
build_image()
```

The two-step approach is useful when you want to customize the generated Dockerfile before building, for example to add system library dependencies or additional non-R tools.

11.3.3.4 Step 4: Run and share the image

```
# Run the container
docker run --rm -it your-package:1.0.0

# Share via Docker Hub or a private registry
docker push your-org/your-package:1.0.0
```

The image tag is derived from the package version in `DESCRIPTION`, so image versions stay in sync with package versions automatically.

> **Tip**
>
> `pracpac` is particularly useful for deploying R-based pipelines alongside non-R tools. After `use_docker()` generates the initial Dockerfile, you can extend it to install Python, command-line bioinformatics tools, or any other software your pipeline needs. The R dependencies are already handled by `renv`; you just add the additional layers on top.

11.3.4 Use Cases

Analysis pipelines. Structure your analysis as an R package, containerize with `pracpac`, and deploy to a server or cloud environment with a guarantee that the environment is identical to your development machine.

Shiny applications. An R package that contains a Shiny app can be containerized with `pracpac`, then deployed to any Docker-capable hosting environment.

Reproducible publications. Sharing a Docker image alongside a paper or report means reviewers and readers can reproduce your results exactly, regardless of what software is installed on their machine.

Mixed-language pipelines. If your R package calls Python scripts or command-line tools, customize the generated Dockerfile to add those dependencies. The result is a single container that runs the complete pipeline.

11.4 Choosing Your Approach

These tools are complementary, not competing. The right level of environment management depends on how much isolation and portability you need:

Scenario	Recommended approach
Solo project, short duration	`renv` only
Team collaboration on analysis code	`renv` + commit `renv.lock` to git
Need identical results across OS and R versions	`renv` + Docker (Rocker base)
Deploying an analysis or app to a server	Docker (Rocker base)
Analysis structured as an R package	`pracpac`
Pipeline mixing R with other tools	`pracpac` with customized Dockerfile

The most common entry point for DSTT teams is `renv`: add it to any collaborative project, commit the lockfile, and let teammates restore the environment with `renv::restore()`. This alone eliminates most "it doesn't work on my machine" problems. Docker and `pracpac` become relevant when you need to share a fully self-contained environment or move an analysis into production.

11.5 Further Reading

The tools covered in this chapter (`renv`, Docker/Rocker, and `pracpac`) are the most practical starting points for most teams. The reproducibility landscape is broader, however, and a few other tools are worth knowing about, specifically **Nix and rix**.

Nix is a package manager built around the principle of fully reproducible, declarative builds. Unlike `renv` (which captures R package versions) or Docker (which captures a container image), Nix works at the level of the entire software supply chain: every package, system library, compiler, and tool is described by a precise specification and built from source in an isolated environment. The result is a level of reproducibility that is difficult to achieve any other way.

For R users, Nix's power is most accessible through `rix`, an rOpenSci package developed by Bruno Rodrigues and Philipp Baumann. `rix` provides an R-native interface for generating Nix environments: you specify the R version, packages, and system dependencies in R code, and `rix` writes the `default.nix` file that Nix uses to build the environment:

```
library(rix)

rix(
  r_ver = "4.4.1",
  r_pkgs = c("dplyr", "ggplot2", "tidyr"),
  system_pkgs = NULL,
  git_pkgs = NULL,
  ide = "rstudio",
  project_path = "."
)
```

This generates a `default.nix` file that, when built with Nix, produces an environment with exactly that R version and those packages, down to the system library level, without Docker, and with no mutable state that can drift over time.

`rix` is a compelling option if your team is willing to install Nix and invest in learning its model. The payoff is reproducibility that is more rigorous than `renv` alone and less operationally complex than Docker for day-to-day development. Bruno Rodrigues's book *Building Reproducible Analytical Pipelines with R* covers this approach in depth.

12. Databases from R

Spreadsheets and CSV files work well for small datasets, but many public health agencies store their surveillance data in relational databases: SQL Server, Oracle, PostgreSQL, or similar systems. These databases are managed by IT, shared across teams, and updated continuously as new data arrives. The data never leaves the server; analysts connect, run a query, and pull back only the rows and columns they need.

This chapter covers how to work with databases from R. The examples use DuckDB, an in-process analytical database that requires no server setup and installs as an R package. DuckDB is practical for local analysis of large files, but the same patterns (DBI, dbplyr) apply to institutional databases like SQL Server and PostgreSQL — only the connection call changes. The chapter ends with connection patterns for those remote systems. Data that lives behind a web API rather than in a database you can connect to (CDC's open data portal, the Census Bureau) is covered in Chapter 13..

> **Note**
>
> Getting access to an institutional database involves IT, data stewards, and sometimes a data use agreement. That process is covered in Chapter 19.. This chapter covers what to do once access is in hand.

12.1 DBI: The Common Interface

The `DBI` package defines a consistent interface for interacting with databases from R. Every database backend (DuckDB, SQL Server, PostgreSQL, SQLite) provides its own driver package, but the functions you call are the same regardless of which system you are connected to.

```
install.packages(c("DBI", "duckdb", "duckplyr", "dbplyr", "odbc"))
```

The four core DBI functions:

Function	What it does
`dbConnect()`	Open a connection to a database
`dbGetQuery()`	Run a SQL `SELECT` and return results as a data frame
`dbExecute()`	Run SQL that modifies data (no results returned)
`dbDisconnect()`	Close the connection

A connection always needs to be closed when you are done. The safest pattern is `on.exit()`, which runs the disconnect even if an error occurs before the end of the block:

```
con <- dbConnect(duckdb::duckdb())
on.exit(dbDisconnect(con), add = TRUE)

# ... work with the database ...
```

12.2 DuckDB

DuckDB is an in-process analytical database designed for fast queries over large datasets. It runs inside your R session with no separate server, installs as a single package, and can read CSV and Parquet files directly without loading them into R first. It is a good fit for local analysis of data that is too large to

work with comfortably as an R data frame, and its DBI interface is identical to remote databases, so code written against DuckDB transfers to SQL Server or PostgreSQL with only the connection call changed.

12.2.1 Setting up

Connect with `dbConnect()`. An in-memory database (which disappears when the connection closes) is the default:

```
library(DBI)
library(duckdb)

con <- dbConnect(duckdb())
```

To persist the database to disk — so it survives between sessions — pass a file path:

```
con <- dbConnect(duckdb("surveillance.duckdb"))
```

12.2.2 Loading data

Use `dbWriteTable()` to load an R data frame into a DuckDB table. The examples in this chapter use a simulated weekly influenza surveillance dataset for Virginia counties, 2021–2023.

```
set.seed(42)
counties <- c(
  "Fairfax",
  "Arlington",
  "Loudoun",
  "Prince William",
  "Alexandria",
  "Chesterfield",
  "Henrico",
  "Richmond",
  "Roanoke",
  "Virginia Beach"
)
pop <- c(
  1150000,
  238000,
  420000,
  460000,
  155000,
  340000,
  330000,
  230000,
  100000,
  460000
)

flu <- expand.grid(
  week = 1:52,
  year = 2021:2023,
  county = counties,
  stringsAsFactors = FALSE
)
flu$cases <- pmax(0L, round(rnorm(nrow(flu), mean = 15, sd = 10)))
flu$population <- rep(pop, each = 156)

nrow(flu)
```

```
[1] 1560
```

```
dbWriteTable(con, "flu", flu, overwrite = TRUE)
```

To see which tables are in the database:

```
dbListTables(con)
```

```
[1] "flu"
```

12.2.3 Querying with SQL

`dbGetQuery()` runs a SQL `SELECT` and returns the result as a data frame:

```
dbGetQuery(
  con,
  "
  SELECT year, SUM(cases) AS total_cases
  FROM flu
  GROUP BY year
  ORDER BY year
"
)
```

```
  year total_cases
1 2021        7852
2 2022        7802
3 2023        7732
```

DuckDB can also read CSV and Parquet files directly using SQL — without loading them into R first:

```
# Read a CSV directly in a query
dbGetQuery(con, "SELECT * FROM read_csv('data/flu-2024.csv') LIMIT 5")

# Read a folder of Parquet files
dbGetQuery(con, "SELECT COUNT(*) FROM read_parquet('data/flu-*.parquet')")
```

12.3 dbplyr: dplyr Syntax on Any Database

Writing SQL by hand is workable for simple queries but awkward for anything that requires multiple joins, filters, and aggregations. The `dbplyr` package lets you write standard dplyr code against a database connection — it translates your code to SQL automatically.

12.3.1 Lazy tables

`tbl()` creates a reference to a database table. The result looks like a data frame but nothing has been pulled into R yet:

```
library(dplyr)
library(dbplyr)

flu_tbl <- tbl(con, "flu")
flu_tbl
```

```
# Source:   table<flu> [?? x 5]
# Database: DuckDB 1.4.4 [root@Darwin 25.5.0:R 4.5.2/:memory:]
    week  year county  cases population
   <int> <int> <chr>   <dbl>      <dbl>
 1     1  2021 Fairfax    29    1150000
 2     2  2021 Fairfax     9    1150000
```

```
 3     3  2021 Fairfax    19    1150000
 4     4  2021 Fairfax    21    1150000
 5     5  2021 Fairfax    19    1150000
 6     6  2021 Fairfax    14    1150000
 7     7  2021 Fairfax    30    1150000
 8     8  2021 Fairfax    14    1150000
 9     9  2021 Fairfax    35    1150000
10    10  2021 Fairfax    14    1150000
# ℹ more rows
```

12.3.2 Writing queries with dplyr

dplyr verbs work on database tables the same way they work on data frames. The key difference is that operations are lazy: they accumulate until you ask for the results with `collect()`.

```
q <- flu_tbl |>
  filter(year == 2023) |>
  group_by(county) |>
  summarize(
    total_cases = sum(cases, na.rm = TRUE),
    rate_per_100k = round(
      sum(cases, na.rm = TRUE) / first(population) * 100000,
      1
    )
  ) |>
  arrange(desc(rate_per_100k))

q
```

```
# Source:     SQL [?? x 3]
# Database:   DuckDB 1.4.4 [root@Darwin 25.5.0:R 4.5.2/:memory:]
# Ordered by: desc(rate_per_100k)
   county         total_cases rate_per_100k
   <chr>                <dbl>         <dbl>
 1 Roanoke                709          709
 2 Alexandria             690          445.
 3 Richmond               851          370
 4 Arlington              836          351.
 5 Chesterfield           887          261.
 6 Henrico                674          204.
 7 Loudoun                845          201.
 8 Prince William         816          177.
 9 Virginia Beach         768          167
10 Fairfax                656           57
```

Notice the output header says `Source: SQL [?? x 3]` — the query has not run yet. Call `collect()` to execute and bring results into R:

```
collect(q)
```

```
# A tibble: 10 × 3
   county         total_cases rate_per_100k
   <chr>                <dbl>         <dbl>
 1 Roanoke                709          709
 2 Alexandria             690          445.
 3 Richmond               851          370
 4 Arlington              836          351.
 5 Chesterfield           887          261.
 6 Henrico                674          204.
```

```
 7 Loudoun                845          201.
 8 Prince William         816          177.
 9 Virginia Beach         768          167
10 Fairfax                656           57
```

12.3.3 Inspecting generated SQL

`show_query()` reveals the SQL that dbplyr would send to the database. This is useful for debugging, learning SQL, or handing a query off to a colleague who prefers SQL:

```
show_query(q)
```

```
<SQL>
SELECT
  county,
  SUM(cases) AS total_cases,
   ROUND_EVEN((SUM(cases) / FIRST(population)) * 100000.0, CAST(ROUND(1.0, 0) AS
INTEGER)) AS rate_per_100k
FROM (
  SELECT flu.*
  FROM flu
  WHERE ("year" = 2023.0)
) q01
GROUP BY county
ORDER BY rate_per_100k DESC
```

The generated SQL is valid and efficient. For most queries, you never need to write SQL directly — but the ability to inspect it means you always know what is happening.

> **Tip**
>
> Pull as little data as possible. Filter, aggregate, and join on the database side before calling `collect()`. Pulling a million rows to R and then filtering in R is far slower than filtering in SQL and pulling only the rows you need.

12.4 duckplyr: DuckDB as a dplyr Backend

`duckplyr` takes a different approach. Instead of requiring an explicit connection, it makes DuckDB the execution engine for ordinary dplyr operations on data frames. You convert a data frame once with `as_duckdb_tibble()`, and subsequent dplyr operations run through DuckDB rather than R's default in-memory engine.

```
library(duckplyr)

flu_duck <- as_duckdb_tibble(flu)
```

The result looks and behaves like a tibble, but dplyr verbs execute via DuckDB:

```
flu_duck |>
  filter(year == 2023) |>
  group_by(county) |>
  summarize(
    total_cases = sum(cases),
    rate_per_100k = round(sum(cases) / first(population) * 100000, 1)
  ) |>
  arrange(desc(rate_per_100k))
```

```
# A tibble: 10 × 3
   county         total_cases rate_per_100k
   <chr>                <dbl>         <dbl>
 1 Roanoke                709          709
 2 Alexandria             690          445.
 3 Richmond               851          370
 4 Arlington              836          351.
 5 Chesterfield           887          261.
 6 Henrico                674          204.
 7 Loudoun                845          201.
 8 Prince William         816          177.
 9 Virginia Beach         768          167
10 Fairfax                656           57
```

12.4.1 How duckplyr differs from dbplyr

	dbplyr	duckplyr
Connection	Explicit (`dbConnect`)	None needed
Data location	Database table	R data frame
Use case	Query a database	Speed up work on large data frames
Result	Lazy (need `collect()`)	Returned directly

duckplyr is most useful when you have a large data frame already in R and want faster aggregations without the overhead of a separate database connection. dbplyr is the right tool when data lives in a database and you want to query it without pulling it all into R.

> **Note**
>
> duckplyr falls back to standard dplyr automatically when it encounters a verb or expression it cannot execute through DuckDB. The output is always correct — the question is only whether the fast path was used. You can see fallback events with `duckplyr::fallback_review()`.

12.5 Connecting to Institutional Databases

In practice, surveillance data often lives on a managed SQL Server, Oracle, or PostgreSQL server that IT administers. The `odbc` package provides R drivers for these systems. The connection syntax differs from DuckDB, but everything else — DBI functions, dbplyr queries — works identically.

12.5.1 SQL Server

The most common database at state and local health departments. Requires the ODBC Driver for SQL Server, which IT typically installs on managed machines:

```
library(odbc)

con <- dbConnect(
  odbc(),
  driver = "ODBC Driver 18 for SQL Server",
  server = "db.health.state.gov",
  database = "surveillance",
  uid = Sys.getenv("DB_USER"),
  pwd = Sys.getenv("DB_PASSWORD"),
  encrypt = "yes"
)
```

12.5.2 PostgreSQL

Common in cloud deployments and some state agencies:

```
con <- dbConnect(
  odbc(),
  driver = "PostgreSQL Unicode",
  server = "db.health.state.gov",
  database = "surveillance",
  uid = Sys.getenv("DB_USER"),
  pwd = Sys.getenv("DB_PASSWORD")
)
```

12.5.3 Storing credentials safely

Never put database credentials in code or commit them to version control. Use environment variables instead. Store them in a project-level `.Renviron` file (which your `.gitignore` should exclude):

```
# .Renviron  — do not commit this file
DB_USER=yourname
DB_PASSWORD=yourpassword
```

Retrieve them in code with `Sys.getenv()`, as shown in the connection examples above. An `.Renviron` file in the project root is loaded automatically when R starts in that directory. This keeps credentials out of scripts and repositories while making them available to anyone with appropriate access to the file.

> ⚠ Warning
>
> Add `.Renviron` to your `.gitignore` before adding credentials to it. A credential committed to a repository is a credential that may be exposed.

12.5.4 Once connected, everything is the same

After the connection is open, all DBI and dbplyr patterns work identically regardless of which database system you are connected to:

```
# Works on SQL Server, PostgreSQL, DuckDB, etc.
cases_tbl <- tbl(con, "flu_cases")

cases_tbl |>
  filter(year == 2023, county == "Fairfax") |>
  arrange(week) |>
  collect()
```

12.6 Practical Guidance

Always close connections. Use `on.exit(dbDisconnect(con), add = TRUE)` immediately after opening a connection so it closes even if an error interrupts the script.

Use dbplyr instead of writing SQL. Unless you need a SQL feature that dbplyr does not support, dplyr verbs are easier to read, easier to debug, and work across database backends without modification.

Filter on the database side. Every row you pull into R costs time and memory. Apply `filter()`, `group_by()`, and `summarize()` before `collect()` so you bring back only the result, not the raw data.

Check the data types after `collect()`. Databases and R sometimes disagree on types (dates, integers, NULLs vs. `NA`). Validate with `str()` or `glimpse()` after the first `collect()` and add type coercions to your setup block if needed.

13. APIs and Public Data Sources

Not every analysis starts with data your agency collects. The denominators for your rates come from the Census Bureau, the national counts you compare against come from CDC, and both live on someone else's server, published for exactly this kind of use.

The usual way to get that data is to browse to the portal, click through some filters, hit Export, and download a CSV, which works exactly once. Six months later, nobody remembers which filters were applied or whether the provider has revised the numbers since, and the download is the one step of the analysis that cannot be re-run.

The fix is to script the download, and web APIs make that possible. This chapter covers how to pull data from public APIs in R: using a dedicated package when one exists, building requests with `httr2` when one does not, handling API keys, and structuring the pull so the rest of the analysis stays reproducible.

> **Note**
>
> This chapter is about *public* data sources, where the data is intended for download and the main concerns are reproducibility and politeness. Getting access to your agency's internal databases is covered in Chapter 12., and the governance process around restricted data in Chapter 19..

13.1 What an API Request Looks Like

An API (application programming interface), for our purposes, is a URL that returns data instead of a web page. Here is a real one. CDC publishes NNDSS weekly notifiable disease counts on its open data portal, and this URL asks for the 2024 pertussis records:

```
https://data.cdc.gov/resource/x9gk-5huc.json?label=Pertussis&year=2024
```

The pieces: `data.cdc.gov` is the portal, `x9gk-5huc` identifies the dataset, `.json` is the response format, and everything after the `?` is a set of query parameters that filter the result. Paste it into a browser and you get back JSON, a plain-text format for structured data that looks like this:

```
library(jsonlite)

flu_json <- '[
  {"county": "Fairfax",   "week": "2024-01-06", "cases": "23"},
  {"county": "Arlington", "week": "2024-01-06", "cases": "41"}
]'

fromJSON(flu_json)
```

```
     county       week cases
1   Fairfax 2024-01-06    23
2 Arlington 2024-01-06    41
```

The `jsonlite` package converts JSON to a data frame, and for a flat, rectangular response like this the conversion is automatic. Notice that `cases` came back as a character column, quotes and all. Many APIs, including CDC's, return every field as a string regardless of what it contains. More on that in Section 13.7.

13.2 Where Public Health Data Lives

A few sources cover most of what public health teams pull from outside their agency:

Source	What's there	How to access it
data.cdc.gov	CDC's open data portal: NNDSS, provisional deaths, vaccination coverage, BRFSS, and hundreds more	Socrata API; `RSocrata` or `httr2`
US Census Bureau	ACS, decennial census, population estimates (your rate denominators)	`tidycensus` (free API key required)
CDC WONDER	Mortality, natality, and other query systems	Web query tool; an API that accepts XML-formatted requests
HealthData.gov	Cross-agency HHS datasets	Varies by dataset
State and local open data portals	Many jurisdictions run their own Socrata or CKAN portals	Same Socrata patterns as data.cdc.gov

13.3 Use a Package When One Exists

Before writing any request code, check whether someone has already wrapped the API in an R package. A good wrapper handles authentication, pagination, and type conversion for you, and it encodes knowledge about the API's quirks that you would otherwise learn the hard way.

The clearest example is `tidycensus`, which wraps the Census Bureau APIs. Getting county population denominators for Virginia is a few lines:

```
library(tidycensus)

# One-time setup: request a free key at
# https://api.census.gov/data/key_signup.html
census_api_key("YOUR_KEY_HERE", install = TRUE)

va_pop <- get_acs(
  geography = "county",
  state = "VA",
  variables = c(population = "B01003_001"),
  year = 2023
)
```

The result arrives as a tidy data frame with proper types, margins of error included. Kyle Walker's book *Analyzing US Census Data* [8], freely available online, covers the census APIs and `tidycensus` in depth.

For Socrata portals like data.cdc.gov, the `RSocrata` package (maintained by the City of Chicago) does the same job. Point it at the dataset URL, filters and all:

```
library(RSocrata)

pertussis <- read.socrata(
  "https://data.cdc.gov/resource/x9gk-5huc.json?label=Pertussis&year=2024"
)
```

`read.socrata()` pages through the full result automatically and converts date fields, two things you would otherwise handle yourself.

13.4 Building Requests with httr2

When no package exists, the `httr2` package is the general-purpose tool for talking to APIs from R. You build a request object piece by piece, then perform it.

```
library(httr2)
```

```
req <- request("https://data.cdc.gov/resource/x9gk-5huc.json") |>
  req_url_query(
    `$where` = "label='Pertussis' AND year='2024'",
    `$limit` = 5000
  )

req
```

```
<httr2_request>
GET  https://data.cdc.gov/resource/x9gk-5huc.json?%24where=label%3D%27Pertussis%27%20
AND%20year%3D%272024%27&%24limit=5000
Body: empty
```

Printing the request shows the method and the assembled URL. Nothing has been sent yet; `req_url_query()` also took care of encoding the spaces and quotes in the `$where` clause. The `$where` and `$limit` parameters are part of Socrata's query language (SoQL), which supports SQL-like filtering, selecting, and ordering directly in the URL, so the server does the subsetting and you download only what you need. The same idea as filtering before `collect()` in Chapter 12..

Performing the request and parsing the response:

```
resp <- req_perform(req)
pertussis <- resp_body_json(resp, simplifyVector = TRUE)
```

`simplifyVector = TRUE` hands the JSON to `jsonlite` for the same automatic data-frame conversion shown earlier.

13.4.1 Pagination

APIs cap how many rows one request returns, and the cap is easy to miss. Socrata's default is 1,000 rows: ask for a dataset with 50,000 rows and you get the first 1,000, with no error and no warning. The `$limit` parameter raises the cap, and `$offset` requests subsequent pages. `httr2` can iterate the pages for you:

```
resps <- req_perform_iterative(
  req,
  next_req = iterate_with_offset(
    "$offset",
    start = 0,
    offset = 5000,
    resp_complete = \(resp) length(resp_body_json(resp)) == 0
  ),
  max_reqs = Inf
)
```

Whatever tool you use, check the row count of what came back against what you expected. A suspiciously round number like exactly 1,000 or exactly 50,000 usually means you hit a cap.

13.4.2 Retries and politeness

Public APIs fail transiently and rate-limit heavy users. Two `httr2` functions handle both concerns declaratively:

```
req <- req |>
  req_retry(max_tries = 3) |>
  req_throttle(capacity = 30, fill_time_s = 60)
```

`req_retry()` retries automatically on transient failures with increasing wait times. `req_throttle()` caps your request rate (here, 30 requests per minute) so a loop over many queries does not hammer the server. Both matter more in automated pipelines (Chapter 8.), where nobody is watching the requests go out.

13.5 API Keys and Tokens

Many APIs require a key, and others work better with one. The Census API requires a free key. Socrata portals work anonymously but throttle anonymous traffic aggressively; a free app token moves you to a much more generous rate limit. Keys are credentials, and the discipline is the same as for database passwords in Section 12.5: keep them in `.Renviron`, read them with `Sys.getenv()`, and never commit them.

```
# .Renviron (git-ignored):  SOCRATA_APP_TOKEN=xxxxxxxxxx

req <- request("https://data.cdc.gov/resource/x9gk-5huc.json") |>
  req_headers(`X-App-Token` = Sys.getenv("SOCRATA_APP_TOKEN")) |>
  req_url_query(`$where` = "label='Pertussis' AND year='2024'")
```

In a GitHub Actions pipeline, the token goes in the repository's secrets and reaches the job as an environment variable, exactly as described in Section 8.3.

13.6 Separate the Pull from the Analysis

Do not put an API call at the top of an analysis script. Put it in its own script, write the raw response to `data/raw/` with a dated filename (Section 2.3), and have the analysis read the file.

```
# 01-pull-nndss.R
resp <- req_perform(req)

writeLines(
  resp_body_string(resp),
  file.path("data", "raw", paste0(Sys.Date(), "-nndss-pertussis.json"))
)
```

The main thing this buys is reproducibility. An API reflects the data as of right now, and many public health sources revise: CDC's provisional death counts, for example, are updated for recent weeks as certificates arrive, so the same query run a month apart returns different numbers. The dated raw file records exactly what you pulled and when, and that file, not the live API, is the input your analysis can be reproduced from. Separating the pull also means the analysis renders offline and keeps working when the API is down or the dataset moves, and iterating on a report no longer re-downloads the same data on every render.

In an automated pipeline, the pull becomes its own step, scheduled with the tools in Chapter 8. or declared as a `targets` node (Section 8.4) so downstream steps rerun only when a new file lands.

13.7 Validate What Comes Back

Data from an API deserves the same skepticism as any other input, and it fails in two ways of its own.

The first is types. As the JSON example at the start of the chapter showed, numeric fields often arrive as character. Coerce them explicitly, immediately after reading:

```
library(dplyr)
pertussis <- pertussis |>
  mutate(
    year = as.integer(year),
    week = as.integer(week),
    m1 = as.integer(m1) # current-week case count
  )
```

Explicit coercion beats automatic guessing because it fails loudly: if a count column suddenly contains `"suppressed"`, `as.integer()` warns and produces `NA`, which your validation will catch. Silent guessing just gives you a character column and a broken sum three steps later.

The second is the schema, which can change under you. API responses often omit empty fields entirely. In the NNDSS data, a record with no current-week count simply has no `m1` element, so a pull where every record is empty for some field has no such column at all. Providers also add, rename, and retire columns

without ceremony. A pull script that ran cleanly for a year can start returning something structurally different.

Both problems have the same answer as in Chapter 3.: validate right after the pull, checking that expected columns exist, types are correct, and values are in range, and fail loudly before anything downstream runs. A `pointblank` agent pointed at the freshly-read file is a few lines and turns a silent upstream change into an immediate, diagnosable error.

13.8 Practical Guidance

Check for a package before writing request code. A maintained wrapper like `tidycensus` or `RSocrata` already handles pagination, types, and authentication.

Filter on the server. Use query parameters to request only the rows and columns you need, for the same reason you filter before `collect()` with a database.

Watch for silent row caps. If the row count of a pull is a suspiciously round number, you probably got page one of many.

Archive what you pull. A dated raw file in `data/raw/` is the reproducible input; the API is not, because the data behind it changes.

Treat keys like passwords. `.Renviron` locally, repository secrets in CI, never in code.

Validate every pull. Coerce types explicitly and check the schema, because the provider can change either one without telling you.

14. Migrating from Legacy Workflows

Most public health data work predates the tools in this book. State health departments run surveillance on SAS programs written over twenty years by people who have since retired, weekly reports come out of Excel workbooks where the analysis lives in formula chains and pivot tables, and case data sits in Access databases on a shared drive. These workflows produce real, trusted outputs every week. What makes them legacy is that they are hard to maintain and getting harder to staff.

The syntax is the easy part of a migration; any competent R programmer, or these days an AI assistant, can translate a `PROC MEANS` into a `summarize()`. What takes judgment is deciding what to migrate and in what order, proving that the new pipeline produces the same numbers as the old one, and retiring the old system without losing the institutional knowledge embedded in it. This chapter is about that process.

14.1 Why Migrate, and Why Not

The case for migrating a workflow is usually some mix of money, staffing, and capability. SAS licensing is a recurring line item that R eliminates; for a small team, the savings alone can fund a training program. The hiring pool tilts the same direction: new analysts arrive knowing R or Python, the people who know SAS are retiring, and every year a workflow stays in SAS it gets harder to find someone who can maintain it. And the legacy stack cannot easily produce what earlier chapters built: parameterized Quarto reports (Chapter 4.), dashboards (Chapter 7.), scheduled pipelines on GitHub Actions (Chapter 8.), or a version-controlled analysis that can be reviewed and handed off (Chapter 1.).

There are equally good reasons to leave a particular workflow alone, at least for now. A report that is being retired should not be rewritten; migration is a chance to prune the product list, not to reproduce all of it faithfully. A rewrite the team cannot maintain is worse than the status quo: if the only SAS analyst retires in six months and nobody else knows R yet, the first investment is training (Section 17.3). And a rewrite in the middle of an outbreak response adds risk exactly when the team can least absorb it.

All of this reduces to one underlying risk: migration is rewriting software, and rewriting working software is dangerous. The legacy program's numbers are trusted; the rewrite's are not until you prove they match. That proof is the core of the work.

14.2 Inventory and Triage

Start by listing every recurring product the legacy stack produces: each report, each dataset handed to another program, each dashboard feed. For each one, record what runs it, how often, who consumes the output, and who understands the code. This inventory is usually revealing on its own. Most teams find a handful of products that matter enormously, a long tail that runs out of habit, and at least one program nobody can explain.

Then triage. Good first candidates for migration score high on all of these:

- It runs on a schedule (weekly, monthly), so the payoff recurs and there are regular opportunities to compare old against new.
- Someone who understands the current logic is still available to answer questions.
- The output has an engaged consumer who will notice, and care, if numbers change.
- It is painful today: manual steps, fragile hand-offs, a single person who can run it.

One-off historical analyses generally should not be migrated at all. Archive the code and its outputs, document what they were for, and leave them alone.

Migrate one product end to end before starting a second. A complete pilot, from raw data to delivered report, running in parallel with the legacy version, teaches the team more than partially migrating ten programs, and it produces a visible win that builds support for the rest of the effort.

14.3 Reading Legacy Data Formats

Whatever else migrates, the data has to. The `haven` package reads SAS (`.sas7bdat`, `.xpt`), SPSS (`.sav`), and Stata (`.dta`) files directly:

```
library(haven)

path <- system.file("examples", "iris.sas7bdat", package = "haven")
read_sas(path)
```

```
# A tibble: 150 × 5
   Sepal_Length Sepal_Width Petal_Length Petal_Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <chr>
 1          5.1         3.5          1.4         0.2 setosa
 2          4.9         3            1.4         0.2 setosa
 3          4.7         3.2          1.3         0.2 setosa
 4          4.6         3.1          1.5         0.2 setosa
 5          5           3.6          1.4         0.2 setosa
 6          5.4         3.9          1.7         0.4 setosa
 7          4.6         3.4          1.4         0.3 setosa
 8          5           3.4          1.5         0.2 setosa
 9          4.4         2.9          1.4         0.2 setosa
10          4.9         3.1          1.5         0.1 setosa
# ℹ 140 more rows
```

A few things about what comes across. SAS stores value labels (formats) separately from the data, in a catalog file typically named `formats.sas7bcat`; pass it to `read_sas()` and the labels arrive as labelled columns, which `as_factor()` converts to R factors:

```
cases <- read_sas(
  "data/raw/cases.sas7bdat",
  catalog_file = "data/raw/formats.sas7bcat"
)
cases <- as_factor(cases)
```

Missing values need more care. SAS distinguishes up to 27 kinds of missing (`.`, `.A` through `.Z`), often used to encode *why* a value is missing: refused, unknown, not applicable. R has one `NA`. `haven` preserves the distinctions as tagged NAs (see `?haven::tagged_na`), but most downstream R code will treat them all as plain `NA`, so if the distinctions carry meaning, convert them to an explicit column before they are lost.

Dates come through correctly, because `haven` knows that SAS counts days from 1960-01-01 while R counts from 1970-01-01. Hand-rolled imports do not get that conversion: a CSV exported from SAS with raw date numbers will be off by exactly 3,653 days, an error distinctive enough to recognize on sight.

For Excel sources, `readxl` reads `.xlsx` and `.xls` files, and the spreadsheet-hygiene guidance in Chapter 2. applies doubly to inherited workbooks. For Access, the `odbc` package can connect directly on Windows, where the Access ODBC driver is usually already installed:

```
library(DBI)

con <- dbConnect(
  odbc::odbc(),
  .connection_string = paste0(
    "Driver={Microsoft Access Driver (*.mdb, *.accdb)};",
    "Dbq=S:/surveillance/cases.accdb;"
  )
)
dbListTables(con)
```

Whichever format the data arrives in, convert it once and save an analysis-ready copy in an open format (CSV or Parquet, per Chapter 2.), keeping the original file as the untouched raw archive. Proprietary formats depend on software that will not always be there to read them.

14.4 Translating SAS Code

SAS and R differ more in mindset than in capability. A SAS program is a sequence of DATA steps (implicit row-by-row loops that read one dataset and write another) punctuated by PROC calls. Idiomatic R is vectorized: operations apply to whole columns at once, chained with pipes. A literal line-by-line translation produces R code that works but reads like SAS; the better approach is to translate what each step accomplishes.

Most of the surface area maps cleanly:

SAS	R equivalent
DATA step assignments, `IF`/`THEN`	`mutate()` with `if_else()` or `case_when()`
`PROC SORT` (and `NODUPKEY`)	`arrange()`, `distinct()`
`PROC MEANS`, `PROC SUMMARY`	`group_by()` + `summarize()`
`PROC FREQ`	`count()`, `janitor::tabyl()`
`PROC TRANSPOSE`	`tidyr::pivot_longer()`, `pivot_wider()`
`MERGE` in a DATA step	`left_join()` and friends
`PROC SQL`	dplyr, or SQL directly via DBI (Chapter 12.)
`PROC FORMAT`	factor levels and labels
`PROC EXPORT`	`readr::write_csv()`
`%MACRO`	a function

The last row deserves emphasis. SAS macros are text substitution; R functions are real functions with arguments and return values, and they are easier to test and document. A macro library that grew over years often translates into a small set of R functions, which is the point at which a team package (Chapter 10.) starts to make sense.

14.4.1 AI-assisted translation

Legacy translation is a task AI coding assistants (Chapter 6.) handle well. Claude Code (Section 6.4) can take a several-hundred-line SAS program and produce a credible R draft in minutes, and it is equally useful in the other direction: asking it to *explain* an inherited program, step by step, before you translate anything. When the original author is gone, an AI walkthrough of what a gnarly DATA step actually does is often the fastest available substitute.

Treat the output as a draft, though. Translation is now cheap; verification is the actual work, and the failure mode of AI translation is code that looks right and runs clean but handles an edge case (usually missing values) differently than the original. Nothing that comes out of an assistant should reach production without passing the parity checks in the next section. And as always, paste code into these tools, not data: the privacy considerations in Section 6.5 apply to the data, and the code itself is rarely sensitive.

14.5 Verifying Parity

The legacy program's output is your test oracle. Freeze an input dataset, run it through both pipelines, and compare the outputs with code rather than by eye. The `waldo` package gives readable, precise diffs:

```
sas_out <- data.frame(
  county = c("Fairfax", "Arlington", "Loudoun"),
  cases = c(23L, 41L, 18L),
  rate = c(2.0, 17.2, 4.3)
)

r_out <- data.frame(
  county = c("Fairfax", "Arlington", "Loudoun"),
  cases = c(23L, 41L, 18L),
```

```
  rate = c(2.0, 17.2, 4.2)
)

waldo::compare(sas_out, r_out)
```

```
old vs new
           rate
  old[1, ]  2.0
  old[2, ] 17.2
- old[3, ]  4.3
+ new[3, ]  4.2

`old$rate`: 2.00 17.20 4.30
`new$rate`: 2.00 17.20 4.20
```

The diff points straight at the discrepancy. Counts should match exactly. Derived numbers like rates should match within a small tolerance (`waldo::compare()` takes a `tolerance` argument), because the two engines can legitimately differ in the last decimal places of floating-point arithmetic.

When outputs differ, the cause is usually a place where SAS and R behave differently by design rather than a translation typo. The differences that bite most often:

Behavior	SAS	R
Rounding halves	Away from zero: `ROUND(2.5)` is 3	Half to even: `round(2.5)` is 2
Missing values	`.` plus special missings `.A`–`.Z`	A single `NA`
Missing values in sorts	Sort first (missing is smallest)	`arrange()` puts `NA` last
Blank strings	`""` is missing	`""` is a value, distinct from `NA`
Date origin	1960-01-01	1970-01-01

The rounding difference is worth seeing once, because it looks so much like a bug:

```
round(c(0.5, 1.5, 2.5))
```

```
[1] 0 2 2
```

R follows the IEEE standard and rounds halves to the nearest even digit, so 0.5 rounds to 0 and 2.5 rounds to 2, where SAS would give 1 and 3. If the legacy program rounds before a suppression threshold or publishes rounded rates, the two pipelines will disagree by one in the last digit on exactly the values ending in five, and only those.

Every discrepancy gets one of two resolutions: fix the R code to match, or accept the difference and write down why. Keep the comparison script and a short parity report in the repository alongside the migrated code, listing each accepted difference and its justification. That document is what lets you tell a program manager, with evidence, that the new numbers are trustworthy, and it is exactly the kind of artifact analytical peer review (Chapter 20.) should examine.

14.6 Running in Parallel and Cutting Over

Parity on a frozen test dataset is necessary but not sufficient, because live data keeps finding cases the test data did not contain. So run both pipelines in parallel on real data for a fixed number of cycles: four to eight weeks for a weekly report is typical. Each cycle, produce both outputs, run the comparison script, and investigate anything new. These investigations are where the legacy program's undocumented behavior surfaces, the special-casing of one bad data year, the county recode nobody mentioned, and each one either becomes deliberate logic in the new pipeline or a documented, accepted difference.

Agree on the cutover criteria before the parallel period starts (for example: N consecutive cycles with no unexplained discrepancies), so the decision is mechanical rather than a matter of nerve. Then, at cutover:

- Move the legacy code into the repository under a `legacy/` directory, read-only, with a README mapping each old program to its replacement (Section 16.1).

- Run the legacy pipeline one final time and archive its outputs as the last reference point.
- Actually decommission it. Remove it from the schedule, and cancel the license when the last workflow that needs it is gone.

The decommissioning step is the one teams skip, and skipping it is costly. Two live pipelines means two sources of truth, and they will eventually diverge, at which point someone has to figure out which one the program has been using. The license line item disappearing from next year's budget is both the payoff and the forcing function.

14.7 Excel and Access Workflows

Spreadsheet migrations differ from SAS migrations because there is no program to translate. The analysis is smeared across formula chains, pivot caches, and cells someone updates by hand, none of it visible to version control or review, and all of it fragile: one inserted row can silently break a range reference three sheets away.

The pattern that works is to split the workbook's two jobs. If data entry happens in Excel, let it keep happening there, in a workbook restructured to the rectangular, one-thing-per-cell standard of Chapter 2.. Everything after entry moves to a script: the workbook becomes an *input* that `readxl` reads, and R produces the outputs. The translations are mostly one-to-one. A `VLOOKUP` chain is a `left_join()`, a pivot table is `group_by()` plus `summarize()` (with `pivot_wider()` for the layout), and logic encoded in conditional formatting becomes an explicit column. Add validation at the seam (Chapter 3.), because scripted analysis surfaces the entry errors that formulas were silently absorbing, which is uncomfortable for a few weeks and then permanently better.

Access databases play two roles that migrate separately. As storage, the tables can be read directly via `odbc` as shown above, or exported once if the database is being retired. As logic, Access queries are just SQL, and they port to dplyr or to DuckDB (Section 12.2) with little friction. If several people still need to enter and look up cases interactively, Access can keep that front-end job for now, with R connecting read-only for analysis; the longer-term home for shared, structured case data is an IT-managed database, which is a conversation to start with the people in Section 19.4.

14.8 The People Side

A legacy program is institutional memory in executable form. Somewhere in that SAS code is the exclusion rule from a 2015 data quality incident and the county recode that exists because of a boundary change, none of it written down anywhere else. The person who wrote the program holds the other half of that memory.

So migrate *with* the legacy author, not around them. Pair them with the team's strongest R programmer: the veteran explains what each step is for and reviews parity discrepancies, while the R partner writes idiomatic new code, and both learn. This is upskilling (Section 17.3) with a concrete deliverable attached, and it reframes the migration for the author as career development rather than obsolescence, which matters for retention (Section 17.6). When the author is already gone, the code is the only informant left, which makes the careful reading, and the AI-assisted explanation described earlier, that much more important.

Finally, pick the timing like the project decision it is (Chapter 18.). A migration has a long middle where both systems run and the team carries double work. Schedule that middle for the quiet season rather than the weeks when respiratory reports are due daily.

14.9 Further Reading

- The haven documentation covers the details of reading SAS, SPSS, and Stata files, including labelled values and tagged missings.
- *R for Excel Users* (Lowndes and Horst) is a free, workshop-format introduction to R aimed specifically at people whose current workflow is Excel.
- *The Epidemiologist R Handbook* includes a "Transition to R" chapter with practical advice, and R equivalents for common tasks, aimed at epidemiologists coming from SAS, Stata, and SPSS.

15. Code Style

Writing code that works is necessary. Writing code that others can read and modify is what makes a team functional over time. As teams grow, people rotate, and analyses are revisited months after they were written, the gap between "it runs" and "anyone can understand and extend it" becomes the gap between a one-time deliverable and a sustainable workflow.

Style conventions close that gap. They are agreements, sometimes explicit and sometimes just accumulated habit, about how code is written: how things are named, where spaces go, how long lines get before they break. None of these decisions affect what the code computes, but all of them affect how quickly a reader can understand it and how cleanly changes show up in version control.

This chapter covers the Tidyverse Style Guide as the standard for R, the Air formatter for automatically applying that standard, and the `lintr` package for catching issues that a formatter cannot.

15.1 The Tidyverse Style Guide

The Tidyverse Style Guide is the de facto standard for R code in data science. It is opinionated but well-reasoned, and following it means your code will look immediately familiar to anyone else working in R.

The most important conventions:

- **Names**: use `snake_case`, all lowercase, words separated by underscores. Prefer `flu_cases` over `fluCases` or `FluCases`. Object names should be nouns; function names should be verbs.
- **Spacing**: put spaces around most operators (`<-`, `+`, `-`, `==`, and others) and after commas. Do not put spaces inside parentheses or before a comma.
- **Line length**: keep lines to 80 characters or fewer. Long expressions should be broken across lines with consistent indentation.
- **Indentation**: two spaces per level. Not tabs.
- **Assignment**: use `<-` for assignment, not `=`.
- **Pipes**: when chaining multiple operations, put each step on its own line, indented two spaces.

The full guide covers function naming, documentation, ggplot2 layering, and much more. It is worth reading once and returning to when questions arise. The key practical point: you do not have to memorize all of it. A formatter handles most of the mechanical rules automatically.

15.2 Formatting with Air

Air is an R code formatter written in Rust. When you save a file, it reformats the code to follow consistent conventions for spacing, indentation, and line breaks. It does not rename variables or change logic, only whitespace and layout.

The value of a formatter is that it removes style decisions from the development loop entirely. You write code, save, and Air handles presentation. Code review conversations shift from "this line is too long" or "there should be a space here" to the actual logic. Diffs in version control (see Chapter 1.) show only meaningful changes, not incidental reformatting.

15.2.1 Setting Up Format on Save in Positron

Air ships bundled with Positron, so no separate installation is needed. To enable format on save, configure a `.vscode/settings.json` file in your project root. The quickest way:

```
usethis::use_air()
```

This creates the settings file with the correct entries. Alternatively, create or edit `.vscode/settings.json` manually:

```
{
    "[r]": {
```

```
        "editor.formatOnSave": true,
        "editor.defaultFormatter": "Posit.air-vscode"
    }
}
```

Committing `.vscode/settings.json` to version control means every collaborator on the project gets format-on-save behavior automatically, without any manual setup.

For Quarto documents, add a second entry so that R code cells inside `.qmd` files are also formatted:

```
{
    "[r]": {
        "editor.formatOnSave": true,
        "editor.defaultFormatter": "Posit.air-vscode"
    },
    "[quarto]": {
        "editor.formatOnSave": true,
        "editor.defaultFormatter": "quarto.quarto"
    }
}
```

To format a single R cell inside a `.qmd` file without saving the whole document, place your cursor in the cell and press `Cmd+K Cmd+F` (Mac) or `Ctrl+K Ctrl+F` (Windows/Linux).

> **Formatting a whole project at once**
>
> To format every R file in a project at once, open the Command Palette (`Cmd+Shift+P` on Mac, `Ctrl+Shift+P` on Windows/Linux) and run **Air: Format Workspace Folder**. This is useful when first introducing Air to an existing codebase.

15.2.2 Before and After

The following two blocks show the same code before and after Air formats it. The computation is identical in both versions; only the layout changes.

Before:

```
flu_summary<-flu|>filter(cases>0,!is.na(county))|>group_by(county,disease)|
>summarise(total_cases=sum(cases),avg_rate=mean(rate,na.rm=TRUE),.groups="drop")|
>arrange(desc(total_cases))
```

After:

```
flu_summary <- flu |>
  filter(cases > 0, !is.na(county)) |>
  group_by(county, disease) |>
  summarise(
    total_cases = sum(cases),
    avg_rate = mean(rate, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(desc(total_cases))
```

The formatted version is longer in line count but far shorter in reading time. Each transformation appears on its own line, arguments that belong together are grouped, and indentation reflects the structure of the pipeline. When this code appears in a pull request, a reviewer can check each step independently rather than scanning a wall of characters for the one thing that changed.

> **Note**
>
> Air's configuration lives in an optional `air.toml` file at the project root. The defaults (2-space indentation, 80-character line width) follow the Tidyverse Style Guide and are suitable for most projects. The most common reason to configure `air.toml` is to exclude specific functions from Air's table-formatting behavior, using the `skip` field.

15.3 Linting with lintr

A formatter handles whitespace. A linter goes further: it reads code statically and flags patterns that are likely wrong or poor style, without running the code and without changing anything. Think of it as a careful reviewer pointing out potential problems for you to decide on.

The `lintr` package is the standard linting tool for R. It checks for issues including:

- Assignment with `=` instead of `<-`
- Missing or extra spaces around operators
- Use of `T` or `F` instead of `TRUE` or `FALSE` (fragile, because `T` can be overwritten as a variable name)
- Redundant comparisons like `x == TRUE`
- Lines that exceed the recommended length
- Variable names that do not follow `snake_case` convention

Unlike Air, lintr does not fix anything. It reports what it finds and leaves decisions to you. This is useful for catching issues that a formatter cannot handle: semantic shortcuts, naming violations, and patterns that are syntactically valid but likely to cause confusion.

15.3.1 Getting Started

Install lintr from CRAN:

```
install.packages("lintr")
```

The primary interface is `lintr::lint("your_file.R")`, which reads a file from disk and prints a list of warnings. To lint every R file in a project at once, use `lintr::lint_dir()`. Each line of output identifies the file, the line and column, the severity, and the name of the rule that triggered it.

15.3.2 Examples

Consider an R script, `analysis.R`, with several common problems:

```
x = 10
y<-x*2
flag = T
if (flag == TRUE) print(y)
```

Running `lintr::lint("analysis.R")` produces:

```
analysis.R:2:3: style: [assignment_linter] Use one of <-, <<- for assignment, not =.
x = 10
  ^
analysis.R:3:2: style: [infix_spaces_linter] Put spaces around all infix operators.
y<-x*2
 ^~
analysis.R:3:5: style: [infix_spaces_linter] Put spaces around all infix operators.
y<-x*2
    ^
analysis.R:4:6: style: [assignment_linter] Use one of <-, <<- for assignment, not =.
flag = T
     ^
analysis.R:4:9: style: [T_and_F_symbol_linter] Use TRUE instead of the symbol T.
flag = T
```

```
      ~^
analysis.R:5:27: style: [trailing_blank_lines_linter] Add a terminal newline.
if (flag == TRUE) print(y)
```

Working through these one at a time: `=` for assignment is flagged because `<-` is the R convention and `=` is reserved for function arguments. The missing spaces around `<-` and `*` on the second line are spacing violations. `T` as a shorthand for `TRUE` is flagged because `T` is just a variable that happens to default to `TRUE` and can be overwritten (`T <- 42` is legal R). And `flag == TRUE` is redundant: if `flag` is already a logical value, `if (flag)` says exactly the same thing.

A second example shows naming convention warnings. Given a file with:

```
totalCases <- 100
meanRate <- totalCases / 30
ReportTitle <- paste("Weekly report:", format(Sys.Date(), "%B %Y"))
```

lintr flags all three names:

```
analysis.R:2:1: style: [object_name_linter] Variable and function name style should
match snake_case or symbols.
totalCases <- 100
^~~~~~~~~~
analysis.R:3:1: style: [object_name_linter] Variable and function name style should
match snake_case or symbols.
meanRate <- totalCases / 30
^~~~~~~~
analysis.R:4:1: style: [object_name_linter] Variable and function name style should
match snake_case or symbols.
ReportTitle <- paste("Weekly report:", format(Sys.Date(), "%B %Y"))
^~~~~~~~~~~
analysis.R:4:68: style: [trailing_blank_lines_linter] Add a terminal newline.
ReportTitle <- paste("Weekly report:", format(Sys.Date(), "%B %Y"))
```

All three names are flagged for camelCase or PascalCase. In an analysis that will be maintained and extended, consistent `snake_case` naming is worth the small upfront effort of renaming.

> **Using Air and lintr together**
>
> Air and lintr complement each other. Air handles layout automatically on every save. lintr catches things that layout cannot: semantic shortcuts like `T` for `TRUE`, naming convention violations, and redundant expressions. A practical workflow is to let Air format on save and run `lintr::lint_dir()` periodically to catch the issues Air does not touch.

16. Documentation

Code that runs correctly but cannot be understood is a liability. Six months after writing an analysis, even the original author may not remember what a variable represents, why a particular filter was applied, or where a dataset came from. For a new team member inheriting existing work, undocumented code is essentially opaque.

Documentation is the investment that makes analytical work reusable, reviewable, and transferable. It does not mean writing a manual for every script. It means leaving enough information that the next person (or your future self) can pick up the work without starting over.

16.1 READMEs

Every project repository should have a README at the root. This is the first thing anyone sees when they open a project, and it should answer the questions a new reader has immediately:

- What does this project do?
- What data does it use, and where does that data come from?
- How do you run the analysis?
- What are the outputs and where do they go?

A README does not need to be long. A one-page README that answers these questions is more valuable than a ten-page document that describes the history of the project and the background literature.

16.1.1 A Minimal README Template

```
# Project Title

One or two sentences describing what this analysis does and why it was done.

## Data

What data is used. Where it comes from (source, date accessed, any access
requirements). If data files are not in the repository (e.g., because
they contain PII), explain where they should be placed.

## Running the Analysis

How to reproduce the results. What to run first, what runs next.
Any packages or environment requirements (see environments chapter).

## Outputs

What the analysis produces. Where outputs are written.
```

Commit the README with the project. Update it when something material changes. A stale README is worse than a brief one, because it creates false confidence that the instructions are current.

16.1.2 README for Shared Utility Code

If a repository contains functions or tools that other projects use rather than a single self-contained analysis, the README should also document the interface: what the key functions are, what arguments they take, and a short example showing typical use.

16.2 Inline Comments

Comments in code explain *why*, not *what*. The code itself shows what is happening; a comment adds value by explaining the reason behind a decision that might otherwise look arbitrary.

A comment that adds no information:

```
# filter to active cases
cases <- cases |> filter(status == "active")
```

The code already says this. The comment is noise.

A comment that adds information:

```
# exclude cases with status "pending" and "transferred": these are not yet
# reportable and would inflate counts. see data dictionary for full status codes.
cases <- cases |> filter(status == "active")
```

This explains why only active cases are kept — information that is not in the code itself and that a reviewer or future maintainer genuinely needs.

Comment when:

- A non-obvious decision was made and the reason matters (a filter that excludes something, a constant that came from an external source)
- A known data quality issue is being worked around
- An approach was chosen over an obvious alternative for a specific reason

Do not comment:

- Code that reads naturally and does what it says
- Every line as a matter of habit
- To explain what R functions do (that is what documentation is for)

16.2.1 Sectioning Longer Scripts

For analysis scripts that run longer than a screen, section headers help readers navigate. In R, headers recognized by RStudio and Positron use `# ----` or `# ====` suffixes:

```
# Load data ---------------------------------------------------------------

# Clean and filter --------------------------------------------------------

# Calculate rates ---------------------------------------------------------

# Produce outputs ---------------------------------------------------------
```

These create an outline in the document navigator (the panel at the bottom of the script editor in Positron) that lets readers jump directly to a section without scrolling.

16.3 Documenting Functions with roxygen2

When a function will be reused – called from multiple scripts, shared across projects, or used by other team members – it should be documented at the function definition, not in a separate file. The standard tool for this in R is roxygen2.

roxygen2 documentation lives in comments immediately above the function definition, starting with `#'`. The most important tags:

- `@param` — documents each argument: its name, expected type, and what it does
- `@return` — describes what the function returns
- `@examples` — shows how to call the function, ideally with a runnable example

A minimal documented function:

```
#' Calculate age-adjusted rate per 100,000
#'
#' Applies direct standardization to compute an age-adjusted rate using
```

```
#' the 2000 U.S. standard population weights.
#'
#' @param cases A data frame with columns `age_group`, `events`, and `population`.
#' @param std_pop A data frame with columns `age_group` and `std_weight`. Defaults
#'   to the 2000 U.S. standard population.
#'
#' @return A single numeric value: the age-adjusted rate per 100,000 population.
#'
#' @examples
#' calculate_age_adjusted_rate(flu_cases_2023)
calculate_age_adjusted_rate <- function(cases, std_pop = us_std_pop_2000) {
  # ... function body
}
```

Even a brief description, one `@param` per argument, and one `@return` is substantially better than no documentation at all. A future reader knows what the function expects without having to read and mentally execute the implementation.

16.3.1 When Functions Are More Than Utilities

A handful of shared utility functions can live in a `functions/` or `R/` subdirectory in a project repository and be sourced with `source()`. This is a reasonable approach for a single project.

When utility functions are shared across multiple projects, or when the team is maintaining a set of internal tools that need to be installed and versioned like any other package, the right container is an R package. A package brings in a testing framework, proper namespace management, and the formal documentation infrastructure that roxygen2 was designed for.

Building a complete R package goes beyond the scope of this chapter. The definitive guide is **R Packages** by Wickham and Bryan, available free online. Even teams that do not plan to publish to CRAN benefit from the structure a package imposes, it is a format that R itself understands, which makes distribution, installation, and documentation tooling available for free.

16.4 Project-Level Documentation

Analysis projects involve more than code: there are data sources, methodological decisions, known data quality issues, and interpretive context that do not belong in code comments or README files but that someone needs to know.

16.4.1 Data Dictionaries

If a project uses a dataset with non-obvious columns — abbreviated names, coded values, or columns whose meaning depends on context — maintain a data dictionary alongside the analysis. This can be a simple table in a Markdown or Quarto document:

Column	Type	Description
`case_id`	character	Unique case identifier; not persistent across system updates
`epi_class`	character	Epidemiologic classification: `confirmed`, `probable`, `suspect`
`rpt_cnty_cd`	integer	FIPS code of the county of report (not necessarily the county of residence)
`onset_dt`	date	Symptom onset date as reported; missing if not collected for this condition

A data dictionary does not need to document every column. Document the columns that are confusing, the ones that carry important caveats, and the ones where the name does not tell the full story.

16.4.2 Decision Logs

Analyses involve choices that are not visible in the code: why a particular date range was chosen, why a jurisdiction was excluded, why one method was preferred over another when both were defensible. These decisions are worth recording, because they will be questioned — by a reviewer, by a stakeholder, or by a future analyst who is maintaining the work.

A simple decision log entry might look like:

> **2024-11-14** — Excluded 2020 data from trend analysis. COVID-related disruptions to case reporting caused substantial undercounting across multiple conditions; the 2020 data points would distort trend estimates in ways that are not informative about the underlying epidemiology. Analysis covers 2018–2019 and 2021–2023.

This does not need to be elaborate. A short paragraph with a date and a rationale is enough to reconstruct the reasoning later.

16.4.3 Methods Documentation

For analyses with non-trivial statistical methods, a methods document that describes the approach in plain language (separate from the code) is worth maintaining. This is the document a reviewer (see Chapter 20.) or stakeholder can read to understand what was done without reading R code. It should describe the data sources, the analytical approach, known limitations, and any validation steps taken.

This document can be a section of the final report, a standalone Quarto document in the repository, or a README section, wherever it is most likely to be found and updated.

16.5 Documentation as a Team Practice

Documentation that exists only on an individual's laptop, or only in one person's head, does not help the team. Documentation practices are only effective when they are shared norms.

Establish a minimum standard. Teams that define what "documented" means (e.g., a README, a data dictionary for any dataset with coded variables, roxygen comments on shared functions) can build review expectations around that standard. The Chapter 20. chapter describes how code review (see Section 20.2) creates a checkpoint for documentation quality.

Document as you go. Documentation written after the analysis is finished is usually thinner and less accurate than documentation written alongside it. Writing the README before writing the code helps clarify what the analysis is supposed to do. Noting a methodological decision at the time it is made is more reliable than reconstructing the rationale later.

Treat stale documentation as a bug. An incorrect README is worse than no README, because it creates false confidence. When something material changes (a data source is updated, a method is revised, a constant is recalculated), updating the documentation is part of the change, not an optional follow-up.

II Nontechnical

17. Building and Sustaining a Data Science Team

Two failure modes bookend public health data science. In the first, a health department has one analyst who understands the surveillance pipeline, who built it, who maintains it, who is the only person who knows why the county denominators are adjusted the way they are. When that person takes another job, the institutional memory leaves with them, and the pipeline becomes a black box that still runs but that nobody can change. In the second, a department receives new funding to stand up a data science function and discovers it has no idea what to hire for, how to classify the positions, or where the new team should sit.

This chapter is about the work between those two failure modes: building a data science capability that does not depend on any single person and that fits the realities of a public health agency. The project management chapter (Chapter 18.) covers running a project well. This one covers building and keeping the team that runs the projects.

17.1 Roles on a Public Health Data Science Team

The titles overlap and the boundaries are fuzzy. An epidemiologist, a data analyst, a data scientist, a data engineer, and an informatician can have job descriptions that read almost identically, and in a small agency the same person may answer to all five. It helps to think less about titles and more about the functions that have to be covered:

- **Subject-matter framing**: turning a program question into an answerable analytic question. This is the epidemiology and domain expertise that keeps the analysis pointed in the right direction.
- **Data engineering**: getting data out of source systems, moving it, and keeping pipelines running. Often the least visible function and the one whose absence hurts most.
- **Analysis and modeling**: the statistical and computational work of producing the result.
- **Communication and delivery**: reports, dashboards, and the translation of findings for people who did not run the analysis (Chapter 21.).

Most public health data scientists are what is sometimes called T-shaped: deep in one of these areas and competent across the others. The practices in this book (version control, reproducible reporting, validation) are the working competence that lets one person cover more than their specialty without the quality collapsing.

The reality for most teams is that the "team" is one to three people, and they wear every hat. Framing roles as functions rather than headcount makes this tractable: you are not trying to hire five specialists, you are trying to make sure all four functions are covered by the people you have. A lightweight skills matrix (functions down one axis, people across the other, proficiency in the cells) is a quick way to see which function is dangerously thin before it becomes a crisis.

17.2 Hiring and Position Descriptions

Hiring a data scientist into a government agency runs straight into a classification problem. Many civil-service systems have no "data scientist" job classification at all, so the role gets filed under whatever existing title is closest: epidemiologist, IT specialist, statistician, research analyst. Each comes with a salary band and a set of required credentials that may not match the work. A position classified as "epidemiologist III" may require a degree the best candidate lacks, or cap salary below what the labor market commands for the skills you actually need.

You cannot always fix the classification, but you can write the position description to describe the work rather than a credential checklist. State what the person will do: build reproducible analytic pipelines, work in version control, query institutional databases, produce reports and dashboards, communicate findings to program staff. Skills described this way let a wider and better pool see themselves in the role, and they give you concrete things to evaluate.

When evaluating candidates, a portfolio tells you more than a transcript. A GitHub profile, a published dashboard, or a short take-home exercise[1] that mirrors the actual work reveals whether someone can do the job in a way that a list of degrees cannot. A junior hire should show they can write clean, working code and learn quickly; a senior hire should show judgment about when an analysis is good enough, how to structure a project so others can pick it up, and how to mentor.

Whether to hire a full-time employee or bring in a contractor is a real tradeoff, not a default. A contractor can deliver specialized skills quickly and is the right call for a bounded, well-specified build. A full-time employee accumulates the institutional knowledge (which data sources lie about what, which stakeholders need what) that makes the next project faster. For anything ongoing, especially anything load-bearing for surveillance, continuity usually wins.

17.3 Growing Skills on the Team You Have

Hiring is not the only way, and often not the best way, to build capacity. Most public health analysts arrive fluent in Excel, SAS, or SPSS rather than R, Git, and Quarto, and many of them are strong analysts who simply have not been given the chance to learn modern tooling. (Migrating the workflows themselves, as opposed to the people who run them, is the subject of Chapter 14..) Upskilling the people you already have is frequently more practical than competing for scarce outside hires, and it builds on institutional knowledge that a new hire would have to acquire from scratch.

The binding constraint on upskilling is almost never motivation or materials. Both are abundant: the resources in Appendix A. are free and excellent, and people generally want to grow. The constraint is protected time. Learning to work reproducibly while also delivering the weekly reports is nearly impossible if the weekly reports consume every hour. Capacity-building that is not given real, defended time on the calendar does not happen, no matter how good the intentions.

Several pathways work, and they compound:

- **Structured curricula** for foundational skills, working through a book or course as a group (Appendix A.).
- **Programs built for this purpose**, like the DSTT program this book grew out of, which pair training with coaching on the team's actual projects.
- **Communities of practice** inside or across agencies, where people doing similar work share patterns and unstick each other.
- **Pairing and code review** (Chapter 20.) as a teaching mechanism, not just a quality control: reviewing a colleague's pull request is one of the most effective ways for both people to learn.

It also helps to be explicit that the practices in this book are learned incrementally. Nobody adopts version control, reproducible reporting, validation, and reproducible environments all at once. A team that picks up one practice per quarter is moving at a healthy pace.

17.4 Onboarding and Knowledge Continuity

The blunt version of the question is the bus factor: how many people could leave before the work stops? A bus factor of one, the lone analyst who is the only one who understands the pipeline, is the most common and most dangerous condition in public health data science. The goal of onboarding and continuity practices is to get that number above one and keep it there.

Good onboarding is mostly the payoff of practices covered elsewhere in this book. A new team member should be able to get productive by following a documented path: set up their environment (Chapter 11.), get access to the repositories (Chapter 1.), learn the project structure conventions the team uses (Chapter 2.), and find where the documentation lives (Chapter 16.). When these exist, onboarding is a checklist. When they do not, onboarding is an apprenticeship that depends on the very person you are worried about losing.

Version control and READMEs are worth singling out, because their real value is continuity and not just developer convenience. A repository with a clear history and a README that explains how the analysis runs is institutional memory that survives staff turnover. It is the direct antidote to the "only one person understands this code" problem already flagged in Section 18.3. The discipline is the same whether the team is one person or ten: write it down, commit it, so that the knowledge lives in the repository and not only in someone's head.

[1] Take-home exercises should be followed up with an oral interview to walk through the exercise, since take-homes can easily be solved perfectly with any of the modern frontier AI models.

17.5 Where the Team Sits

Where a data science function reports within an agency shapes what it can do. There is no single right answer, but the common options trade off in predictable ways:

- **Inside an epidemiology or surveillance program** keeps the team close to the subject-matter questions and the people who will use the findings, at the cost of distance from the data infrastructure and the IT relationships that provision it.
- **Inside informatics or IT** puts the team next to the databases, servers, and access controls, at the cost of distance from the program questions and sometimes from the analytic culture.
- **A standalone analytics unit** serving the whole agency can set its own standards and serve many programs, at the cost of having to build relationships with each program rather than being embedded in one.

A related choice is the service model. A centralized team is a shared resource that programs request work from; it builds deep technical capability and consistent standards but can become a bottleneck and can drift away from any one program's needs. Embedded analysts sit inside programs, close to the work, but can become isolated and reinvent each other's solutions. Many agencies land on a hybrid: a small central team that sets standards and handles cross-cutting infrastructure, with analysts embedded in the largest programs.

Whatever the placement, the relationship with IT and data governance is structural and not incidental. The team's ability to access data, run pipelines, and publish results depends on it, which is why Section 18.4 and Chapter 19. treat those relationships as core to the work rather than as obstacles to route around.

17.6 Retention and Sustainability

Public sector salaries rarely match what a data scientist can earn in industry, and pretending otherwise helps no one. We must be clear-eyed about the gap and deliberate about the levers that do work. People stay in public health data science for the mission, for autonomy and ownership over meaningful work, for the chance to keep growing, and for the conditions that let them do the work well. Each of those is something an agency can actually offer.

Tooling is one of those conditions, and an underrated one. Skilled people who are forced to work in outdated, click-heavy, irreproducible environments (exporting from one system, pasting into another, hand-checking numbers) will leave for somewhere that lets them work the way they know how. The modern practices in this book are not only about output quality; giving capable people good tools is itself a retention strategy.

Sustainability is the team-level version of the same concern. Succession planning, documentation treated as insurance against turnover, and cross-training so that no pipeline has a single owner all reduce the damage any one departure can do. A team built so that losing one person is a setback rather than a catastrophe is also a team that is more pleasant to work on, because no one is trapped as the irreplaceable holder of a critical system. Building for continuity and building for retention turn out to be the same work.

17.7 Further Reading

- *Executive Data Science* [9] is a short, practical book on structuring and managing data science teams and working with executive stakeholders. Useful for team leads and for analysts who want to understand the decisions being made above them.
- *Build a Career in Data Science* [10] covers hiring, job descriptions, interviewing, and career growth from both sides of the table, and is a useful reference for managers writing position descriptions and evaluating candidates.
- CSTE and the Public Health Informatics Institute publish workforce and competency resources specific to public health data and informatics roles, which are helpful when mapping these general practices onto agency classifications.

18. Project Management

Data science projects in public health fail more often from unclear scope, misaligned expectations, and coordination breakdowns than from analytical errors. A rigorous analysis of the wrong question, delivered six months late to an audience that can't act on it, is a failed project regardless of its technical quality. This chapter covers the non-technical practices that help projects succeed: defining scope before work begins, collaborating as a team, working with IT and data governance partners, and communicating findings to people who will actually use them.

For a broader treatment of managing data science work, Roger Peng, Brian Caffo, and Jeff Leek's *Executive Data Science* [9] is a concise, practical resource for both team leads and the analysts working alongside them.

18.1 Defining the Project

18.1.1 Start with the question

The most important work in a data science project often happens before any data is touched. Peng and Matsui's *The Art of Data Science* [11] frames the entire analysis process around one deceptively simple task: state the question precisely. Vague questions produce vague analyses. "Can you look at overdose trends?" is not a question that will lead anywhere useful. "Has the age distribution of overdose deaths in our state shifted since 2019, and does the pattern differ by rural versus urban counties?" is a question that can be answered.

Before agreeing to take on a project, work with the requestor to articulate:

- What specific question are we answering?
- Who will use the findings, and what decision or action do they enable?
- What would a satisfying answer look like – a number, a chart, a report, a recommendation?
- What does success look like, and how will we know when we've reached it?

Getting these questions answered upfront is the difference between a project that delivers value and one that drifts.

18.1.2 Project charters

For anything beyond a quick turnaround request, a brief project charter documents the shared understanding between the data science team and its stakeholders before work begins. A charter doesn't need to be a formal document – a shared notes page or a few paragraphs in a project tracking tool is sufficient. What matters is that the key parties have read it and agreed to it before anyone opens a dataset.

A useful project charter for public health data science covers:

- **The question**: stated precisely, as described above.
- **Data sources and access**: what data will be used, where it lives, and what approvals are needed (data use agreements, IRB review, IT provisioning).
- **Deliverables**: what will actually be produced and in what format (report, dashboard, dataset, presentation slides)?
- **Timeline**: key milestones and a realistic completion date, with explicit acknowledgment of dependencies.
- **Stakeholders**: who the primary requestor is, who the audience is, who has final say on scope.
- **Out of scope**: what the project will explicitly *not* address; this is often as important as what it will.

The charter is also the right place to surface risks early. If the analysis depends on data that hasn't been collected yet, or on linking two systems that have never been joined, that should be visible before commitments are made.

18.2 Managing Scope

Scope creep – the gradual expansion of a project beyond its original definition – is endemic to data science work and is usually well-intentioned. Stakeholders see early results and ask "can you also look at…?", and analysts, wanting to be helpful, say yes. Over time, a focused project becomes an unfocused one.

Make scope changes explicit. When a new request comes in mid-project, treat it as a scope change. Acknowledge it, discuss whether it belongs in the current project or a future one, and adjust the timeline if it's added. The charter is the reference point for these conversations.

Protect the last mile. The gap between "the analysis is done" and "the findings are in front of the people who need them" is consistently larger than anticipated. Time for writing, review, revision, and presentation needs to be built into project plans – not squeezed in at the end.

Know when to say "not yet." Not every analytic question can be answered well with available data, methods, or time. It is better to scope a project to what can be answered defensibly than to produce a shaky analysis of a broader question. A clear, well-supported answer to a narrow question is more useful to a program than a hedged non-answer to a large one.

18.3 Collaborating as a Team

18.3.1 Version control

The foundation of collaborative data science work is version control. Chapter 1. covers Git and GitHub in depth – the mechanics of branches, pull requests, and conflict resolution. The project management implication is straightforward: a shared repository is the canonical record of a project's code and analysis. Work that lives only on one person's laptop is not team work; it is a liability.

For collaborative projects, adopt a simple branching convention (see Section 1.2) where changes are reviewed before being merged into the main branch. Even lightweight review (e.g. a colleague reading a pull request before it's merged) catches errors, shares knowledge, and prevents the "only one person understands this code" problem that becomes acute when someone leaves.

18.3.2 Reproducibility as a management goal

A reproducible analysis is one that can be re-run by a different person, on a different machine, at a future date, and produce the same result. In public health, this matters for accountability (can we explain exactly how this number was produced?), for updating results when new data arrives, and for knowledge transfer when team composition changes.

Reproducibility is a project management commitment. Building it in from the start costs far less than retrofitting it after a number is questioned.

18.3.3 Project structure

Consistent project organization makes it easier for team members to find things, onboard to a project quickly, and hand off work. A minimal structure that works well for most public health data science projects is shown below, but Chapter 2. covers project structure and data organization in more detail.

```
project-name/
├── data/
│   ├── .gitignore    # Don't commit data to version control.
│   ├── raw/          # original data -- never edited directly
│   └── processed/    # cleaned, analysis-ready files
├── R/                # scripts and functions
├── output/
│   ├── .gitignore    # Don't commit large output files.
│   ├── figures/
│   └── tables/
└── report.qmd
```

The most important convention: raw data is read-only. All transformations happen in code, so there is always a reproducible path from the original files to any result.

18.4 Working with Your IT Team

IT is a necessary partner in any institutional public health settings. Server access, database credentials, software installation, network permissions, and data transfer approvals all flow through IT. Projects that treat IT as an afterthought routinely stall at the moment they are otherwise ready to move.

18.4.1 Start early

IT requests take time, and often more time than anticipated. Getting new software approved and installed on a shared server may take days or weeks. Database access for a sensitive dataset may require formal requests, security reviews, and management sign-off at multiple levels. If the analysis depends on accessing a new data source or running on infrastructure you haven't used before, initiate those conversations at project kickoff.

18.4.2 Be specific about what you need

A clear, specific request is far easier to fulfill than a vague one. When approaching IT, be prepared to specify:

- What software or tools are needed, including version numbers if relevant.
- What data needs to be accessed, where it lives, and in what format.
- What level of compute and storage the analysis requires.
- Whether the work involves sensitive or regulated data (HIPAA, PII, restricted-use files) that requires special handling.
- Any hard deadlines driven by reporting requirements or funding timelines.

Framing requests in terms of the program need (e.g., "this analysis supports the quarterly overdose surveillance report due to the state health officer in March") helps IT understand the stakes and prioritize accordingly.

18.4.3 Secure data environments

Much of the data used in public health analytics is sensitive: vital records, case surveillance data, insurance claims, linked longitudinal datasets. Familiarize yourself with your organization's data governance policies and the specific terms of any data use agreements governing the datasets you work with. Some analyses must be conducted within designated secure environments (managed servers, data enclaves, air-gapped systems) and outputs may need to pass a disclosure review before leaving.

These constraints shape what tools and workflows are actually feasible. It is better to understand them at project start than to build an analysis pipeline that cannot be used where the data actually lives. See Chapter 19. for a more detailed treatment of data use agreements, HIPAA, disclosure review, and working in secure data environments.

18.5 Communicating Findings

A finding that is not communicated effectively is a finding that does not exist, for practical purposes. Public health data science ultimately serves program and policy decisions made by people who did not run the analysis and may not read a methods section.

18.5.1 Know your audience

Different stakeholders need different things:

- **Program staff** doing day-to-day work need actionable findings at the right level of detail, often a summary with supporting data available for those who want to dig in.
- **Program directors and health officers** need the bottom line and its implications, typically in a brief format (one page, one slide) with uncertainty communicated plainly.
- **Methodologically sophisticated collaborators** (epidemiologists, biostatisticians, peer reviewers) need full methods, sensitivity analyses, and honest discussion of limitations.

One analysis rarely serves all three audiences from a single document. Plan to produce multiple outputs from the same underlying work.

18.5.2 Communicate uncertainty honestly

Public health decisions are made under uncertainty, and analysis should convey what is known, what is estimated, and with what confidence – not project false precision. An administrator told "overdose deaths increased by exactly 12.3%" who later learns the estimate carried substantial uncertainty will trust the team less the next time. One given "we estimate a 10–15% increase, with the uncertainty driven primarily by changes in reporting completeness" has something they can actually act on.

That said, excessive hedging is its own failure mode. When the data clearly shows something, say so clearly. Uncertainty does not mean ambiguity about everything.

18.5.3 Match the output format to the need

A written report, an interactive dashboard, a two-page brief, and a slide deck are all appropriate in different contexts. The analysis determines what can be said; the output format determines whether it is heard. See Chapter 7. for guidance on building dashboards for data that needs to be explored or updated regularly. See Chapter 21. for a more detailed treatment of message structure, visualization choices, and communicating uncertainty.

18.6 Further Reading

- *Executive Data Science* [9]. A short, practical book on managing data science teams and working with executive stakeholders. Essential reading for team leads; useful context for analysts.
- *The Art of Data Science* [11]. Covers the full data analysis process from question formulation through communication. The epicycles-of-analysis framework is particularly useful for understanding why projects rarely proceed in a straight line, and why that is normal.

19. Working with IT and Data Governance

Data science in public health does not happen in a vacuum. Before a single line of code is written, the data has to be located, accessed, and cleared for use. Doing that requires navigating institutional infrastructure, legal agreements, and policies that determine what can be done with health data and where. Section 18.4 introduces IT as a project management concern; this chapter goes into the operational detail: the types of environments public health data scientists actually work in, the legal frameworks that govern data use, and the practical steps for getting access to data and staying on the right side of both.

19.1 Data Environments in Public Health

Public health data science happens across a wide range of computing environments, and the environment shapes what tools and workflows are feasible. Understanding where data lives, and what constraints that imposes, is the starting point.

Laptops and local files are the simplest setup. Data is on your machine, code runs locally, and there are no network permissions to navigate. This works for small, non-sensitive datasets (public-use files, aggregated data, synthetic data) but fails for anything regulated: sensitive data should not live on a personal laptop without explicit authorization.

Shared network drives are the most common data storage arrangement at state and local health departments. The data lives on a server managed by IT, mapped as a drive letter or network path on each authorized user's machine. As described in Chapter 2., this creates a specific challenge: code lives in version control, but data cannot leave the drive. The solution is project-relative paths that work from any authorized machine, not absolute paths tied to a specific user's drive mapping.

Institutional databases (SQL Server, Oracle, PostgreSQL) are increasingly common for high-volume surveillance data, vital records, and claims. Queries run on the server and return only results, so raw data never has to move. This is generally preferable to a shared drive for structured, regularly updated data. Appendix A. covers R's database tools (DBI and dbplyr) in more detail.

Secure remote desktops and data enclaves are used for the most sensitive data, including restricted-use vital records, Medicaid claims, and linked longitudinal datasets. Analysts connect via remote desktop to a server where the data lives. Copy-paste and file transfer are often disabled. Internet access may be restricted. The data never leaves the server; only approved outputs do, after passing a disclosure review. CDC's Research Data Center and many state-level restricted data programs work this way.

Air-gapped systems go further: no external network connection at all. These are rare but exist for certain federal and state datasets. Code and packages must be brought in through an approved intake process, and every output leaves through a formal review.

> Tip
>
> The more restrictive the environment, the more planning is required upfront. In a data enclave, you cannot install a package on a whim or look up documentation online. Test your code on a representative subset of data in a less restricted environment first, resolve all dependency questions, and then bring the finalized analysis into the enclave.

19.2 Data Use Agreements

A data use agreement (DUA) is a legal contract between the organization providing data and the organization receiving it. DUAs define what data is being shared, who is allowed to access it, what it can be used for, and what happens to it at the end of the project. They are routine in public health. Any time data is obtained from another agency, a federal program, a health system, or a research partner, expect a DUA.

19.2.1 What DUAs Cover

The most consequential terms are:

Permitted uses. DUAs typically specify the exact purpose for which the data was obtained. Using data for a secondary purpose (even one that seems obviously fine) may require an amendment. Read this section carefully before scoping the analysis.

Authorized users. The agreement usually names individuals or roles who may access the data. Adding a new team member or sharing data with a collaborator at another agency requires notifying the data provider and may require an amendment.

Data environment requirements. Many DUAs specify where data may be stored and processed, for example only on a designated secure server, only within the jurisdiction's network, or only in a HIPAA-compliant environment. "I'll just put it on my laptop for a quick look" is a DUA violation if the agreement prohibits local storage.

Dissemination restrictions. Some DUAs restrict publication (requiring advance review by the data provider), prohibit releasing record-level data, or limit which findings can be publicly reported. Know these terms before drafting a report or dashboard.

Data destruction. Most DUAs require that data be destroyed or returned at the end of the project period, and that destruction be documented. This includes copies, backups, and any data that was transferred for analysis.

19.2.2 Practical Guidance

Get a copy of the DUA before the data arrives. It is much easier to negotiate terms before signing than to discover a constraint mid-project. Loop in your supervisor and, if the terms are unfamiliar or restrictive, your agency's legal or compliance office. When in doubt about whether a specific use is permitted, ask the data provider in writing (email creates a record).

> ⚠ Warning
>
> Data received informally (as an email attachment, a Dropbox link, or a thumb drive) still carries the same governance obligations as data received through a formal DUA process. "We didn't sign anything" does not mean "there are no restrictions." Ask how the data can be used before you use it.

19.3 HIPAA and Protected Health Information

The Health Insurance Portability and Accountability Act (HIPAA) establishes federal standards for protecting the privacy and security of individually identifiable health information. This section is not a comprehensive HIPAA guide (your organization's privacy officer is the right resource for that), but public health data scientists need a working understanding of what HIPAA covers and how it applies to their work.

19.3.1 What HIPAA Covers

HIPAA applies to *covered entities* (healthcare providers, health plans, and healthcare clearinghouses) and their *business associates* (organizations that handle protected health information on their behalf). Many public health agencies are covered entities or business associates; many are not. The rules differ across jurisdictions, and some public health functions have explicit carve-outs under the HIPAA Privacy Rule (disease reporting, for example, is generally permitted without individual authorization).

Protected health information (PHI) is any individually identifiable health information held or transmitted by a covered entity. HIPAA defines 18 identifiers whose presence in a dataset makes that dataset PHI:

- Names
- Geographic subdivisions smaller than a state (including ZIP codes, street addresses, cities in some cases)
- All dates more specific than year (dates of service, birth dates, admission dates, death dates)
- Telephone numbers, fax numbers, email addresses
- Social Security numbers
- Medical record numbers, health plan beneficiary numbers, account numbers
- Certificate and license numbers

- Vehicle identifiers (license plates, VINs)
- Device identifiers and serial numbers
- URLs and IP addresses
- Biometric identifiers (fingerprints, voiceprints)
- Full-face photographs
- Any other unique identifying number or code

If a dataset contains any of these and relates to an individual's health, it is PHI.

19.3.2 The Minimum Necessary Standard

HIPAA's minimum necessary standard requires that covered entities use, disclose, or request only the minimum amount of PHI needed to accomplish the intended purpose. For a data scientist, this means requesting only the variables needed for the analysis, not pulling a complete medical record when you only need diagnosis codes and dates.

19.3.3 De-identification

Properly de-identified data is not PHI and is not subject to HIPAA restrictions. HIPAA provides two paths to de-identification:

Safe harbor: Remove all 18 identifiers listed above, plus ensure that the data provider has no actual knowledge that the remaining information could identify an individual. Geographic identifiers must be aggregated to the state level or to ZIP code prefixes with population greater than 20,000.

Expert determination: A qualified statistician applies generally accepted statistical principles to certify that the risk of identifying any individual is very small. This allows more flexibility than safe harbor (for example, three-digit ZIP codes or specific dates) but requires documented statistical analysis.

> **i** Note
>
> De-identification under HIPAA is a formal process, not a judgment call. Simply removing a name is not de-identification. If you are preparing data for release or for use in a less restricted environment, work with your privacy officer to ensure the de-identification is compliant.

19.4 Getting Data Access

Data access in public health institutions involves more parties than most analysts expect. A clear map of who needs to be involved, and what each party needs to hear, prevents the most common delays.

The data steward owns or is responsible for the dataset within the source organization. They can tell you what is in the data, how it is structured, its known quality issues, and what access process is required. For internal datasets (data your own agency collects), the steward is typically in the program that runs the surveillance system. For external datasets (from another state agency, a federal program, or a partner organization), the steward is at the providing organization.

IT controls the infrastructure: server access, database credentials, software installation, network permissions, and VPN. A request to IT should be specific: what software is needed (including version numbers), which database or server needs to be accessed, what level of compute and storage the analysis requires, and whether the data is sensitive or regulated.

Legal and compliance reviews DUAs and data sharing agreements. Loop them in early (DUA review can take weeks at some institutions) and provide the full draft agreement rather than a summary.

Your supervisor typically needs to sign DUAs and data sharing agreements on behalf of the agency. They should know about any data access requests that involve legal agreements before those agreements are presented for signature.

19.4.1 Framing Requests

A request framed in terms of program need is easier to prioritize than a vague technical ask. Compare:

"I need access to the vital records SQL Server database."

versus:

> *"This analysis supports the quarterly overdose mortality report due to the State Health Officer in March. I need read access to the `vr_deaths` table in the vital records database and the ability to install `odbc` and `DBI` in my R environment. The data is governed by our existing MOU with vital records. I need this by February 15 to have time to validate before the report deadline."*

The second version tells IT what the deadline is, what the legal basis is, and exactly what is needed. It also signals that someone outside IT will notice if the request is not addressed.

19.5 Working in Secure Data Environments

When data cannot leave a controlled environment, the analysis has to come to the data. Secure remote desktops and data enclaves impose constraints that require adapting the workflows described elsewhere in this book.

19.5.1 Common Constraints

- **No internet access**, or access limited to specific approved sites
- **No USB or removable media**, preventing local file transfer
- **Copy-paste disabled** between the remote session and the local machine
- **No administrator rights**, preventing ad-hoc software or package installation
- **Output review**, meaning no file can leave the environment until it passes a disclosure review

19.5.2 Adapting Your Workflow

Resolve package dependencies before entering the environment. Use `renv` (see Chapter 11.) to capture your project's exact package list. Submit the lockfile to IT for review; they can pre-install those packages in the secure environment, or arrange an approved intake path. Discovering mid-analysis that a package is unavailable is a significant delay.

Plan the analysis before entering. Write and test your code on a representative mock dataset (public-use data with similar structure, or synthetic data generated from the real data's structure) before bringing the code into the secure environment. The enclave is not the place to iterate on exploratory analysis.

Keep code in version control accessible from within the environment. Some enclaves have access to an internal Git server, or allow code to be transferred via an approved intake process. Maintain your code in a repository (see Chapter 1.) so it can be moved in as a single unit.

Document outputs before requesting export. Every output (tables, figures, and reports) that leaves the environment will be reviewed. Organize outputs clearly, understand the suppression rules that apply (see Section 19.6), and apply them yourself before submission. Self-review before the formal review reduces round trips.

19.6 Disclosure Review and Small Number Suppression

Even properly de-identified data can allow re-identification when cell counts are small. A table showing that a specific rural county had 2 opioid overdose deaths among people aged 25–34 may effectively identify those individuals in a small community. Disclosure review is the process of checking whether outputs could allow such re-identification before they are released.

19.6.1 When Disclosure Review Applies

Disclosure review is typically required for:

- Any output leaving a secure data environment
- Outputs derived from restricted-use data, regardless of environment
- Public-facing reports and dashboards based on individual-level surveillance data
- Analyses submitted for publication when the underlying data is restricted

Your DUA and your agency's data governance policies will specify what review process applies. For internally-collected surveillance data, the process is usually internal. For federally restricted data (NCHS, CMS), the providing agency conducts the review.

19.6.2 Small Number Suppression

The standard threshold in public health is to suppress cells where the count is fewer than 5. A count of 0, 1, 2, 3, or 4 is suppressed, typically displayed as an asterisk or "data not shown."

Complementary suppression is required when suppressing one cell would allow back-calculation of another. If a row shows a total and several cells, and only one cell is suppressed, the suppressed value can be calculated by subtraction. The fix is to suppress an additional cell (typically the next-smallest) so that the suppressed value cannot be derived. This often requires checking row and column totals as well as individual cells.

> **i** Note
>
> **Example:** A county reports 7 total influenza deaths. By age group: 25–34: 4 (suppressed, < 5), 35–44: 2 (suppressed), 45+: 1 (suppressed). The total (7) may still be releasable if it is not small, but all three age group cells must be suppressed. If only the first cell were suppressed, the other two would reveal the suppressed value.
>
> In practice, suppression logic can be complex for multi-dimensional tables. Review tools or explicit code (rather than manual review) reduce the risk of errors.

19.6.3 What Disclosure Review Examines

A reviewer will typically check:

- All cells meet the minimum count threshold
- Complementary suppression has been applied correctly
- Geographic or demographic combinations that are highly specific (a single rural county × a specific age group × a specific cause) do not expose individuals even if each cell alone is above the threshold
- Percentages, rates, and proportions are checked: a rate computed from a suppressed count is itself suppressed

When reporting suppressed data to a lay audience, explain why: "Data suppressed to protect individual privacy" is preferable to an unexplained asterisk or blank cell.

20. Peer Review for Data Analysis

Analysis code is not the same as production software, but the reasons for reviewing it overlap substantially. Errors in analytical code produce wrong results. Wrong results inform wrong decisions. In public health, wrong decisions have consequences. Peer review is the primary mechanism for catching errors before they reach a decision-maker, and for building a team culture where good work is a shared responsibility rather than a solo performance.

This chapter covers what analytical peer review looks like in practice, how to structure a review, and the tools that make it easier — including AI as a review aid.

20.1 What Analytical Peer Review Is (and Is Not)

Peer review in this context means having someone other than the original analyst read and evaluate the analysis before its results are communicated. This is different from:

- **Software code review**, which focuses on correctness, security, and maintainability of code that runs in production. Analytical code has different concerns: Does the question match the data? Are the methods appropriate? Are the results interpreted correctly?
- **Statistical peer review** in the academic sense, where a journal reviewer evaluates methodological rigor. The bar here is lower and the turnaround is much faster — the goal is to catch errors, not to grant imprimatur.
- **Proofreading a report**, which catches writing problems but misses analytical ones. A clearly written report can present a wrong answer clearly.

A good analytical peer review evaluates three things:

1. **Correctness of the code**: Does the code do what the analyst says it does? Are there off-by-one errors, filter inversions, or joins that silently drop records?
2. **Appropriateness of the methods**: Is the analysis designed to answer the question being asked? Are there obvious alternative interpretations the analyst did not consider?
3. **Validity of the conclusions**: Do the stated findings follow from the results? Are uncertainty and limitations communicated accurately?

None of these require the reviewer to rerun the analysis from scratch, though doing so is valuable when stakes are high.

20.2 Code Review Mechanics

The practical vehicle for analytical peer review is a pull request (see Section 1.2). When an analyst finishes a piece of work, they open a pull request from their branch to the main branch. The reviewer reads the diff, leaves comments, and either approves or requests changes before the code is merged.

This workflow has several advantages over emailing scripts back and forth or reviewing a report after the fact:

- The review is tied to the specific changes that were made, not a reconstruction from memory.
- Comments are attached to specific lines of code, making feedback precise.
- The history of the review (what was flagged, what was addressed) is preserved alongside the code.
- Analysis results are not incorporated into a deliverable until the review is complete.

20.2.1 What to Look for in a Code Review

When reviewing analytical code, focus on:

Data handling

- Are filters applied correctly? A common error is filtering to the wrong values (e.g., keeping `status != "active"` when the intent was to keep only active records).

- Are joins producing the expected number of rows? Silent duplications or dropped records from a many-to-many join or a misspecified key are among the most consequential errors in analytical work.
- Is missing data handled explicitly, or silently dropped? If dropped, is that appropriate?
- Are date ranges and cutoffs correct? Off-by-one errors on dates are easy to introduce and easy to miss.

Calculations

- Do aggregations (sums, means, rates) match what is described in the analysis?
- Are rates calculated with the correct denominators? Are numerator and denominator from the same population?
- Are any constants hard-coded where they should be calculated from the data?

Outputs

- Do the summary statistics match reasonable expectations for the data? A mean that seems implausibly high or low is worth investigating.
- Do the plots match what the code is computing? A chart labeled "rate per 100,000" should be computing and displaying a per-100,000 rate.
- Are the conclusions stated in the narrative consistent with the numbers in the output?

20.2.2 Giving and Receiving Feedback

A review is useful only if feedback is acted on, and feedback is acted on only if it is received well. A few principles that help:

For reviewers: Be specific about what you are flagging and why. "This join might be producing duplicates: there are multiple rows per `case_id` in `contact_table`, which means a left join on `case_id` will multiply the case rows. Can you verify the row count before and after?" is more actionable than "check the join."

For analysts: Treat a review comment as a question, not a criticism. The reviewer may be wrong; if so, a short explanation resolves it. But the reviewer may have caught something real and the fact that the analysis worked correctly in a different case does not mean it works correctly in this one.

For teams: Normalize review as a default, not an exception reserved for high-stakes work. A team where review is standard for all analysis will catch errors earlier and distribute knowledge more evenly than one where review is triggered only by anxiety.

20.3 Checklists for Analytical Review

Checklists help reviewers maintain consistency and avoid the bias of reading the analysis looking to confirm that it is correct. Reviewing analytically requires some skepticism, reading the code as if looking for errors rather than checking off steps.

A basic checklist for most analytical peer reviews:

Data

- ☐ The data source is correctly identified and loaded
- ☐ The date range and population scope match the stated analysis parameters
- ☐ Filters are correctly specified and applied in the right order
- ☐ Missing values are accounted for
- ☐ Join keys are unique (or duplicates are intentional and handled)
- ☐ Row counts at key steps are plausible

Analysis

- ☐ Numerators and denominators are correctly specified for any rates
- ☐ Grouping variables match the level of the desired analysis
- ☐ Any statistical tests or models use methods appropriate for the data and question
- ☐ Results match expectations from prior knowledge of the data

Outputs

- ☐ Chart labels, titles, and units are correct
- ☐ Summary tables match what the code computes
- ☐ Stated conclusions follow from the results
- ☐ Uncertainty and limitations are acknowledged appropriately

Not every item applies to every analysis. For a simple descriptive summary, several of the statistical items are irrelevant. For a complex linked dataset with multiple joins and rates, the data section deserves more attention. Use judgment.

20.4 AI as a Review Aid

AI coding tools (see Chapter 6.) can supplement but not replace human peer review. They are useful for certain review tasks and unreliable for others.

Where AI tools add value in review:

- Explaining what a section of code does, in plain language, so a reviewer can quickly verify that the code matches the analyst's stated intent
- Identifying common patterns that are likely to be errors (filter inversions, joins without explicit key specification, implicit coercions)
- Checking that variable names, column references, and function arguments are used consistently
- Generating a summary of what changed between versions of a script

Where AI tools are not reliable:

- Verifying that the analytical methods are appropriate for the question (this requires domain knowledge)
- Detecting errors that require understanding the real-world data (e.g., knowing that a particular jurisdiction stopped reporting in 2021, so a sudden drop is a data artifact rather than a true change)
- Evaluating whether conclusions are well-supported (this requires both analytical judgment and knowledge of the context)

A practical workflow: use the Positron Assistant (see Section 6.2) or Claude Code (see Section 6.4) to review a script for mechanical issues (does the logic match the description, are there obvious error patterns) and then have a human reviewer focus on whether the analysis is asking the right question and drawing the right conclusions from the results.

> **i** Note
>
> AI tools will sometimes confidently identify a "problem" that is not actually a problem. Any AI-generated review comment should be treated as a starting point for investigation, not a definitive finding. The human reviewer is responsible for the review.

20.5 High-Stakes Analyses

Routine analyses, i.e. a weekly case count report, a descriptive summary for an internal meeting, benefit from light-touch review. Analyses that inform significant decisions warrant more thorough review.

Indicators that an analysis warrants a higher review standard:

- The results will be used to allocate resources or change programs
- The results will be released publicly or to the press
- The analysis involves a method the team has not used before on this data
- The analysis contradicts prior findings and there is no obvious explanation

For high-stakes work, consider independent replication: a second analyst replicates the key computations independently and compares results. Discrepancies must be resolved before the analysis is finalized. This is more expensive than a standard review but is warranted when errors would have significant consequences.

20.6 Building a Review Culture

Peer review works best when it is a team norm, not a gatekeeping mechanism. Teams that review well share a few characteristics:

Review is not optional for consequential work. There is a clear, shared expectation that analyses above a certain threshold of importance are reviewed before results are communicated.

Review is timely. A review request that sits for two weeks creates pressure to skip it. Teams that review quickly, i.e. same or next business day for most requests, make review practical rather than theoretical.

Reviewers are recognized for good reviews. Catching an error that would have been embarrassing is a contribution. Teams that treat review as invisible overhead underinvest in it.

Review is a learning mechanism. The goal is not to police quality but to improve it. Analysts who receive good reviews get better. Reviewers who read a lot of others' code develop a broader perspective on the team's methods and practices.

21. Communicating Findings

A technically correct analysis that does not reach its audience (or reaches them in a form they cannot act on) has not done its job. In public health, findings inform decisions made by program directors, health officers, elected officials, and practitioners who did not run the analysis and may not read a methods section. Getting from rigorous analysis to useful communication is its own skill, and one that is easy to underinvest in when the pressure is on to finish the analysis itself.

Section 18.5 introduces the core principles: know your audience, communicate uncertainty honestly, match the output format to the need. This chapter goes deeper on each, with concrete guidance on structure, visualization, and how to handle the cases that are hardest to communicate well.

21.1 Start with the Bottom Line

The most common failure mode in communicating analysis results is burying the finding. Reports structured like academic papers (Introduction, Methods, Results, Discussion) force busy decision-makers to read through to the end to learn what the analysis concluded. Many do not make it that far.

The fix is to lead with the finding, not with the context. State the bottom line first. Then provide the evidence that supports it.

Compare these two openings for the same analysis:

> *Version A:* "Overdose mortality in the state has been a growing public health concern. This analysis used death certificate data from the National Vital Statistics System for 2015–2023 to examine trends in overdose deaths by county and age group. The results showed that rates in rural counties increased substantially."

> *Version B:* "Overdose death rates in rural counties increased 34% from 2019 to 2023, compared to a 22% increase in urban counties, a reversal from the 2015–2019 period, when urban rates were higher. The shift is driven primarily by illicitly manufactured fentanyl, which first appeared in rural county toxicology reports at scale in 2020."

Version B leads with what changed, by how much, and what is driving it. A health officer who reads only the first two sentences of Version B has the main finding. Version A makes them work for it.

21.1.1 The "So What" Test

After stating a finding, ask: *So what?* What should a program manager, health officer, or policymaker consider doing differently because of this finding? The analysis team often knows the answer but leaves it unstated, assuming the audience will draw the right implication. They often do not.

A finding without an implication is incomplete:

> *Finding only:* "Rural overdose death rates increased 34%."

> *Finding with implication:* "Rural overdose death rates increased 34%, and rural counties currently have significantly less harm reduction infrastructure per capita than urban counties, suggesting that expanding naloxone distribution and syringe services to rural areas is a priority intervention point."

This does not mean advocating for a specific policy. It means stating what the data implies for programmatic decisions. The program can decide what to do with that implication; the analyst's job is to make it explicit.

21.2 Audience and Format

Different stakeholders need different outputs from the same underlying analysis. Plan to produce more than one. The analysis runs once; the communication takes multiple forms.

21.2.1 Decision-Makers

Health officers, board members, program directors, and elected officials typically need:

- The bottom line first (what is the finding and what does it imply?)
- One or two key numbers, not a table of 40 statistics
- Uncertainty stated plainly in language they can repeat to others
- A clear sense of what decision or consideration the finding should inform
- One page, or one slide

They do not need methods details unless they specifically ask. If the methods are rigorous, say so briefly ("These findings are based on complete death certificate data; preliminary estimates from provisional data are consistent with these results") and move on.

21.2.2 Program Staff

Staff doing day-to-day work (case investigators, disease intervention specialists, program coordinators) need findings they can act on. The most useful outputs for this audience:

- Are specific enough to change what they do tomorrow
- Include enough supporting data that they can answer follow-up questions from the people they work with
- Are formatted for reuse in their own presentations and communications

A table they can sort, a chart they can copy into a slide, or a brief they can forward to a partner agency is often more valuable than a polished report they cannot extract from.

21.2.3 Technical Peers

Epidemiologists, biostatisticians, and methodologically sophisticated collaborators need the full picture: data sources, methods, sensitivity analyses, limitations, and honest uncertainty. For this audience, the concerns are whether the analysis was done correctly and whether the conclusions are defensible, not whether the one-page summary is clear.

When sharing work with technical peers for review (see Chapter 1. for making code available), include the analysis code alongside the report. A methods section that says "code available at [repository]" is more credible than one that does not.

21.2.4 An Executive Summary Template

For findings that need to reach multiple audiences, a structured one-page executive summary gives decision-makers what they need while pointing technical readers to the full report. A minimal template:

```
SUMMARY

[One to two sentences: the main finding and its most important implication.]

KEY FINDINGS

- [Finding 1, stated plainly, with the key number]
- [Finding 2]
- [Finding 3]

IMPLICATIONS

[One to two sentences on what the findings suggest for programs, policy,
or practice. Frame as considerations, not mandates.]
```

```
DATA AND METHODS

[One sentence: data source, time period, geographic scope, population.]
Full methods available at [link or report title].
```

The summary fits on one page. The full report follows. Decision-makers read the summary; technical reviewers read both.

21.3 Choosing the Right Output

The analysis determines what can be said; the output format determines whether it is heard. A well-chosen format reduces friction between the finding and the decision-maker.

Situation	Best format
One-time finding for a decision	Written brief or slide deck
Ongoing data that updates regularly	Parameterized report (Chapter 4.) or dashboard (Chapter 7.)
Data that stakeholders need to explore	Interactive dashboard
Findings for public release	Report with executive summary
Technical findings for peer review	Full report with methods appendix and code
Data that will be used in others' presentations	Table or chart files, clearly labeled

A few questions that sharpen the choice:

Will the audience consume a summary or explore the data? If the finding can be stated in a few sentences and a chart, a brief is right. If stakeholders will want to slice by county, time period, or demographic group, a dashboard lets them do that without requiring a new analysis each time.

Is this a one-time analysis or will it recur? If the analysis will be repeated monthly or annually with updated data, building a reproducible, parameterized report (see Chapter 4.) costs more upfront and dramatically less over time. If it is truly one-time, a polished static output is fine.

Will this be printed? HTML dashboards and interactive reports do not print well. If physical copies or PDFs are a requirement, design for that from the start.

21.4 Visualizing to Communicate

Visualization for communication is different from visualization for exploration. Exploratory charts help you understand data; communication charts help an audience understand a specific finding. The design goal shifts from "show me what is in this data" to "make this one finding impossible to miss."

21.4.1 Match Chart Type to Message

Message	Chart type
Compare values across categories	Horizontal bar chart (especially with many categories)
Show change over time	Line chart
Show a part of a whole	Stacked bar, waffle chart; avoid pie charts with more than 3 slices
Show a relationship between two variables	Scatter plot
Show geographic distribution	Choropleth map (with caution, since land area ≠ population)
Show a single number	Large text, value box

No chart type is universally wrong, but some common combinations fail reliably:

- **Pie charts with many slices:** humans cannot judge angles accurately; a bar chart is almost always clearer
- **Dual y-axes:** charts with two y-axes are nearly always misleading because the visual relationship between the two lines depends entirely on how the axes are scaled, which the designer chooses
- **3D charts:** the added dimension carries no data but introduces distortion; never use them
- **Truncated y-axes:** starting a bar chart's y-axis above zero exaggerates differences; if differences are small, say so in text rather than making them look large on a chart

21.4.2 Design for the Finding, Not the Data

A communication chart has one job: make the finding clear. Every element that does not support that job should be removed.

Highlight what matters. Use color, annotations, or callout lines to draw attention to the specific element that supports the finding. A line chart showing trends for 50 states, all in gray, with a single state highlighted in the primary color, is far more effective than 50 differently-colored lines competing for attention.

Label directly. Annotations on the chart (for example, "34% increase" at the relevant data point, or a text label at the end of each line) are clearer than legends the reader has to match to the chart. If the chart requires a legend to interpret, consider whether direct labeling is possible.

Use accessible colors. Approximately 8% of men have color vision deficiency. Red/green combinations are the most common problem. Colorblind-friendly palettes (such as the Okabe-Ito palette used by default in many ggplot2 themes) are safe choices. Check any palette at a tool like Coblis before finalizing.

Remove decoration that carries no information. Heavy gridlines, background colors, borders, and 3D effects subtract from clarity. The data should be the most visually prominent element on the chart.

21.5 Communicating Uncertainty

Every analysis involves uncertainty, and communicating it honestly, without either overstating it into uselessness or understating it into false precision, is one of the harder craft challenges in public health communication. Section 18.5 introduces this; what follows is more specific guidance.

21.5.1 Natural Language Over Statistical Formalism

Decision-makers who did not take a statistics course will not correctly interpret "95% confidence interval." Many will read a CI as a margin of error or, worse, as the probability that the true value falls in the range. Natural language is clearer:

> *Avoid:* "The estimated increase was 34.2% (95% CI: 28.1%–40.3%)."
>
> *Prefer:* "We estimate the increase was between 28% and 40%, with our best estimate at 34%."
>
> *Or:* "We estimate the increase was roughly 34%, though the true figure could reasonably be as low as 28% or as high as 40%."

When explaining what drives the uncertainty, be specific: "The estimate is imprecise because few deaths occurred in this county" is more useful than "the confidence interval is wide" or "p = 0.07."

21.5.2 Null Results Are Findings

An analysis that finds no statistically significant difference has found something: the absence of a detectable effect with the available data. This is not a failure; it is a result, and it should be communicated as one.

> *Weak:* "There was no statistically significant difference in rates between the two groups (p = 0.23)."

Stronger: "We found no evidence of a meaningful difference in rates between the two groups. With the available data, we would have been able to detect a difference of 15 percentage points or larger; smaller differences, if they exist, are not detectable with this dataset."

The second version tells the reader what the analysis could and could not detect, which helps them calibrate what further inquiry is warranted.

21.5.3 Preliminary and Provisional Data

Public health data is often released in provisional form before it is final. Death certificate data typically takes 12–24 months to complete as late reports arrive and codes are finalized. When using provisional data, say so explicitly:

"These counts are based on provisional data as of [date]. Counts are typically revised upward by 5–10% as late reports arrive; the trend is reliable even if specific numbers will change."

Telling the audience the direction and magnitude of likely revision is more useful than a generic caveat.

21.5.4 Data Quality Problems

When data has known quality issues (a reporting system that changed, a year with underreporting, a jurisdiction that did not report), be transparent about them without letting them swallow the finding.

A pattern that holds despite a known data quality problem is stronger evidence than one that depends on a single year or jurisdiction. Point that out explicitly:

"Reporting completeness improved in 2021 after the new case management system was deployed, which contributes to the apparent increase that year. The trend from 2022 onward, when reporting was stable, shows a continued increase of approximately 8% per year."

21.5.5 Suppression

When cells are suppressed due to small counts (see Section 19.6), explain why in plain language rather than leaving an unexplained asterisk:

"Data for counties with fewer than 5 deaths in a given year are not shown to protect individual privacy."

If the suppression is consequential (meaning the reader might draw incorrect conclusions from the visible cells because suppressed cells would change the picture), say so:

"Several small rural counties are not shown due to small counts, but their inclusion would not change the overall trend."

Or, if suppression does change the picture:

"Data for several small counties are suppressed. Statewide totals include these counties; county-level rates for the suppressed jurisdictions are not available."

Appendices

References

A. Additional Resources

This appendix points to external resources for topics that go deeper than what this book covers. The selections here are oriented toward public health practitioners building data science skills in R, with emphasis on free and openly available material.

A.1 Learning R

R for Data Science (Wickham, Çetinkaya-Rundel, and Grolemund) is the standard starting point for learning the tidyverse. It covers data import, transformation with dplyr, visualization with ggplot2, and the broader workflow of doing data analysis in R. The second edition is freely available online. Most DSTT participants will benefit from working through at least the first half.

What They Forgot to Teach You About R (Bryan and Hester) is not about R syntax — it is about working in R effectively: project-oriented workflows, path discipline, R startup files, and the habits that separate reproducible work from fragile one-off scripts. Short and practical.

The tidyverse style guide documents naming conventions, spacing, and code organization conventions used across the tidyverse. Following a consistent style makes code easier to read, review, and maintain. The `styler` package can apply many of these rules automatically.

A.2 Going Deeper in R

Advanced R (Wickham) covers R's object systems, functional programming, metaprogramming, and performance. It is aimed at people who already use R regularly and want to understand why things work the way they do, write more expressive code, and debug problems that go beyond "my package isn't loading."

R Packages (Wickham and Bryan) is the definitive guide to writing R packages. A package is the right container for analysis code that needs to be shared across projects or team members — it bundles functions, documentation, and tests in a standardized way that makes everything easier to reuse. Even teams that do not plan to publish to CRAN benefit from the structure packages impose. This book walks through the complete workflow using `devtools` and `usethis`.

Mastering Shiny (Wickham) covers building interactive web applications with Shiny. Shiny apps require a running R process (unlike Quarto dashboards) but support arbitrary interactivity — user-driven filtering, on-the-fly modeling, custom inputs. This book covers reactive programming, app architecture, and deployment.

A.3 Visualization

ggplot2: Elegant Graphics for Data Analysis (Wickham, Navarro, and Pedersen) goes beyond the basics of `geom_*` functions to cover the grammar of graphics underlying ggplot2, how to extend it, and how to build custom themes and scales. Useful for teams that produce a lot of publication-quality figures and want consistent, polished output.

R Graphics Cookbook (Chang) is organized as a collection of recipes: a problem followed by a solution. Less conceptual than the ggplot2 book, more useful as a quick reference when you know what you want a plot to look like but not how to produce it.

A.4 Modeling and Statistics

Tidy Modeling with R (Kuhn and Silge) is the companion book to the tidymodels ecosystem, a collection of packages that bring tidyverse conventions to statistical modeling: splitting data, specifying model workflows, tuning hyperparameters, and evaluating performance. Tidymodels works with hundreds of model types through a consistent interface. The book and the tidymodels website are the primary resources for teams moving from exploratory analysis into predictive modeling.

A.5 Public Health Data Science

The Epidemiologist R Handbook is a comprehensive, freely available reference manual written specifically for epidemiologists and public health practitioners using R. It covers data management, descriptive analyses, outbreak investigation, time series, spatial analysis, and reporting – all with worked examples using realistic public health data. For DSTT teams, this is the most directly applicable reference for day-to-day analytical work.

Building Reproducible Analytical Pipelines with R (Rodrigues) covers functional programming, package-based project structure, Docker, and Nix as a complete framework for building analyses that can be re-run reliably. It connects topics from the environments chapter (Chapter 11.) and goes substantially deeper on each.

A.6 Databases from R

Most public health data lives in relational databases – SQL Server, PostgreSQL, Oracle, SQLite – not in CSV files on a shared drive. R can query these databases directly, returning results as data frames without any manual export step.

Posit's database guide is the best starting point. It covers the overall architecture (DBI as the connection layer, specific driver packages per database type), connection setup in Positron and RStudio, and best practices for credentials and connection management.

DBI is the foundational package. It defines the interface that all R database drivers implement: `dbConnect()` to open a connection, `dbGetQuery()` to run a SQL query and return results, `dbDisconnect()` to close the connection. If you know the SQL you want to run, DBI is the direct path.

dbplyr extends dplyr to work against database tables. Instead of writing SQL, you write normal dplyr code — `filter()`, `mutate()`, `summarize()` — and dbplyr translates it to SQL and runs it on the database. This is useful when the team knows dplyr well and the database queries are not highly complex. `collect()` pulls the results into R as a local data frame.

A.7 APIs and Public Data

httr2 is the package for talking to web APIs from R, introduced in Chapter 13.. Beyond the basics covered there, its documentation includes a "Wrapping APIs" vignette that walks through building a reusable interface to an API, useful if your team pulls from the same source in many projects.

Analyzing US Census Data (Walker) [8] is the definitive guide to working with census data in R via the `tidycensus` package: ACS and decennial data, geography and mapping, and the margins-of-error handling that census estimates require. Freely available online. For public health teams, this is where rate denominators come from.

The Socrata developer documentation covers the API behind data.cdc.gov and many state open data portals, including the SoQL query language for filtering and aggregating on the server.

A.8 Migrating from Legacy Tools

R for Excel Users (Lowndes and Horst) is a free, workshop-format course aimed at analysts whose current workflow is Excel, covering the transition to R, RStudio/Positron, and reproducible project habits. A good group-study resource for teams working through the migration described in Chapter 14..

haven reads SAS, SPSS, and Stata files into R, preserving value labels and special missing values. Its documentation covers the details of labelled data that matter when converting legacy datasets to open formats.

A.9 Online Courses

The **Johns Hopkins Data Science Specialization** on Coursera is a 10-course sequence by Roger Peng, Jeff Leek, and Brian Caffo covering the complete applied data science workflow in R: tooling, data acquisition, exploratory analysis, reproducible research, statistical inference, regression, machine learning, and building data products. Individual courses can be audited free. The most standalone-useful courses for public health practitioners are:

- **R Programming**: the core language, data structures, control flow, and functions. A solid foundation if you are new to R or coming from another language.

- **Exploratory Data Analysis**: visualization and summarization as an investigative tool, not a reporting tool.
- **Reproducible Research**: the rationale and practice of literate programming in R, closely aligned with the approach in Chapter 4..

The **Johns Hopkins Executive Data Science Specialization** (also by Peng, Leek, and Caffo) is a shorter, less technical sequence aimed at managers who lead or commission data science work. It pairs well with Chapter 18. and is useful for team leads who want to communicate more effectively with analysts.

A.10 Version Control

Happy Git and GitHub for the useR (Bryan) is the companion reference to Chapter 1.. It covers installation and configuration in detail, the full range of workflows for collaborating on GitHub, and a substantial troubleshooting section for the inevitable moments when Git does something unexpected. Required reading for anyone who wants to move past the basics.

A.11 Quarto

The **Quarto documentation** at quarto.org is comprehensive and well-maintained. Key sections beyond what this book covers:

- **Quarto Projects** — organizing multiple documents with shared configuration, a prerequisite for generating many parameterized reports at once.
- **GitHub Actions for Quarto** — automating rendering and publishing on push, so reports update without manual intervention.
- **Quarto Extensions** — community-contributed output formats, shortcodes, and filters that extend what Quarto can produce.

Bibliography

[1] K. W. Broman and K. H. Woo, "Data Organization in Spreadsheets," *The American Statistician*, vol. 72, no. 1, pp. 2–10, 2018, doi: 10.1080/00031305.2017.1375989.

[2] H. Wickham, "Tidy Data," *Journal of Statistical Software*, vol. 59, no. 10, pp. 1–23, 2014, doi: 10.18637/jss.v059.i10.

[3] J. Bryan, "How to Name Files." [Online]. Available: https://github.com/jennybc/how-to-name-files

[4] W. M. Landau, "targets: Dynamic Function-Oriented 'Make'-Like Declarative Pipelines." 2021. [Online]. Available: https://docs.ropensci.org/targets/

[5] H. Wickham, "conflicted: An Alternative Conflict Resolution Strategy." 2023. [Online]. Available: https://conflicted.r-lib.org/

[6] H. Wickham and J. Bryan, *R Packages*, 2nd ed. O'Reilly Media, 2023. [Online]. Available: https://r-pkgs.org/

[7] V. P. Nagraj and S. D. Turner, "Pracpac: Practical R Packaging with Docker," no. arXiv:2303.07876. arXiv, Mar. 2023. doi: 10.48550/arXiv.2303.07876.

[8] K. Walker, *Analyzing US Census Data: Methods, Maps, and Models in R*. CRC Press, 2023. [Online]. Available: https://walker-data.com/census-r/

[9] R. D. Peng, B. Caffo, and J. T. Leek, *Executive Data Science*. Leanpub, 2015. [Online]. Available: https://leanpub.com/eds

[10] E. Robinson and J. Nowling, *Build a Career in Data Science*. Manning Publications, 2020. [Online]. Available: https://www.manning.com/books/build-a-career-in-data-science

[11] R. D. Peng and E. Matsui, *The Art of Data Science*. Leanpub, 2015. [Online]. Available: https://leanpub.com/artofdatascience